LDACS1310

Introduction to Applied Cryptography

Course notes — Chapters 1–4

Course taught by Thomas Peters and
François-Xavier Standaert

Working draft — Chapter 4 awaiting review.
Definitions, concise explanations, proofs, and illustrative examples accompany the original visual diagrams. Later chapters will be developed sequentially after review.

Updated:

October 18, 2026

Contents

How to use these notes **v**

1 Introduction and foundations of cryptographic security **1**

1.1 Information security and cryptography 1

1.2 Secure communication and adversarial models 2

1.3 Encryption: syntax before security 2

1.3.1 Notation 2

1.4 Integrity: MACs and signatures 3

1.5 The one-time pad and perfect secrecy 4

1.5.1 Bitwise exclusive OR 4

1.5.2 What exactly does the ciphertext reveal? 4

1.5.3 Individually fair bits are not enough 5

1.5.4 Reusing the key reveals relations 6

1.6 Computational security and exhaustive search 6

1.6.1 A hypothetical 128-bit key search 6

1.6.2 Security changes over the lifetime of data 7

1.7 Cryptographic methodology and reductions 7

1.7.1 Define, design, and justify 7

1.7.2 Account for the cost of the reduction 8

1.8 Concrete security and security experiments 8

1.8.1 Winning probability versus distinguishing advantage 9

1.9 Asymptotic security and negligible functions 10

1.9.1 What grows with the parameter? 10

1.10 Course scope and how to study the constructions 10

1.11 Review questions 11

2 Symmetric encryption **12**

2.1 A dictionary of pseudorandom objects 12

2.1.1 Real and ideal distributions 12

2.1.2 Pseudorandom generators 12

2.1.3 Pseudorandom functions and permutations . . . 13
2.2 Single-message eavesdropper security . . . 14
2.2.1 Emulating the one-time pad with a PRG . . . 15
2.2.2 Concrete cost and limitations . . . 16
2.3 Chosen-plaintext security . . . 17
2.3.1 Why deterministic encryption fails here . . . 18
2.3.2 Visible structure: independent block encryption . . . 18
2.4 Cipher block chaining . . . 19
2.4.1 Notation, encryption, and decryption . . . 19
2.4.2 The concrete security bound . . . 21
2.4.3 Why CBC needs suitable initialisation . . . 22
2.5 Output feedback and repeated IVs . . . 22
2.6 Message padding . . . 23
2.6.1 The PKCS #7 rule . . . 23
2.7 Padding oracles and active attacks . . . 24
2.7.1 The information exposed by a validity bit . . . 24
2.7.2 Recovering the last bytes . . . 25
2.7.3 Work factor and security meaning . . . 26
2.7.4 Authentication before decryption-dependent processing . . . 26
2.8 Review questions . . . 27

3 Block ciphers: construction and cryptanalysis 28
3.1 From functions to permutations: Feistel networks . . . 28
3.1.1 Inverting a construction without inverting its function . . . 28
3.1.2 Security and the cost of the reduction . . . 29
3.1.3 Using a permutation as a function . . . 30
3.2 From generators to functions, and back . . . 30
3.2.1 A binary tree of pseudorandom expansions . . . 30
3.2.2 Expanding one extra bit into a longer output . . . 32
3.2.3 The efficiency challenge . . . 33
3.3 Why easy descriptions need careful design . . . 33
3.3.1 A random table is too large . . . 33
3.3.2 A linear or affine map is too transparent . . . 34
3.4 Key alternation, confusion, and diffusion . . . 34
3.5 Algebraic cryptanalysis . . . 35
3.5.1 Turning observations into equations . . . 35
3.5.2 Degree growth and the number of monomials . . . 36
3.6 Linear cryptanalysis: finding a statistical imbalance . . . 38

3.6.1 Approximations and bias . . . 38
3.6.2 Using the bias to rank key guesses . . . 39
3.7 A worked SPN: from an S-box table to a linear key ranking . . . 41
3.7.1 Fixing the cipher and the indexing . . . 41
3.7.2 Computing a linear approximation table . . . 41
3.7.3 Cancelling intermediate parities across three rounds . . . 43
3.7.4 Guess only the final-key bytes the relation needs . . . 44
3.8 Differential cryptanalysis: following chosen XOR differences . . . 45
3.8.1 What changes between two encryptions? . . . 45
3.8.2 Counting an S-box's difference distribution . . . 45
3.8.3 A three-round difference trail . . . 46
3.8.4 Filtering and partially decrypting chosen pairs . . . 46
3.8.5 Data requirements and the difference between a trail and a differential . . . 48
3.9 Making useful approximations hard to obtain . . . 49
3.9.1 Active S-boxes and diffusion . . . 49
3.9.2 A trail versus a linear hull . . . 50
3.10 AES as a concrete design . . . 51
3.11 Quantum search and the composition challenge . . . 52
3.11.1 What the square-root speedup means . . . 52
3.11.2 Why double encryption does not double classical strength . . . 52
3.12 Review questions . . . 53

4 Message authentication **55**
4.1 What must the receiver be able to check? . . . 55
4.2 Why checksums and short tags fail . . . 56
4.2.1 Detecting accidents does not stop an adversary . . . 56
4.2.2 The tag length is a separate security parameter . . . 56
4.3 Unforgeability as an experiment . . . 57
4.3.1 What counts as a forgery? . . . 57
4.3.2 A simple way to connect MACs to pseudorandomness . . . 58
4.4 CBC-MAC: what the chain authenticates . . . 58
4.4.1 Raw construction and its fixed-length scope . . . 58
4.4.2 A tag becomes a continuation state . . . 59
4.5 Marking message boundaries securely . . . 60
4.5.1 Separating the final operation . . . 60
4.5.2 Putting the length first . . . 61
4.5.3 Padding must preserve the identity of the message . . . 62
4.6 Reading a concrete unforgeability bound . . . 63

4.7 Protocol state and implementation behaviour . . . 63
4.7.1 An authentic message can still be a replay . . . 63
4.7.2 Early rejection can leak a correct prefix . . . 63
4.8 Collision-resistant hashing . . . 64
4.8.1 Public fingerprints and their security task . . . 64
4.8.2 Building compression from a block cipher . . . 64
4.9 Sponge hashing and SHA-3 . . . 65
4.9.1 Where the message enters and the output leaves . . . 65
4.9.2 Rate, capacity, and output length . . . 66
4.10 Combining confidentiality and authentication . . . 66
4.10.1 Three arrangements, three information flows . . . 66
4.11 Concrete constructions and how to recognise their roles . . . 68
4.11.1 Counter mode provides encryption . . . 68
4.11.2 CMAC provides block-cipher-based authentication . . . 68
4.11.3 HMAC provides hash-based authentication . . . 69
4.11.4 Authenticated-encryption schemes combine the roles . . . 69
4.12 Review questions . . . 70

A Finite fields and the algebra behind AES diffusion **71**
A.1 Groups, rings, and fields: which operations can be undone? . . . 71
A.2 Finite fields, characteristic, and coordinates . . . 72
A.3 Representing field elements by polynomials . . . 72
A.3.1 Bit strings as formal coefficients . . . 72
A.3.2 Reduction and why irreducibility matters . . . 73
A.4 AES byte arithmetic, calculated step by step . . . 73
A.4.1 The reduction polynomial and multiplication by two . . . 73
A.4.2 A complete product . . . 74
A.4.3 Computing inverses . . . 74
A.5 Four-byte columns: an invertible map in a ring . . . 75
A.5.1 A second polynomial variable . . . 75
A.5.2 Deriving the MixColumns matrix . . . 75
A.6 Linear codes and the meaning of an optimal branch number . . . 76
A.6.1 Weight, distance, and matrix descriptions . . . 76
A.6.2 Singleton bound and MDS codes . . . 76
A.6.3 The graph of a mixing layer is a code . . . 76
A.7 Review questions . . . 77

How to use these notes

This syllabus is a separate workspace for Applied Cryptography. Chapter 1 follows the introductory lecture, from the goals of information security to formal ways of stating cryptographic security. Chapter 2 develops symmetric encryption, pseudorandom primitives, security reductions, block-cipher modes, and padding-oracle attacks.

Definitions, examples, remarks, and results share a chapter-based counter. Algorithms have numbered lines and their own chapter-based numbering. Each mathematical symbol is introduced before use. Probability expressions use explicit notation such as $P(A \mid B)$ rather than unnecessary substitutions.

The graphical figures reproduce the actual diagrams from the lecture; tables, formulas, and algorithms are typeset in LaTeX. Explanations and examples are integrated directly into the text: they connect formal statements to what the parties and adversary know, show why each construction works, and state the scope of its guarantee. Figure captions identify the object; the surrounding text walks through its data flow or reads its axes, then links it to the formulas. Small calculations make the key transitions explicit. Read these alongside the definitions and proofs.

Progress: Chapters 1–3 are available with detailed figure readings and explanations. Chapter 4 develops message authentication, CBC-MAC, hashing and authenticated-encryption composition, and is awaiting review. Chapter 3 now includes worked linear and differential attacks; Appendix A develops finite-field arithmetic and the coding-theory basis of AES diffusion.

Chapter 1

Introduction and foundations of cryptographic security

Cryptography studies mechanisms and mathematical guarantees for protecting information against adversaries. The first step is to identify the intended property and the adversary's capabilities. An algorithm that looks complicated is not, by that fact alone, secure.

1.1 Information security and cryptography

Definition 1.1 (Asset, security property, and threat)**.** A *digital asset* is information or a digital capability with value to someone: messages, credentials, financial records, photographs, or access to a service. A *security property* states what should remain protected. A *threat* is a potential way to violate that property.

Definition 1.2 (Core information-security properties)**.**

- **Confidentiality:** prevent an unauthorised party from learning protected information.
- **Integrity:** prevent or detect unauthorised modification of protected information.
- **Availability:** make a service or asset accessible to authorised users when needed.
- **Anonymity:** hide an entity's identity within a specified set or interaction, under a stated observation model.

For a bank transfer, confidentiality concerns who can read the amount; integrity concerns whether someone can change it; availability concerns whether the transfer service can be reached; anonymity concerns whether the participants can be identified. An encrypted transfer may still be blocked or visibly associated with its sender. Each property therefore answers a separate question.

Information security also addresses software faults, hardware leakage, malicious programs, networks, human deception, and operational controls. Cryptography provides mathematical tools within this broader setting. Its guarantees rely on assumptions about the scheme, keys, and surrounding system.

Example 1.3 (A protected message on a compromised laptop)**.** Even if an encryption scheme hides a transmitted message, malware at the receiving endpoint may read the decrypted plaintext. This does not automatically contradict the mathematical encryption claim: the claim may concern only what an observer of the communication learns.

1.2 Secure communication and adversarial models

Definition 1.4 (Passive and active adversary)**.** A *passive* adversary observes communication without changing it. An *active* adversary can also modify, inject, reorder, replay, or suppress messages, according to the model. A man-in-the-middle adversary positions itself between intended communicating parties and can interfere with their exchange.

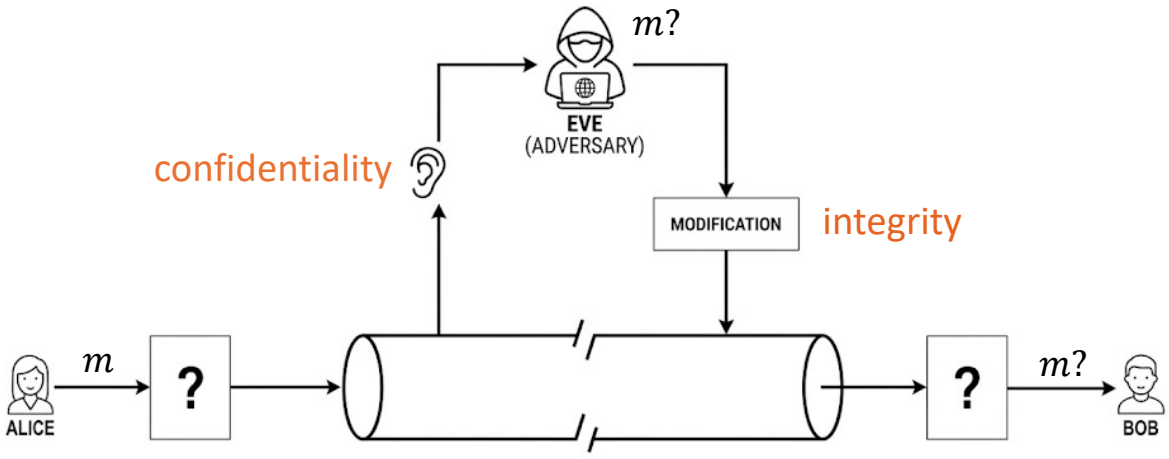


Figure 1.1: Alice sends through two protection boxes; Eve can listen to the channel or change what travels through it.

In Figure 1.1, start at Alice on the left. Her plaintext m enters the first box before reaching the exposed channel; Bob's box must recover the intended message on the right. The question marks ask which algorithms belong in those boxes. The upper listening branch asks whether Eve can learn m from what she intercepts. The modification branch asks whether she can change the transmitted data and have Bob accept the altered result. For a payment, these are two different questions: can Eve read the amount, and can she change it? Encryption addresses the first; an appropriate integrity mechanism is needed for the second.

Remark 1.5 (State both the goal and the access)*.* Before judging a scheme, describe the attacker's position. Is Eve only listening to the channel, or can she replace a message and observe Bob's reaction? The second attacker has an extra source of information and may defeat a scheme that withstands listening alone. A security experiment records these permissions precisely. It also fixes the goal: preventing a forged message from being accepted does not force a channel to deliver any message at all.

1.3 Encryption: syntax before security

1.3.1 Notation

Let n be a positive integer *security parameter* controlling the scheme's scale. Write m for a plaintext message, c for a ciphertext, and k for a secret key. A lowercase symbol denotes a particular value; its uppercase version, such as M or K, denotes a random variable when distributions matter.

The assignment arrow $\leftarrow$ records an algorithm's output. Algorithms may use internal randomness. The notation $x \xleftarrow{\$} \mathcal{X}$ means sampling x uniformly from a finite set $\mathcal{X}$. The set $\{0,1\}^n$ contains all n-bit strings and has size 2^n.

Definition 1.6 (Symmetric encryption scheme)**.** A symmetric scheme specifies key generation, encryption, and decryption:

$$k \leftarrow \mathrm{Gen}(n), \qquad c \leftarrow \mathrm{Enc}_k(m), \qquad m' \leftarrow \mathrm{Dec}_k(c).$$

Alice and Bob share the secret key k. Read the interface as three steps: create the shared secret, turn a message into a ciphertext using it, then use that same secret to recover the message. Correctness requires $m' = m$ for supported messages and valid generated keys, with the scheme's specified correctness probability. The schemes considered here have perfect correctness.

Definition 1.7 (Public-key encryption scheme)**.** Key generation outputs a public encryption key pk and a private decryption key sk:

$$(pk, sk) \leftarrow \mathrm{Gen}(n), \qquad c \leftarrow \mathrm{Enc}_{pk}(m), \qquad m' \leftarrow \mathrm{Dec}_{sk}(c).$$

Anyone with pk can encrypt for its owner; the owner decrypts using sk. Correctness requires recovery of the original message. The public key may be disclosed, but its binding to the intended recipient must be established separately.

Example 1.8 (The recipient's key)**.** If Alice wants confidentiality from everyone except Bob, she encrypts using Bob's public key. Using a public key supplied by an unauthenticated intermediary may instead protect the message for the intermediary. Public-key encryption does not eliminate the identity-binding problem.

Remark 1.9 (Correctness is not secrecy). Correctness asks whether Bob can recover what Alice sent. Secrecy asks what Eve can learn along the way. Simply sending the message unchanged, $\mathrm{Enc}_k(m) = m$ and $\mathrm{Dec}_k(c) = c$, passes the first test and fails the second. A proof that encryption and decryption cancel therefore establishes only correctness. Separately, the security parameter controls the scheme's scale; it need not equal every key or message length.

1.4 Integrity: MACs and signatures

Definition 1.10 (Message authentication code)**.** A message authentication code (MAC) uses a shared secret key:

$$k \leftarrow \mathrm{Gen}(n), \quad t \leftarrow \mathrm{Mac}_k(m), \quad \mathrm{Ver}_k(m, t) \in \{\top, \bot\}.$$

The value t is a tag, $\top$ means accept, and $\bot$ means reject. Correctness means that a legitimately generated tag is accepted. Unforgeability requires an attacker without the key to have only a suitably bounded chance of producing a valid tag for a new message, under the specified interaction model.

Definition 1.11 (Digital signature)**.** A signature scheme uses separate signing and verification keys:

$$(pk, sk) \leftarrow \mathrm{Gen}(n), \quad s \leftarrow \mathrm{Sign}_{sk}(m), \quad \mathrm{Verif}_{pk}(m, s) \in \{\top, \bot\}.$$

Here s denotes the signature. Verification returns a decision, not a recovered message. Correctness requires acceptance of valid signatures; unforgeability bounds creation of a valid signature on a new message without the signing key, even after the allowed prior interactions.

Example 1.12 (Who can create the evidence?)**.** Suppose Bob receives the instruction "pay 10 with a valid MAC. Under the security assumptions, this protects against an outsider who lacks the key; however, Bob also holds that key and could have produced the tag himself. The tag alone cannot distinguish which key holder created it. With Alice's signature, Bob verifies using a public key that does not let him sign as Alice. In both cases the instruction can remain readable: checking who can authenticate a message is a separate task from hiding it.

Remark 1.13 (Authenticity is not freshness). An attacker can record an authentic "pay 10 instruction and resend the identical authenticated message. Its tag is still valid: the attacker has copied evidence rather than forged it. To reject this replay, the receiver must also check freshness, for example by authenticating a sequence number and recording which numbers have already been accepted. A valid tag answers an authenticity question; the receiver's context and state determine whether this particular instruction should be accepted now.

1.5 The one-time pad and perfect secrecy

1.5.1 Bitwise exclusive OR

Definition 1.14 (Exclusive OR)**.** For bits a, b, their exclusive OR $a \oplus b$ is 1 exactly when they differ. For equal-length strings, apply it coordinate by coordinate. It satisfies

$$a \oplus 0 = a, \qquad a \oplus a = 0, \qquad (m \oplus k) \oplus k = m.$$

Definition 1.15 (One-time pad)**.** For an n-bit message, sample an independent uniform key $k \in \{0,1\}^n$ and use it for exactly one encryption:

$$\mathrm{Enc}_k(m) = m \oplus k, \qquad \mathrm{Dec}_k(c) = c \oplus k.$$

The resulting scheme is the *one-time pad* (OTP). Its key is as long as its message. Uniformity concerns the entire key string, and independence concerns its relation to the message. At each position, a key bit of 0 keeps the message bit and a key bit of 1 flips it. Bob knows which choice was made and can undo it; Eve lacks that information.

Algorithm 1.1 One-time pad: generation, encryption, and decryption

Require: A length $n \geq 1$; messages and ciphertexts of that length
Ensure: Correct encryption and decryption; fresh independent key per message

```
function GEN(n)
    k ←$ {0,1}^n
    return k
end function
function ENC(k, m)
    return m ⊕ k
end function
function DEC(k, c)
    return c ⊕ k
end function
```

Example 1.16 (Eight-bit encryption)**.** Take $m = 10110010$ and $k = 01101100$. Then $c = m \oplus k = 11011110$, and $c \oplus k = 10110010$ recovers the message. This calculation illustrates correctness; secrecy is a statement about the distribution of keys, not about one particularly complicated-looking ciphertext.

1.5.2 What exactly does the ciphertext reveal?

Write P for probability and $P(A \mid B)$ for conditional probability. Let M be the random message, K the random key, and $C = \mathrm{Enc}_K(M)$ the random ciphertext.

Definition 1.17 (Perfect secrecy)**.** An encryption scheme has perfect secrecy if, for every allowed message distribution and every m, c with $P(C = c) > 0$,

$$\boxed{P(M = m \mid C = c) = P(M = m).}$$

The right-hand side is the attacker's belief before seeing the ciphertext; the left-hand side is the belief after seeing it. Equality means that this observation supplies no evidence favouring one message over another. The initial beliefs may already be strongly biased.

Theorem 1.18 (Perfect secrecy of a fresh one-time pad)**.** *If K is uniform on $\{0,1\}^n$ and independent of M, then one-time-pad encryption of one n-bit message has perfect secrecy against ciphertext-only observation.*

Proof. Fix a message m with positive probability and any ciphertext c. Exactly one key, $k = m \oplus c$, produces that ciphertext. Independence and uniformity give

$$P(C = c \mid M = m) = P(K = m \oplus c) = 2^{-n}.$$

Every possible message gives this same ciphertext probability. By the law of total probability,

$$P(C = c) = \sum_{m' :\, P(M=m')>0} P(M = m')P(C = c \mid M = m') = 2^{-n} \sum_{m'} P(M = m') = 2^{-n}.$$

Bayes's theorem therefore yields

$$P(M = m \mid C = c) = \frac{P(M = m)P(C = c \mid M = m)}{P(C = c)} = \frac{P(M = m)2^{-n}}{2^{-n}} = P(M = m).$$

Zero-probability messages remain impossible as well. Intuitively, any proposed message can explain the observed ciphertext using exactly one key, and all those keys were equally likely. The ciphertext therefore gives no reason to prefer one proposed message over another beyond the initial message probabilities. □

Remark 1.19 (The message need not be uniform). Suppose equal-length encodings of `yes` and `no` have prior probabilities 0.9 and 0.1. An attacker can already guess `yes` and be right 90% of the time. Perfect secrecy allows this: after seeing the OTP ciphertext, the probabilities are still 0.9 and 0.1. The guarantee is that the ciphertext does not improve the attacker's information. It neither removes prior knowledge nor automatically hides message length.

1.5.3 Individually fair bits are not enough

Example 1.20 (A correlated two-bit key)**.** Choose $K = 00$ or $K = 11$, each with probability $1/2$. Each individual key bit is uniform, yet only two of the four possible keys occur. For $C = M \oplus K$, the XOR of the two ciphertext bits equals the XOR of the two message bits, so the message's parity is revealed.

More explicitly, if M is uniform on $\{00, 01, 10, 11\}$ and $C = 00$, then M can only be 00 or 11, each with posterior probability $1/2$ instead of prior probability $1/4$. Perfect secrecy fails despite the fair marginal key bits.

Here the same fair random choice controls both key positions: either keep both message bits or flip both. Neither operation changes whether the two message bits are equal. This explains the parity leak without knowing the key. To avoid such relations, the full key must have the required joint distribution. Sampling each bit uniformly *and independently* produces a uniform bit string; checking the bits one at a time misses their dependence.

1.5.4 Reusing the key reveals relations

Proposition 1.21 (Two-time-pad relation)**.** *If the same key encrypts two equal-length messages, then*

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k \quad \Longrightarrow \quad c_1 \oplus c_2 = m_1 \oplus m_2.$$

Proof. XOR is associative and commutative, and $k \oplus k$ is the zero string. The key cancels from the XOR of the two ciphertexts. □

Example 1.22 (One known message exposes the other)**.** Once m_1 is known, the other message is $m_2 = m_1 \oplus c_1 \oplus c_2$. For example, if $m_1 = 0101$, $m_2 = 1100$, and $k = 1010$, then $c_1 = 1111$ and $c_2 = 0110$. Their XOR is 1001, exactly $m_1 \oplus m_2$. Knowing m_1 now recovers m_2 by XOR with 1001. Even without a known message, predictable structure can make this relation useful. The shared unknown key disappears because it was used twice.

Remark 1.23 (Secrecy is not integrity). An attacker need not read a message to change it. Let δ be a chosen bit string of the same length as c. Replacing c by $c' = c \oplus \delta$ changes the decrypted result to $m' = m \oplus \delta$. For example, putting a single 1 in δ flips the corresponding plaintext bit. The attacker may not know its original value, but still controls this change. The OTP provides secrecy against observation; detecting modification requires an integrity mechanism.

The practical cost is key distribution: each message needs a fresh, secret, independent key of equal length. Expanding a short key deterministically cannot produce an independent uniform pad of arbitrary length. A computational construction may instead make the expansion look unpredictable to a bounded adversary, under a different security definition.

1.6 Computational security and exhaustive search

Definition 1.24 (Computational security)**.** Protection against adversaries with bounded computational resources, rather than against every possible computation. The claim must state the security task, attacker access, resource bound, and success or advantage bound.

1.6.1 A hypothetical 128-bit key search

Suppose the only effective attack is exhaustive search over a uniform 128-bit key and the attacker can recognise a correct candidate. Assume a computer tests 2^{30} keys per second and the attacker has $10^6 \approx 2^{20}$ such computers. These are the exercise's assumptions, not a measured performance claim. The approximate combined rate is 2^{50} candidates per second.

An average successful search tests about 2^{127} candidates. Its expected duration is therefore

$$\frac{2^{127}}{2^{50}} = 2^{77} \text{ seconds} \approx 4.8 \cdot 10^{15} \text{ years}.$$

Searching all keys takes about twice that time. With Q distinct tested candidates, before exhaustive coverage, the chance of finding a uniformly selected key is $Q/2^{128}$ under these assumptions.

Remark 1.25 (A search-space estimate is conditional). The estimate asks how long it takes to try keys one after another. It says nothing about a shortcut that avoids this search. A generator that selects only a tiny subset of nominally 128-bit keys reduces the search space; a protocol might leak message information without exposing the key at all. The large number is persuasive only after justifying why exhaustive search is the relevant attack model.

1.6.2 Security changes over the lifetime of data

Faster hardware, more parallel resources, and better algorithms can increase an adversary's ability over time. Encryption often needs confidentiality for the entire useful lifetime of a recorded message: future decryption reveals the historical plaintext.

Authentication has a different temporal question. A future forgery does not necessarily undo a correctly completed, freshness-checked interaction, but long-lived signatures and archived evidence can still need protection against later forgery. The distinction is about the required lifetime of the guarantee, not a general licence to ignore future authentication breaks.

1.7 Cryptographic methodology and reductions

1.7.1 Define, design, and justify

A disciplined methodology separates the tasks:

1. **Define** the security property and adversary model.
2. **Design** an efficient construction intended to meet that property.
3. **Reduce** the construction's security to explicit, simpler assumptions.
4. **Cryptanalyse** those assumptions and examine their parameters.

The algorithms and design are assumed public; the secret keys remain protected. This is the principle that security must not rely on hiding the mechanism.

Definition 1.26 (Security reduction)**.** Let S be a cryptographic scheme and $\mathcal{P}$ a computational problem. A reduction shows how a successful adversary against S can be used to solve $\mathcal{P}$ with specified efficiency and success. Consequently, hardness of $\mathcal{P}$ implies the claimed security of S, provided the reduction's assumptions and model apply.

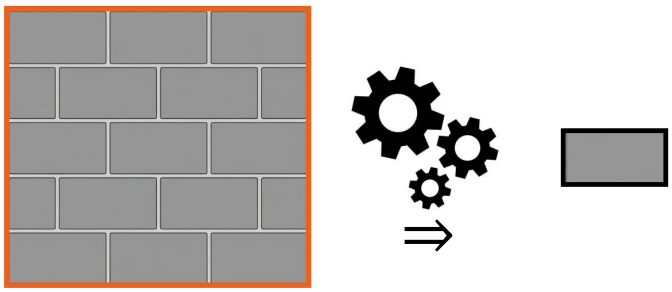


Figure 1.2: The gears depict a reduction: turning a way to break the whole construction into a way to solve an underlying problem.

The wall in Figure 1.2 represents the complete scheme; the single brick represents the underlying problem whose difficulty we assume. The gears are a procedure we must actually describe. Given an instance of that problem, the procedure creates a situation in which a hypothetical scheme attacker can operate, then uses the attacker's answer to solve the instance. We do not have to possess such an attacker to prove the implication. We must show that *if* one existed, the procedure would work within the claimed time and probability bounds. The picture is a reminder to build that procedure, rather than evidence that a wall-shaped construction is secure.

Remark 1.27 (Read the implication in the correct direction). The implication is

$$\text{hard underlying problem} \implies \text{secure scheme},$$

or, contrapositively,

$$\text{efficient scheme attack} \implies \text{efficient problem solver}.$$

To use the second implication, temporarily imagine that someone hands us an attacker that breaks the scheme. The reduction must turn that attacker into a solver for the underlying problem. If we trust that no efficient solver exists, we must then reject the imagined efficient attacker. This is a conditional argument: the reduction explains why the construction inherits hardness, while the hardness assumption still needs independent scrutiny.

1.7.2 Account for the cost of the reduction

For a particular reduction, suppose an adversary running in time t with success ε_S yields a solver running in time at most $t + \Delta$ with success at least ε_S/L. Here $\Delta \geq 0$ is overhead and $L \geq 1$ a success-loss factor.

If every solver within the relevant time budget succeeds with probability at most $\varepsilon_{\mathcal{P}}$, then

$$\varepsilon_S/L \leq \varepsilon_{\mathcal{P}}, \qquad \varepsilon_S \leq L\varepsilon_{\mathcal{P}}.$$

For example, if the problem solver's success is bounded by 2^{-80} at the required time budget and $L = 2^{20}$, the inherited scheme bound is only 2^{-60}. The reduction loses a factor 2^{20} in the guarantee, even if the scheme's actual attack probability could be lower. Likewise, large overhead demands a hardness bound against a more expensive solver. This illustrates why efficiency and loss are part of a useful reduction; actual reductions may have different formulas.

Remark 1.28 (Choosing the level of the assumption). Suppose an encryption scheme is built from a small component whose behaviour has been studied in isolation. A reduction can then identify precisely which property of that component suffices for the whole scheme. Building a fast permutation through several generic transformations, however, may require many component calls. Alternatively, directly designing a fast permutation avoids those calls but asks us to trust a more complex object. Chapter 3 develops this comparison concretely. The choice concerns what must be trusted and what it costs to evaluate; a smaller component is not automatically a stronger assumption.

1.8 Concrete security and security experiments

Definition 1.29 (Concrete security)**.** A scheme is (t, ε)-secure for a stated goal if every adversary within the resource bound t has success probability or advantage at most ε, according to that goal's experiment. The parameters are meaningful only together with the interaction rules and the definition of winning.

To read Figure 1.3, choose a value of n on the horizontal axis and compare the heights of the two curves at that same point. The dashed curve is the work for a legitimate operation; the rising solid curve is the best attack cost in this conceptual picture. The desired operating region makes the legitimate operation feasible while putting the attack beyond the adversary's budget. For example, a candidate parameter is useful if encryption fits within the service's time budget while the best attack exceeds the attacker's available work. A larger vertical gap is helpful, but the plot supplies neither measured times nor the success probability of a particular scheme; those require a concrete analysis.

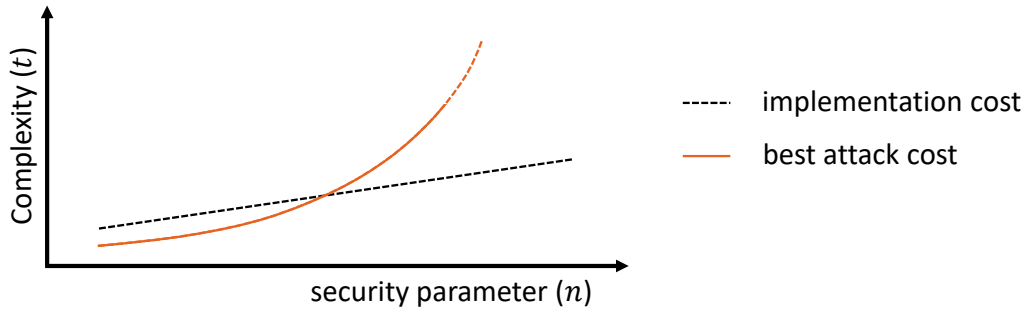


Figure 1.3: Increasing the parameter should make attacks much more expensive while keeping legitimate use affordable; the curves illustrate this design goal.

1.8.1 Winning probability versus distinguishing advantage

For key recovery or a fresh MAC forgery, small success probability can be the direct goal. For distinguishing between two equally likely hidden choices, however, blind guessing already succeeds with probability $1/2$.

Definition 1.30 (One-challenge confidentiality experiment)**.** An adversary $\mathcal{A}$ chooses two supported equal-length messages m_0, m_1. A challenger samples a secret bit $b \in \{0,1\}$ uniformly and gives the adversary an encryption of m_b. The adversary returns a guess $\widehat{b}$. In the convention used here, its distinguishing advantage is

$$\mathrm{Adv}_S(\mathcal{A}) = \left| P(\widehat{b} = b) - \frac{1}{2} \right|.$$

Imagine the two messages encode "approve" and "decline" using the same number of bits. The adversary chooses both candidates, so the test asks whether the ciphertext reveals which one was used, even when the candidates are already known. Random guessing succeeds with probability $1/2$; a success probability of 0.51 gives advantage 0.01 in this convention. Equal lengths remove the trivial clue of ciphertext length. This experiment permits observation alone; later models grant additional access.

Algorithm 1.2 One-challenge symmetric-encryption confidentiality experiment

Require: Security parameter n, scheme S, adversary $\mathcal{A}$
Ensure: A Boolean result indicating whether $\mathcal{A}$ guessed correctly

1: $k \leftarrow \mathrm{Gen}(n)$
2: Obtain equal-length supported messages (m_0, m_1) from $\mathcal{A}$
3: $b \xleftarrow{\$} \{0,1\}$
4: $c \leftarrow \mathrm{Enc}_k(m_b)$
5: Give c to $\mathcal{A}$, retaining its earlier internal state
6: Receive $\widehat{b}$ from $\mathcal{A}$
7: **return** whether $\widehat{b} = b$

Example 1.31 (Perfect secrecy gives zero advantage)**.** For one correctly used OTP challenge, both messages induce the same ciphertext distribution. The ciphertext therefore gives no information about b, so even an unbounded adversary has success probability $1/2$ and advantage 0. Reusing the key in another exchange would change the experiment and may invalidate the conclusion.

1.9 Asymptotic security and negligible functions

1.9.1 What grows with the parameter?

Definition 1.32 (Probabilistic polynomial-time adversary)**.** A probabilistic polynomial-time (PPT) adversary is a randomised algorithm whose running time is bounded by a polynomial in the security parameter n, such as an^b for fixed positive constants a, b. It describes an asymptotic class of algorithms, not a fixed number of seconds.

Definition 1.33 (Negligible function)**.** A nonnegative function f is negligible if it eventually becomes smaller than the reciprocal of every positive polynomial. Formally,

$$\boxed{\text{for every positive polynomial } p, \text{ there exists } N \text{ such that } \forall n > N, \quad f(n) < \frac{1}{p(n)}.}$$

Read the quantifiers in order: first choose any polynomial benchmark, however demanding; then beyond some threshold, the function must stay below its reciprocal. One successful benchmark is insufficient. In particular, merely tending to zero is too weak: $1/n$ tends to zero but never falls below $1/n^2$ for $n > 1$. The threshold N may depend on the chosen polynomial p.

Proposition 1.34 (Two instructive examples)**.** *The function 2^{-n} is negligible, whereas n^{-1000} is not.*

Proof. An exponential eventually dominates every fixed polynomial, so $p(n)2^{-n} \to 0$ for every polynomial p. Thus $2^{-n} < 1/p(n)$ for sufficiently large n.

For $f(n) = n^{-1000}$, choose the polynomial $p(n) = n^{1001}$. For every $n > 1$, $n^{-1000} > n^{-1001} = 1/p(n)$. The required eventual inequality fails for this particular polynomial, so f is not negligible. □

Example 1.35 (Tiny at a fixed parameter is a different statement)**.** At $n = 128$, $n^{-1000} = 128^{-1000} = 2^{-7000}$, an extremely small value. Nevertheless the function is not negligible in the asymptotic sense. Conversely, the definition of negligibility alone does not tell us whether a particular small value of n gives an acceptable concrete bound.

Definition 1.36 (Asymptotic cryptographic security)**.** For every PPT adversary, the relevant success probability or advantage must be bounded by a negligible function of n, according to the chosen security experiment. In a confidentiality experiment, the negligible quantity is the advantage over guessing, not the full winning probability.

Remark 1.37 (Quantifiers and practical interpretation). The adversary's polynomial and the point where its bound becomes useful may vary with the adversary. Asymptotic security organises theory and supports reductions; concrete analysis asks what happens for actual key sizes, query counts, and resource budgets. Increasing n is meaningful only through the scheme's defined parameter choices and proven assumptions.

1.10 Course scope and how to study the constructions

The course develops symmetric encryption, block ciphers, symmetric authentication, public-key encryption, signatures, authenticated encryption, post-quantum cryptography, and secure communication with Transport Layer Security (TLS), followed by a panorama of more advanced topics.

Some constructions are chosen to make a security argument transparent; others are selected for practical efficiency. Distinguish a teaching example from a deployment recommendation. A good

study routine is to state the syntax and correctness first, identify the security experiment, then examine the construction's assumptions and limitations. The introductory material describes a closed-book written examination, so being able to reconstruct definitions and arguments matters more than memorising diagram layouts.

1.11 Review questions

Exercise 1.38 (Check your understanding)**.**

1. Give a confidentiality goal and an integrity goal for the same communication. What can an active adversary do that a passive one cannot?
2. State the three encryption algorithms and distinguish symmetric keys from public/private key pairs.
3. What does signature verification return? Why can a MAC holder generate evidence that another key holder can also generate?
4. Prove OTP correctness and derive $P(M = m \mid C = c) = P(M = m)$ without assuming a uniform message distribution.
5. Why do individually uniform key bits not suffice? Use the two-bit counterexample.
6. Derive the two-time-pad relation and explain how a known message exposes the other one.
7. Why can a perfectly secret ciphertext still be modified undetectably?
8. Reproduce the hypothetical 128-bit exhaustive-search estimate and state its assumptions.
9. Write both directions used to explain a security reduction. Which hardness claim remains an assumption?
10. Why can a large reduction loss matter for concrete parameters?
11. Why is a confidentiality adversary's full success probability not required to be negligible?
12. State the quantifiers in the definition of negligible and prove the answers for 2^{-n} and n^{-1000}.

Chapter 2

Symmetric encryption

The one-time pad achieves perfect secrecy with a fresh key as long as the message. Computational encryption replaces that large supply of secret randomness by a short key and an explicit hardness assumption. This chapter develops the resulting security argument, strengthens the adversary's access, and explains why a confidentiality proof alone does not prevent active attacks.

2.1 A dictionary of pseudorandom objects

2.1.1 Real and ideal distributions

For the following dictionary, n is the key or seed length, l an output length, m a function-input length, and o a function-output length. These lengths can depend on the security parameter. All algorithms and lengths used in an asymptotic experiment must be polynomially bounded in that parameter. Write U_l for an independent uniform l-bit string.

Definition 2.1 (Distinguisher and computational indistinguishability)**.** A distinguisher Dist is an algorithm that receives a sample or access to an object and returns one bit. Two ensembles of random objects X_n, Y_n are computationally indistinguishable if, for every probabilistic polynomial-time (PPT) distinguisher,

$$|P(\mathrm{Dist}(X_n) = 1) - P(\mathrm{Dist}(Y_n) = 1)| \leq \mathrm{negl}(n).$$

Here negl denotes a negligible function, as defined in Chapter 1. The probabilities include the randomness of the object and of the distinguisher. The bound may depend on the chosen distinguisher.

An ensemble is a family of distributions indexed by n. Think of the distinguisher as applying a test to decide whether an object came from the real construction or from an ideal random source. Its output 1 means that the test accepts. If it accepts 80% of real samples but only 30% of ideal samples, its acceptance rates differ by 0.5: the test detects a substantial difference. If every efficient test has almost the same acceptance rate in both worlds, the objects look equivalent to an efficient observer. This difference-of-rates convention is distinct from excess over 1/2 in a hidden-bit guessing game.

2.1.2 Pseudorandom generators

Definition 2.2 (Pseudorandom generator)**.** A pseudorandom generator (PRG) is an efficiently computable deterministic function

$$G : \{0,1\}^n \longrightarrow \{0,1\}^l, \qquad l > n,$$

such that, for a uniform secret seed K and every PPT distinguisher,

$$|P(\mathrm{Dist}(G(K)) = 1) - P(\mathrm{Dist}(U_l) = 1)| \leq \mathrm{negl}(n).$$

The integer $l - n$ is the stretch. The seed supplies all the generator's randomness; evaluating G on the same seed always returns the same string.

Proposition 2.3 (The counting gap)**.** *There are 2^l possible l-bit strings but only 2^n seeds. Thus the image $\mathrm{Im}(G) = \{G(k) : k \in \{0,1\}^n\}$ has size at most 2^n. A PRG output cannot be uniformly distributed over all l-bit strings.*

Proof. Each seed contributes one output, possibly already produced by another seed. The test "does the input lie in $\mathrm{Im}(G)$?" accepts $G(K)$ with probability 1 and accepts U_l with probability at most 2^{n-l}. Its distinguishing gap is at least $1 - 2^{n-l}$. However, implementing this test by enumerating the seeds may require exponential time. □

Remark 2.4 (Pseudorandomness is a computational claim)*.* A short seed selects an output from a relatively small set of long strings. An observer with unlimited time could list that set and test membership. The PRG assumption says that an efficient observer has no useful shortcut for recognising it. Thus counting shows a difference exists, while security asks how much computation is needed to detect it. The assumption never turns a deterministic expansion into truly uniform randomness.

Example 2.5 (Stretch alone does not give a PRG)**.** The function $G(k) = k\|k$ doubles the length; $\|$ denotes concatenation. An efficient test compares the two halves. It accepts a generated string with probability 1 and a uniform $2n$-bit string with probability 2^{-n}. The distinguishing gap $1 - 2^{-n}$ is large.

2.1.3 Pseudorandom functions and permutations

Definition 2.6 (Oracle access)**.** An oracle gives an algorithm access to a function without revealing its internal description or key. The algorithm submits an input and receives the corresponding output. Queries may be adaptive: a later input can depend on earlier answers. A function oracle must return the same output whenever the same input is repeated.

Definition 2.7 (Pseudorandom function)**.** A pseudorandom function (PRF) is an efficient keyed family

$$F : \{0,1\}^n \times \{0,1\}^m \longrightarrow \{0,1\}^o, \qquad F_k(x) = F(k,x),$$

whose oracle for a uniform key K is computationally indistinguishable from an oracle for a uniformly chosen function $f : \{0,1\}^m \to \{0,1\}^o$:

$$\left|P(\mathrm{Dist}^{F_K(\cdot)} = 1) - P(\mathrm{Dist}^{f(\cdot)} = 1)\right| \leq \mathrm{negl}(n).$$

The superscript denotes oracle access. The random function is selected once per experiment and remains consistent across queries.

Picture the ideal function as a table whose entries are filled in only when requested. A first query on an input receives a uniform output, which is written into that row. Returning to the same row gives the stored value. Different rows can contain equal values. This gives exactly the same observable answers as choosing the whole random table at the start, while making consistency explicit. A PRF must imitate the table across many adaptive queries, not merely make one isolated output look random.

Proposition 2.8 (How many functions?)**.** *There are 2^n keys and at most 2^n distinct keyed functions, whereas the number of all functions from m bits to o bits is*

$$(2^o)^{2^m} = 2^{o \cdot 2^m}.$$

Proof. There are 2^m inputs, and each can independently be assigned one of 2° outputs. Multiply the number of choices over all inputs. □

Definition 2.9 (Pseudorandom permutation)**.** A pseudorandom permutation (PRP) is a keyed family

$$P : \{0,1\}^n \times \{0,1\}^m \longrightarrow \{0,1\}^m$$

such that every P_k is a bijection with an efficiently computable keyed inverse P_k^{-1}, and forward oracle access to P_K is computationally indistinguishable from access to a uniformly chosen permutation p on $\{0,1\}^m$:

$$\left| P(\mathrm{Dist}^{P_K(\cdot)} = 1) - P(\mathrm{Dist}^{p(\cdot)} = 1) \right| \leq \mathrm{negl}(n).$$

This is the forward-oracle notion. Allowing the distinguisher access to both the permutation and its inverse gives the stronger PRP notion.

Proposition 2.10 (How many permutations?)**.** *There are $(2^m)!$ permutations of the 2^m possible input blocks. The keyed family still has at most 2^n distinct members.*

Proof. Assigning outputs to successive inputs gives 2^m, then $2^m - 1$, then $2^m - 2$ choices, and so on: outputs cannot repeat. □

Example 2.11 (Two-bit inputs)**.** For $m = o = 2$, there are $4^4 = 256$ functions but only $4! = 24$ permutations. Think of assigning four output labels to four input positions. A function may reuse a label; a permutation must use each label exactly once, as when rearranging four distinct cards. This explains both the counting difference and why a permutation can be inverted uniquely. Pseudorandomness must always specify which of these ideal objects the family is meant to imitate.

Remark 2.12 (A primitive is not a full encryption scheme)*.* A block cipher should make its input/output behaviour look like a uniformly selected permutation to an efficient observer. Once that key is fixed, however, the same input block always has the same output. Processing a full document therefore raises another question: how should successive blocks and different messages use the primitive? A mode of operation supplies this organisation. The later ECB example shows why simply encrypting each block independently can reveal a document's structure even when the primitive itself is secure.

2.2 Single-message eavesdropper security

Write $\mathrm{SE} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ for a symmetric encryption scheme. In the experiment $\mathrm{Exp}^{\mathrm{eav}}_{\mathrm{SE},\mathcal{A}}(n)$, an adversary $\mathcal{A}$ chooses two supported messages m_0, m_1 of equal length. The challenger samples a key and a uniform hidden bit b, returns $c_b = \mathrm{Enc}_k(m_b)$, and the adversary returns a guess $\widetilde{b}$. The experiment returns 1 exactly when $\widetilde{b} = b$.

The adversary receives one challenge encryption and no encryption or decryption oracle. Both messages must have the same length because length can otherwise identify the choice. Their common length l is at most polynomial in n: a PPT adversary cannot explicitly output an exponentially long message.

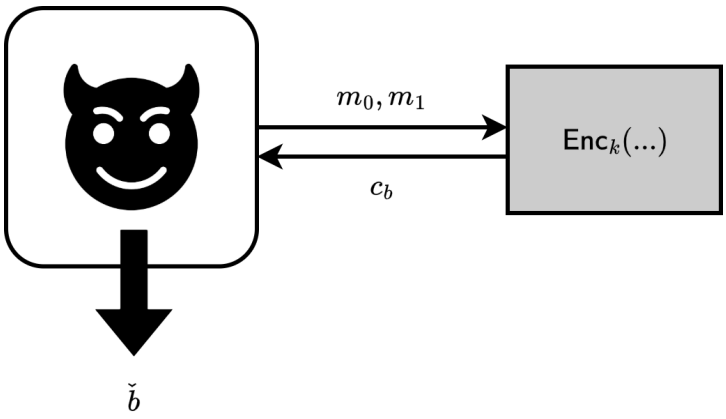


Figure 2.1: The attacker sends two candidate messages, receives one hidden-choice encryption, then guesses which candidate was encrypted.

Read Figure 2.1 as a sequence, not as two parties encrypting independently. The attacker first names both candidates; only then does the challenger choose b secretly. If $b = 0$ it returns an encryption of m_0; otherwise it encrypts m_1. The final downward arrow is the attacker's guess $\widetilde{b}$. For candidates 0000 and 1111 under a fresh four-bit OTP, ciphertext 1010 is compatible with either: the respective keys would be 1010 and 0101, both equally likely. Seeing that ciphertext therefore does not select between the candidates. The experiment tests precisely whether an actual scheme leaves a useful clue.

Definition 2.13 (Eavesdropper security)**.** The scheme is secure against single-message eavesdropping if, for every PPT adversary,

$$P(\mathrm{Exp}^{\mathrm{eav}}_{\mathrm{SE},\mathcal{A}}(n) = 1) \leq \frac{1}{2} + \mathrm{negl}(n).$$

Equivalently, its absolute advantage over guessing is negligible. If an adversary systematically guesses the wrong bit, flipping its output gives an adversary with the opposite bias; this explains the equivalence with an absolute-value formulation.

2.2.1 Emulating the one-time pad with a PRG

Let $G : \{0,1\}^n \to \{0,1\}^l$ be a PRG. Generate a uniform n-bit key and define, for l-bit messages,

$$\mathrm{Enc}_k(m) = m \oplus G(k), \qquad \mathrm{Dec}_k(c) = c \oplus G(k).$$

Correctness follows from XOR cancellation. The idea is to give Bob a reproducible long pad while storing only the short seed. An attacker does not know the seed. If that attacker can exploit some visible feature of the pad to learn the challenge bit, the same behaviour should also reveal that the pad came from G rather than from a uniform source. The reduction makes this intuition precise. Its claim concerns one encrypted message.

Theorem 2.14 (PRG-based eavesdropper security)**.** *If G is a secure PRG, the above scheme is secure in the single-message eavesdropper experiment.*

Proof. The distinguisher receives a string w whose origin it must identify. It cannot test that origin directly, so it uses the encryption adversary as a subroutine. It obtains m_0, m_1, samples a uniform bit b, and returns $w \oplus m_b$ as the challenge. Because it chose b itself, it can check the

adversary's guess and output 1 exactly when the guess is right. It never needs to know the seed that may have generated w.

If $w = G(K)$, the simulation has exactly the real encryption distribution. If $w = U_l$ is independent and uniform, the simulation is a one-time pad and the guess is correct with probability exactly $1/2$. Hence

$$\begin{aligned} &|P(\mathrm{Dist}(G(K)) = 1) - P(\mathrm{Dist}(U_l) = 1)| \\ &= \left| P(\mathrm{Exp}^{\mathrm{eav}}_{\mathrm{SE},\mathcal{A}}(n) = 1) - \frac{1}{2} \right|. \end{aligned}$$

A non-negligible encryption advantage would give the same distinguishing gap against the PRG, contradicting its assumed security. □

Algorithm 2.1 Reduction from an eavesdropper adversary to a PRG distinguisher

Require: A candidate pad $w \in \{0,1\}^l$ and an adversary $\mathcal{A}$
Ensure: One bit: 1 if the simulated challenge was guessed correctly

1: Obtain supported equal-length messages (m_0, m_1) from $\mathcal{A}$
2: $b \xleftarrow{\$} \{0,1\}$
3: $c \leftarrow w \oplus m_b$
4: Give c to $\mathcal{A}$, keeping its previous internal state
5: Receive its guess $\widetilde{b}$
6: **return** 1 if $\widetilde{b} = b$, and 0 otherwise

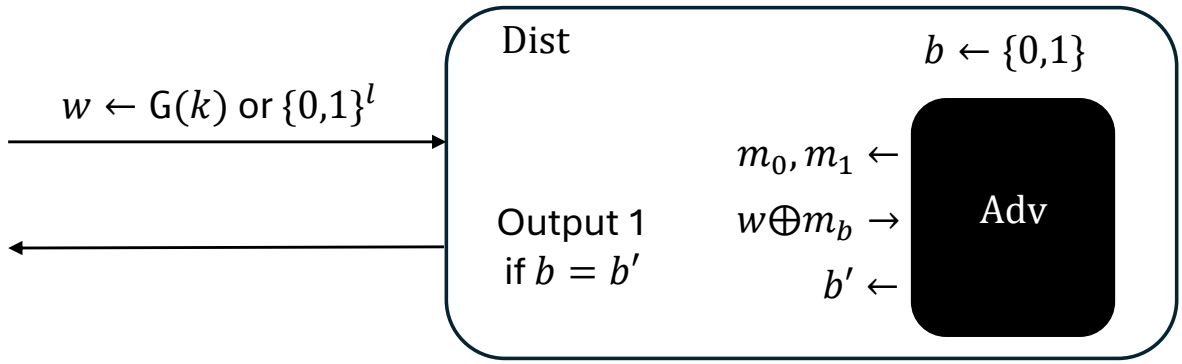


Figure 2.2: Dist uses its unknown-source string as the encryption pad, then checks whether Adv correctly guesses the bit chosen by Dist.

In Figure 2.2, the outer box Dist is trying to identify the origin of w; the inner box Adv is trying to identify an encrypted message. Dist connects these two tasks by sending Adv the ciphertext $w \oplus m_b$. Dist knows the sampled b, so it can judge the answer b' even though it does not know whether w was generated or uniform. If Adv succeeds with probability 0.60 for generated pads but only 0.50 for uniform pads, the observable event "Adv guessed correctly" has probabilities differing by 0.10. Dist has thereby obtained a PRG test with gap 0.10. This explains what the reduction's output bit measures; it is a correctness check on a simulated guess, not a direct inspection of w.

2.2.2 Concrete cost and limitations

Suppose a PRG is (t, ε_G)-secure: every distinguisher running for at most time t has distinguishing gap at most ε_G. If the reduction's additional work costs c_{red}, every encryption adversary running

for at most $t - c_{\text{red}}$ has advantage at most ε_G.

The overhead includes sampling b, forming an l-bit XOR, and comparing the guess. Thus it is small, but generally depends on the message length and computational model. It can be treated as a fixed cost at fixed parameters; it is not a universal asymptotic constant. The ideal reduction has small overhead and no large loss in advantage, preserving a large attacker time budget and a small success bound.

Example 2.15 (Why a second encryption changes the claim)**.** Under the same key, the pad is always $G(k)$. Two ciphertexts satisfy

$$c_1 \oplus c_2 = m_1 \oplus m_2.$$

An encryption of the all-zero string directly reveals $G(k)$. A secure PRG does not fix repeated use of the same pad: the single-message proof did not give the adversary another encryption under that key.

The construction therefore depends both on a computational assumption and on a restrictive interaction model. A useful multi-message scheme needs an appropriate way to initialise each encryption and a stronger definition.

2.3 Chosen-plaintext security

Definition 2.16 (Chosen-plaintext attack)**.** In a chosen-plaintext attack (CPA), an adversary can ask for encryptions of messages it chooses, adaptively, under one fixed secret key. In the indistinguishability experiment it also submits equal-length challenge messages m_0, m_1, receives an encryption of one of them selected by a hidden uniform bit b, and tries to identify b.

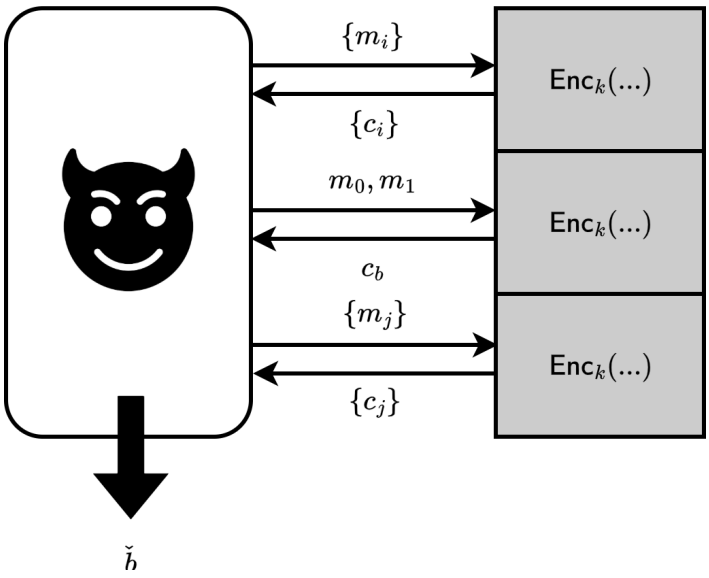


Figure 2.3: All encryption boxes share a key: ordinary chosen-message queries surround a single challenge whose chosen message remains hidden.

Follow the arrows in Figure 2.3 from top to bottom. Before the challenge, the attacker can build examples by requesting encryptions of messages it knows. In the middle it supplies m_0, m_1 and receives an encryption of the secret choice. Afterwards it may request more examples before guessing. The indices i, j distinguish queries before and after that challenge; the braces do not

require simultaneous batches. Why allow these extra queries? A scheme may conceal one isolated ciphertext while exposing links to previous encryptions. In the repeated-pad construction, asking to encrypt zero reveals the pad, immediately making the challenge readable. The CPA experiment deliberately includes that possibility.

Let q bound the number of ordinary encryption queries. For a PPT adversary, q and the total number of processed message bits are polynomially bounded in the security parameter. Each encryption uses the scheme's prescribed fresh randomness. The adversary receives no decryption oracle in this experiment.

Algorithm 2.2 Indistinguishability experiment under chosen-plaintext attack

Require: Security parameter n, scheme SE, adversary $\mathcal{A}$
Ensure: One bit indicating whether the hidden bit was guessed correctly

1: $k \leftarrow \mathrm{Gen}(n)$
2: Give $\mathcal{A}$ access to $\mathcal{O}(m) = \mathrm{Enc}_k(m)$
3: Obtain supported equal-length challenge messages (m_0, m_1)
4: $b \xleftarrow{\$} \{0,1\}$
5: $c_b \leftarrow \mathrm{Enc}_k(m_b)$ with fresh encryption randomness
6: Give c_b to $\mathcal{A}$ and continue its access to $\mathcal{O}$
7: Receive its guess $\widetilde{b}$
8: **return** 1 if $\widetilde{b} = b$, and 0 otherwise

Definition 2.17 (CPA security)**.** The scheme is CPA-secure if for every PPT adversary

$$P(\mathrm{Exp}^{\mathrm{cpa}}_{\mathrm{SE},\mathcal{A}}(n) = 1) \leq \frac{1}{2} + \mathrm{negl}(n).$$

The name IND-CPA means *indistinguishability under chosen-plaintext attack*. Concrete CPA advantage will use the convention $|P(\text{correct guess}) - 1/2|$.

2.3.1 Why deterministic encryption fails here

Proposition 2.18 (Deterministic stateless encryption is not CPA-secure)**.** *A perfectly correct, deterministic, stateless encryption scheme with at least two distinct equal-length messages cannot satisfy the above CPA definition.*

Proof. Choose distinct messages m_0, m_1. Query an encryption of m_0 to get c_0, then request the challenge on (m_0, m_1). Output 0 if the challenge equals c_0, and 1 otherwise. Determinism ensures equality in the first case. Correctness prevents distinct messages from sharing a ciphertext under the same key. The guess is always correct, giving advantage $1/2$. □

Remark 2.19 (Randomisation and state). The comparison attack works because the attacker can obtain a reference ciphertext and recognise it later. Fresh randomness allows the same plaintext to have different ciphertexts, so simple equality is no longer a reliable test. Randomisation is necessary in this stateless model but is not sufficient by itself: its use must also be secure. Stateful or nonce-based schemes instead manage a changing value explicitly, and their experiments must specify the rules for that value. A nonce is a value required not to repeat in the relevant key context.

2.3.2 Visible structure: independent block encryption

Definition 2.20 (Electronic codebook mode)**.** Electronic codebook (ECB) mode splits a message into blocks and sets $c(i) = P_k(m(i))$ independently for each block. Repeated plaintext

blocks give repeated ciphertext blocks because P_k is deterministic and injective.

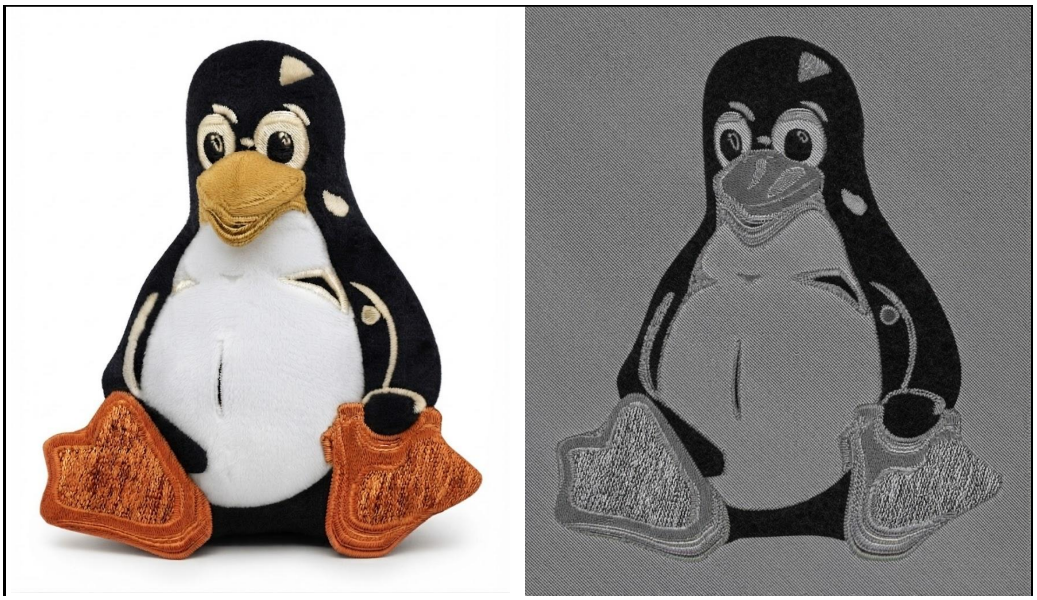


Figure 2.4: The left image is readable; the right illustrates how repeated encrypted blocks can still reveal the same outline.

In Figure 2.4, the important observation is that the right-hand outline can be recognised without recovering the original block values. The attacker can group equal ciphertext blocks and mark their positions. With ECB, equality means that the corresponding plaintext blocks were equal too. The following sequence isolates this mechanism before applying it to an image.

To read the figure, separate the unknown *value* of a block from the visible *pattern of repetitions*. Let A, B, C be three distinct plaintext blocks and let $X = P_k(A)$, $Y = P_k(B)$, $Z = P_k(C)$. Independent block encryption transforms

$$(A, A, A, B, B, C, A) \quad \longmapsto \quad (X, X, X, Y, Y, Z, X).$$

The attacker may not know what X, Y, Z mean, but still knows which positions held the same plaintext block. When repeated blocks occur across uniform regions of an image, their spatial arrangement can reveal regions and outlines. Recognising the penguin is already information about the message; recovering every pixel is unnecessary to demonstrate a confidentiality failure.

The exact visual result depends on the image encoding and block alignment. The mechanism illustrated here is equality leakage from independent deterministic block encryption. A secure message-encryption mode must address those relations as well as the secrecy of individual block values.

2.4 Cipher block chaining

2.4.1 Notation, encryption, and decryption

For the block-cipher modes in the remainder of this chapter, n denotes the *block length in bits*, as in the mode formulas, and κ denotes the key length. These need not be equal. Write P_k for the keyed permutation of n-bit blocks and P_k^{-1} for its inverse. The symbol $P(\cdot)$ still denotes probability when applied to an event; the subscripted P_k is the block cipher.

Split an aligned message into l blocks, numbered from zero:

$$m = m(0) \parallel m(1) \parallel \cdots \parallel m(l-1), \qquad m(i) \in \{0,1\}^n.$$

An *initialisation vector* (IV) is an additional n-bit input used to initialise a message encryption. It is transmitted with the ciphertext and need not be secret.

Definition 2.21 (Cipher block chaining mode)**.** Cipher block chaining (CBC) encryption samples a fresh independent uniform IV, sets $c(-1) = IV$, and computes

$$c(i) = P_k\big(m(i) \oplus c(i-1)\big), \qquad 0 \le i < l.$$

Each block is mixed with the previous ciphertext before entering the permutation. For the first block, the IV supplies this changing context. Thus the same plaintext block can be presented to the permutation with different inputs depending on its predecessor. This is the chaining mechanism; the security claim still requires the IV rules and the PRP assumption.

The transmitted ciphertext is $IV \| c(0) \| \cdots \| c(l-1)$. Decryption computes

$$m(i) = P_k^{-1}(c(i)) \oplus c(i-1), \qquad c(-1) = IV.$$

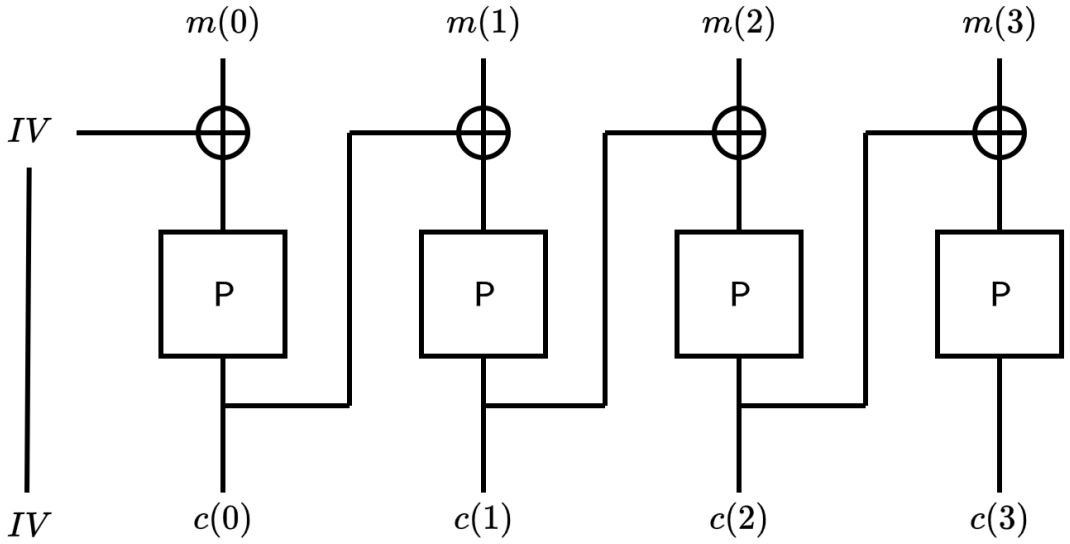


Figure 2.5: In CBC, XOR each plaintext block with its predecessor ciphertext, then apply the keyed permutation; the IV starts the chain.

Trace Figure 2.5 from the first top input $m(0)$. The circled plus combines it with IV; the box P then produces $c(0)$. The bent arrow carries that output to the next circled plus, where it meets $m(1)$. Every P box applies the same P_k; the changing input comes from the chaining value.

For a four-bit calculation, suppose a toy permutation has $P_k(1100) = 0011$ and $P_k(0110) = 1110$. If $IV = 0110$, $m(0) = 1010$, and $m(1) = 0101$, the first XOR gives 1100, hence $c(0) = 0011$. The second XOR gives $0101 \oplus 0011 = 0110$, hence $c(1) = 1110$. To recover the second message block, Bob obtains $P_k^{-1}(1110) = 0110$ and XORs 0011, recovering 0101. These specified entries illustrate the calculation; they are not a proposed secure cipher.

Algorithm 2.3 CBC encryption and decryption of full blocks

Require: Key k and $l \geq 1$ blocks of length n; plaintext must be padded if needed
Ensure: Encryption returns an IV and ciphertext blocks; decryption recovers blocks

function CBC-ENC($k, m(0), \ldots, m(l-1)$)
  $IV \xleftarrow{\$} \{0,1\}^n$; $v \leftarrow IV$
  for $i = 0, \ldots, l-1$ **do**
    $c(i) \leftarrow P_k(m(i) \oplus v)$
    $v \leftarrow c(i)$
  end for
  return $IV \| c(0) \| \cdots \| c(l-1)$
end function
function CBC-DEC($k, IV, c(0), \ldots, c(l-1)$)
  $v \leftarrow IV$
  for $i = 0, \ldots, l-1$ **do**
    $m(i) \leftarrow P_k^{-1}(c(i)) \oplus v$
    $v \leftarrow c(i)$
  end for
  return $m(0) \| \cdots \| m(l-1)$
end function

Proposition 2.22 (CBC correctness)**.** *For every block index i, decryption recovers the original $m(i)$.*

Proof. Applying P_k^{-1} to $c(i)$ yields $m(i) \oplus c(i-1)$. XOR with $c(i-1)$ cancels the chaining value, including the IV for $i = 0$. □

2.4.2 The concrete security bound

Let q now count *all* encrypted messages, including the challenge, and let each contain at most l blocks. Thus at most ql block-cipher calls are used for their encryption. Let τ_P be the execution time of one permutation evaluation. If the block cipher is a (t, ε_P)-secure PRP, the stated concrete CBC bound has the form

$$\varepsilon_{\text{cpa}} \leq 2\varepsilon_P + \frac{(ql)^2}{2^n}, \qquad t_{\text{Adv}} \lesssim t - ql\tau_P.$$

Here ε_{cpa} bounds the challenge-guessing advantage under the stated model and conventions. The time expression displays the dominant permutation cost; a full implementation also has bookkeeping and XOR overhead. Constants in such bounds depend on the exact experiment and advantage convention.

Read the right-hand side as two sources of possible failure. The term $2\varepsilon_P$ accounts for replacing the real block cipher by an ideal permutation in the proof. The other term remains even after that replacement: internal values may collide and disrupt the intended behaviour of the mode. With $N = ql$ processed blocks, there are $N(N-1)/2$ pairs to consider, and a specified coincidence of independent uniform n-bit values has probability 2^{-n}. This pair-counting intuition explains the quadratic scale. The full proof handles dependencies between internal values; the mode's bound should not be read as a claim that all those values are independent.

Example 2.23 (Reading the data limit)**.** For $n = 128$ and $ql = 2^{32}$ blocks, the displayed collision term is 2^{-64}. At $ql = 2^{64}$ it becomes 1, so this bound no longer gives a small guarantee. A negligible key-search probability alone does not eliminate a mode's data-volume term.

Remark 2.24 (Is the bound tight?). An upper bound need not equal an achievable attack advantage. For example, with only one one-block message ($q = l = 1$), an ideal random permutation gives zero challenge advantage: for each IV and either message, its one queried output is uniform. Yet the displayed collision term is $2^{-n} > 0$. For longer messages, even $q = 1$ allows several internal calls, so one cannot conclude from $q = 1$ alone that every collision effect vanishes. The bound is a sufficient guarantee, not an exact success formula for every parameter choice.

2.4.3 Why CBC needs suitable initialisation

Proposition 2.25 (Fixed IV leaks message prefixes)**.** *Under a fixed key and fixed IV, CBC is deterministic. Equal plaintext prefixes produce equal ciphertext prefixes. In particular,*

$$c(0) = c'(0) \quad \Longleftrightarrow \quad m(0) = m'(0).$$

Proof. Both first blocks are obtained by applying the same injective permutation to the plaintext XOR the same IV. Equality therefore holds exactly when the first plaintext blocks agree. The same reasoning continues along any common prefix. □

The IV changes the first input to the block cipher even when the plaintext and key stay the same. It need not be hidden afterwards: Bob must know it to undo the first XOR. What matters for CBC is that the attacker cannot anticipate it while choosing the plaintext and compensate for it in that XOR. Independent uniform sampling at encryption time supplies this unpredictability, with IV collisions covered by the security bound. A fixed IV enables the deterministic comparison attack; merely choosing different predictable values leaves the problem illustrated next.

Example 2.26 (Why predictability can matter)**.** Suppose an earlier encryption of first block x used IV v, producing $y = P_k(x \oplus v)$. If the next IV v' is predictable, choose a new first block $x' = x \oplus v \oplus v'$. Its first ciphertext block is again y. This lets a chosen-plaintext adversary arrange a comparison even though $v' \neq v$. Thus uniqueness alone does not provide the usual CBC guarantee.

2.5 Output feedback and repeated IVs

Definition 2.27 (Output feedback mode)**.** Output feedback (OFB) mode first generates a keystream independently of the message:

$$z(-1) = IV, \qquad z(i) = P_k(z(i-1)), \qquad c(i) = m(i) \oplus z(i).$$

Decryption regenerates $z(i)$ and computes $m(i) = c(i) \oplus z(i)$. It uses the forward permutation, not P_k^{-1}. A final partial plaintext block can use the corresponding prefix of a keystream block.

Figure 2.6 has two different paths. Along the feedback path, start with IV and repeatedly apply P_k to generate $z(0), z(1), \ldots$. At each XOR, a plaintext block joins the corresponding keystream block to produce a ciphertext block. The ciphertext is not the value returned to the next P box.

Continuing the four-bit toy permutation, add the entry $P_k(0011) = 0100$. With $IV = 1100$, the first two keystream blocks are 0011 and 0100. Plaintext blocks $1010, 0101$ therefore encrypt to $1001, 0001$. A different plaintext would change those ciphertexts but leave the two keystream blocks unchanged. This is exactly why repeating the same key and IV repeats the mask.

Proposition 2.28 (OFB IV reuse repeats the entire keystream)**.** *For two messages encrypted with the same OFB key and IV,*

$$c(i) \oplus c'(i) = m(i) \oplus m'(i)$$

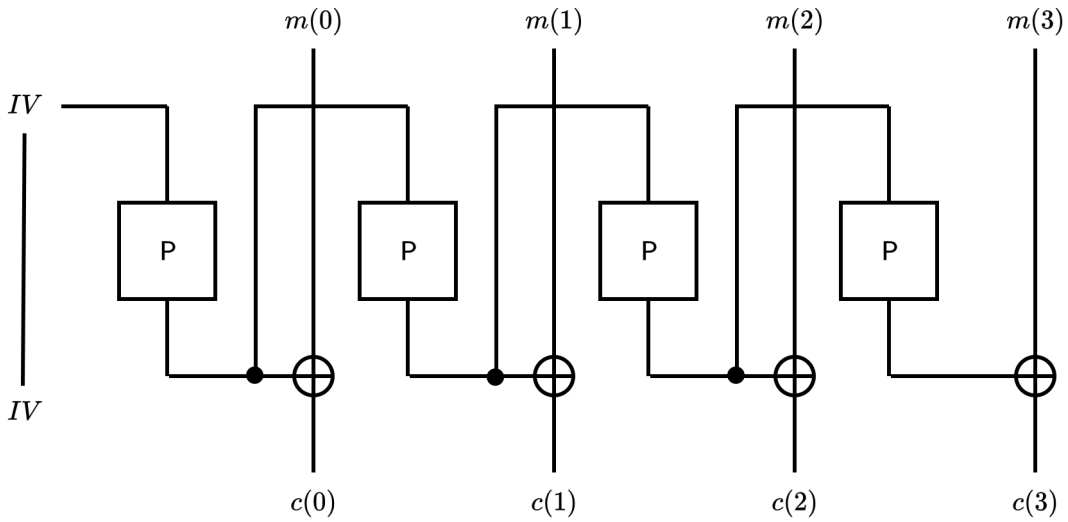


Figure 2.6: In OFB, the permutation outputs feed the next permutation and also mask the message; plaintext never enters the feedback loop.

at every overlapping block or bit position.

Proof. The recurrence for $z(i)$ depends only on the key and IV, so the two keystreams coincide and cancel under XOR. □

The OFB feedback loop never sees the message: once the key and IV are fixed, its entire keystream is fixed too. Reusing the IV therefore reuses the mask at every overlapping position, recreating the two-time-pad failure. Knowing one plaintext reveals the corresponding part of the other. Fixed-IV CBC already leaks equality and common-prefix structure, but the OFB equation exposes a direct XOR relation throughout the overlap. This explains the stronger immediate leakage in this comparison; both fixed-IV constructions fail the desired confidentiality goal.

OFB requires nonrepeating IVs under a given key, with generation following the mode's rules; its requirement is not CBC's unpredictability condition. Random sampling makes IV collisions possible, so the permitted data volume still matters. None of CBC or OFB by itself authenticates a ciphertext.

2.6 Message padding

2.6.1 The PKCS #7 rule

Definition 2.29 (PKCS #7 padding)**.** This padding rule belongs to the Public-Key Cryptography Standards (PKCS). It completes a byte string to a multiple of a block size B bytes, where $1 \le B < 256$. For an original length L bytes, set

$$r = B - (L \bmod B), \qquad 1 \le r \le B,$$

and append exactly r bytes, each with numeric value r. Here $L \bmod B$ is the remainder of dividing L by B.

For a 16-byte block ($B = 16$), an incomplete last block with one data byte receives fifteen padding bytes of value 15; a last block with two data bytes receives fourteen bytes of value 14.

An aligned message receives a full block of sixteen bytes of value 16. Even an empty message receives a complete padding block.

Example 2.30 (Byte values versus written numbers)**.** ASCII means American Standard Code for Information Interchange. With this encoding and $B = 16$, the message `HELLO` has 5 bytes, so it receives 11 padding bytes, each equal to hexadecimal `0B`. A 15-byte message receives one byte `01`; a 16-byte message receives sixteen bytes `10`. Hexadecimal `10` represents decimal 16, not the two text characters `1` and `0`.

Algorithm 2.4 PKCS #7 padding and validation

Require: A byte string m to pad, or a byte string u to unpad; $1 \le B < 256$
Ensure: Padding returns aligned bytes; unpadding returns bytes or $\perp$ (reject)

function PAD(m, B)
    $r \leftarrow B - (|m| \bmod B)$ ▷ $|m|$ is the byte length
    return m followed by r copies of the byte with value r
end function
function UNPAD(u, B)
    if $|u| = 0$ or $|u| \bmod B \neq 0$ **then**
        return $\perp$
    end if
    $r \leftarrow$ numeric value of the last byte of u
    if $r < 1$ or $r > B$ or the last r bytes are not all equal to r **then**
        return $\perp$
    end if
    return u without its last r bytes
end function

Remark 2.31 (Why always add padding?). The receiver must decide where the data ends using the decrypted bytes alone. Suppose an aligned message genuinely ends in `01`. If aligned messages were left unchanged, that data byte could be mistaken for one byte of padding and removed. The rule "always append padding avoids this ambiguity: the final suffix is always a length marker that can be removed. A correctly shaped suffix only identifies padding; an attacker may also manufacture such a suffix, so validity does not authenticate the message.

2.7 Padding oracles and active attacks

2.7.1 The information exposed by a validity bit

Definition 2.32 (Padding oracle)**.** A padding oracle reveals whether the decrypted bytes have valid padding, for ciphertexts submitted by an adversary. Different replies, connection behaviour, or timing can reveal this bit even if no plaintext is returned. This provides information about *decryption*, which the CPA experiment does not permit.

To match the decryption diagram, let c_1 be the targeted ciphertext block and c_0 its genuine preceding block (or the IV if the target is the first block). Define the intermediate block

$$X = P_k^{-1}(c_1), \qquad p_1 = X \oplus c_0.$$

The attacker keeps c_1 fixed, so its decryption X stays fixed, and replaces only the predecessor by a chosen block a. The tested plaintext block becomes $X \oplus a$. This separates what the attacker cannot see (X) from what it can control (a). The oracle then checks whether their XOR ends in a recognised padding suffix. Number the bytes from 0 to 15 for $B = 16$.

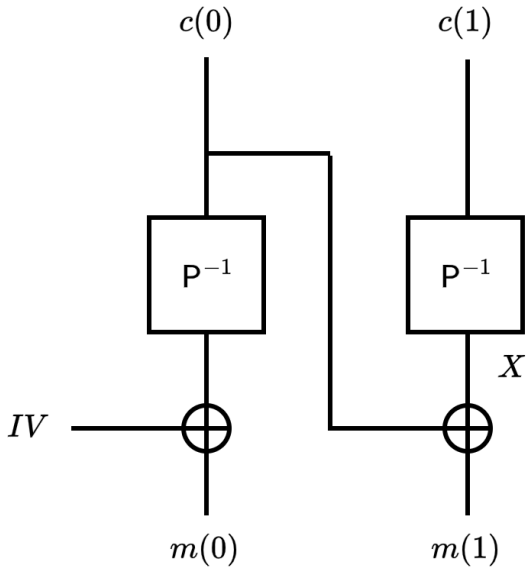


Figure 2.7: The target decrypts to a fixed hidden value X; changing its predecessor changes the final XOR and therefore the padding.

In Figure 2.7, hold the right-hand ciphertext $c(1)$ fixed. Its P^{-1} box then always outputs the same unknown X. The wire arriving from the left carries $c(0)$ into the final XOR, giving the target plaintext $m(1) = X \oplus c(0)$. Here these are the c_1, c_0, p_1 of the attack formulas. Replacing only that left ciphertext changes the XOR mask while keeping X fixed. The attacker asks whether the resulting plaintext ends in valid padding. Repeated yes/no answers constrain X until its bytes can be recovered. The attack does not require predicting the block cipher's inverse output directly.

2.7.2 Recovering the last bytes

A "valid" reply gives an equation about the hidden decrypted bytes. Start by aiming for the simplest suffix, the single byte `01`. Vary the controlled byte $a(15)$ until the reply confirms this one-byte padding (the longer-padding ambiguity is handled below). The attacker then knows the XOR result even though it never observes $X(15)$ directly. Hence

$$X(15) \oplus a(15) = 1, \qquad X(15) = a(15) \oplus 1.$$

The number 1 here is a byte value. Once $X(15)$ is known, force a last byte of value 2 by setting $a(15) = X(15) \oplus 2$. Vary $a(14)$ until the last two bytes form valid padding. This gives

$$X(14) = a(14) \oplus 2.$$

For a desired padding length r, set every already recovered suffix byte $a(j) = X(j) \oplus r$, then search for $a(16-r)$. An accepted candidate gives $X(16-r) = a(16-r) \oplus r$. Continue through all 16 bytes. The original plaintext is then recovered using the *original* predecessor: $p_1 = X \oplus c_0$.

Example 2.33 (A numerical last-byte step)**.** Suppose a trial value $a(15) = \texttt{A6}$ yields confirmed one-byte padding. Then $X(15) = \texttt{A6} \oplus \texttt{01} = \texttt{A7}$. If the original $c_0(15) = \texttt{D2}$, the genuine last plaintext byte was $\texttt{A7} \oplus \texttt{D2} = \texttt{75}$. To search for the next byte, fix $a(15) = \texttt{A7} \oplus \texttt{02} = \texttt{A5}$.

Algorithm 2.5 Recovering one CBC intermediate block from a padding-validity oracle

Require: Target block c_1, original predecessor c_0, block size B bytes
Require: Oracle $\mathcal{O}(a, c_1)$ validating the padding of the last decrypted block
Ensure: Recovered original plaintext block p_1

```
Choose a trial predecessor a of B bytes
for r = 1, ..., B do
    i ← B − r
    for j = i + 1, ..., B − 1 do
        a(j) ← X(j) ⊕ r
    end for
    for each byte value v in {0, ..., 255}, without replacement do
        a(i) ← v
        if O(a, c_1) accepts then
            if r = 1 and B > 1 then
                Query again after temporarily flipping a bit in a(B − 2)
                Restore that byte; discard v if the confirmation rejects
            end if
            X(i) ← v ⊕ r; proceed to the next padding length
        end if
    end for
end for
return X ⊕ c_0
```

Remark 2.34 (Accidental longer padding). At the first step, acceptance might correspond to padding of length 2 or more rather than length 1. Flipping a bit of the penultimate trial byte preserves genuine one-byte padding but destroys such longer padding. The confirmation in Algorithm 2.5 prevents interpreting an accidental match as $X(15) \oplus a(15) = 1$. The algorithm assumes a reliable oracle and that the target can be placed last with a controllable predecessor or IV.

2.7.3 Work factor and security meaning

With a uniformly located correct candidate among 256 values searched without replacement, the mean position is $(256 + 1)/2 = 128.5$. Thus the customary estimate is about 128 queries per byte and about $16 \cdot 128 \approx 2048$ queries per 16-byte block, plus confirmations and protocol overhead. Independent random trials *with* replacement would instead have mean 256. The rough 128 estimate assumes enumeration, not repeated independent guesses.

The attack learns the intermediate value X and then the plaintext; it need not recover the encryption key. It also does not contradict a CPA proof for properly initialised CBC: the padding oracle supplies decryption-dependent information beyond that experiment.

2.7.4 Authentication before decryption-dependent processing

Definition 2.35 (Chosen-ciphertext attack)**.** In a chosen-ciphertext attack (CCA), the adversary obtains decryption-related access in addition to encryption access. A full indistinguishability experiment with a decryption oracle excludes directly decrypting the challenge ciphertext. A padding oracle is weaker than full decryption access but can already be sufficient to recover plaintext through related ciphertexts.

A suitable message authentication code (MAC) can stop the attack when the complete encrypted

representation, including the IV and required context, is authenticated and verified *before* decryption and padding processing. An encrypt-then-MAC composition encrypts first, computes a tag over that representation with an appropriate independent authentication key, and rejects invalid tags before exposing decryption-dependent behaviour.

Remark 2.36 (Placement and failure behaviour matter). The attack depends on getting an answer about each manipulated ciphertext. A valid authentication check performed first stops this conversation: without a valid tag for the modified representation, the attacker receives no padding-dependent reply. If padding is checked before the tag, the useful answer may already have leaked. The complete construction therefore needs both a security argument and failure handling that preserves this order. Authenticated encryption (AE) combines confidentiality and ciphertext integrity. Authenticated encryption with associated data (AEAD) also authenticates specified public context, such as a message header that must remain readable but must not be altered unnoticed.

2.8 Review questions

Exercise 2.37 (Check the assumptions and the experiment)**.**

1. Why can a PRG be secure even though its output support has size at most 2^n inside a space of size 2^l?
2. Derive $2^{o \cdot 2^m}$ and $(2^m)!$. How does a random-function oracle answer a repeated input?
3. State what the eavesdropper receives. Why must the two challenge messages have equal, polynomially bounded lengths?
4. Reconstruct the PRG reduction and compute its output probability in both worlds. Where does the $1/2$ arise?
5. Explain the difference between distinguishing gap and hidden-bit guessing advantage. Why is small reduction overhead desirable?
6. Break the repeated-pad scheme with a chosen encryption of zero. Prove the deterministic-encryption CPA attack.
7. Derive CBC decryption from encryption. Which value replaces $c(-1)$, and which parts of the ciphertext are public?
8. Interpret the two terms in the CBC bound. Why does setting $q = 1$ not by itself imply a zero collision term for long messages?
9. Compare fixed-IV CBC and OFB. Why does CBC need unpredictability rather than merely uniqueness?
10. Pad 0, 5, 15, and 16 bytes for a 16-byte block size. Why is a full padding block sometimes necessary?
11. Derive the first two padding-oracle recovery steps. How do you reject an accidental match for longer padding?
12. Why does this attack not refute CPA security? What must be authenticated, and when must the check occur?

Chapter 3

Block ciphers: construction and cryptanalysis

A mode such as CBC relies on a block cipher behaving like a pseudorandom permutation (PRP). We now open that primitive: how can a short key describe a fast, invertible transformation that withstands observation and chosen inputs? Generic constructions explain how pseudorandom objects are related. Direct designs address efficiency, while cryptanalysis tests whether their internal structure creates shortcuts.

3.1 From functions to permutations: Feistel networks

3.1.1 Inverting a construction without inverting its function

Let $F_k : \{0,1\}^n \to \{0,1\}^n$ be a pseudorandom function (PRF). Here n is a half-block length. Split a $2n$-bit input into two halves L, R. A Feistel round with round key k_i maps

$$(L, R) \longmapsto (L', R') = (R, L \oplus F_{k_i}(R)).$$

The function F_{k_i} may have collisions and need not have an inverse. The unchanged half R, however, survives as L'. This lets us recompute the same function value when undoing the round.

Proposition 3.1 (Feistel inversion)**.** *Each Feistel round is a permutation, with inverse*

$$R = L', \qquad L = R' \oplus F_{k_i}(L').$$

Proof. Substituting $L' = R$ and $R' = L \oplus F_{k_i}(R)$ cancels the two copies of $F_{k_i}(R)$. Thus every output has exactly one input, regardless of whether F_{k_i} is injective. □

At the top of Figure 3.1, the right wire has two uses: it supplies the input to F and also continues unchanged towards the crossing. The left wire meets $F(R)$ at the circled XOR. After crossing, the old R is the new left half and the XOR result is the new right half. For example, if $L = 1010$, $R = 0110$, and this round gives $F(R) = 1101$, the output is $(0110, 0111)$. From that output, the inverse reads the old R as 0110, recomputes 1101, and recovers $L = 0111 \oplus 1101 = 1010$. The unchanged copy of R makes this possible even if F has collisions. The dotted final crossing records an output-order convention; it is not an extra function call.

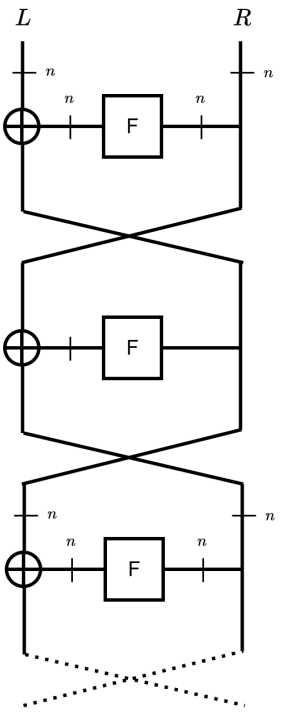


Figure 3.1: Copy the right half through the round function, XOR its result into the left half, then swap the two branches.

Algorithm 3.1 Feistel evaluation and inversion

Require: Halves L, R of n bits; round keys $k_1, \ldots, k_r$
Ensure: Forward evaluation or inversion of the r-round permutation

function FEISTEL($L, R, k_1, \ldots, k_r$)
  for $i = 1, \ldots, r$ **do**
    $(L, R) \leftarrow (R, L \oplus F_{k_i}(R))$ ▷ Simultaneous assignment
  end for
  return (L, R)
end function
function FEISTEL-INVERSE($L, R, k_1, \ldots, k_r$)
  for $i = r, r-1, \ldots, 1$ **do**
    $(L, R) \leftarrow (R \oplus F_{k_i}(L), L)$
  end for
  return (L, R)
end function

3.1.2 Security and the cost of the reduction

Theorem 3.2 (Three-round Feistel security, qualitative form)**.** *Three Feistel rounds with independently keyed secure PRFs yield a PRP against an adversary with forward queries only. At q queries, the reduction has the schematic bound*

$$\varepsilon_P \;\le\; O\!\left(\frac{q^2}{2^n}\right) + 3\varepsilon_F,$$

where ε_F bounds the distinguishing gaps for the round functions at the required resources. The O notation suppresses a constant factor in the collision term.

Each round spreads information between the halves. The proof compares a network of pseudorandom functions with one of independent random functions, then compares that network with a random permutation. Replacing the three functions accounts for three distinguishing terms; possible coincidences among half-block values account for the quadratic term. A full proof requires tracking these transcripts; inversion alone does not establish security.

Remark 3.3 (Which key and which interface?). If each round-function key has n bits, three independent keys require $3n$ secret bits. Writing a single key symbol for the entire network does not make them one n-bit key. Deriving round keys from a shorter master key requires its own justification. Likewise, three rounds give the forward PRP notion, whereas four independent PRF rounds support the stronger notion with both forward and inverse queries. The access model is part of the theorem.

Example 3.4 (Half-block size matters)**.** The permutation acts on $2n$ bits, but the displayed collision term has denominator 2^n. With $n = 64$, the term $q^2/2^{64}$ ceases to be small around $q = 2^{32}$. The construction's full 128-bit block size does not change that particular reduction bound into $q^2/2^{128}$.

Remark 3.5 (More rounds and tightness). A bound becoming large does not itself exhibit an attack. Tightness asks whether an attack can approach the bound at comparable resources. Additional Feistel rounds can support stronger bounds beyond the half-block birthday scale, under precise round and query assumptions. An abbreviated expression for "seven or more rounds" is not a universal formula: it must specify how q grows, the round independence, and the oracle interface. We retain the three-round bound above as the explicit guarantee used here.

3.1.3 Using a permutation as a function

A permutation already has the syntax of a function, but its ideal target differs: a random permutation never collides on distinct inputs, while a random function can. On a domain of 2^b blocks, the PRP-to-PRF switching bound gives

$$\varepsilon_F \le \varepsilon_P + \frac{q(q-1)}{2 \cdot 2^b}.$$

To understand the extra term, answer distinct queries with random outputs. Until a repeated output occurs, these answers can also be interpreted as drawing without replacement, as a permutation does. The probability of such a repeated output is bounded by summing over query pairs. Repeated *inputs* are answered consistently and do not create this difference. For the $2n$-bit Feistel domain, set $b = 2n$.

3.2 From generators to functions, and back

3.2.1 A binary tree of pseudorandom expansions

Let $G_n : \{0,1\}^n \to \{0,1\}^{2n}$ be a pseudorandom generator (PRG). Split its output into two n-bit halves:

$$G_n(s) = G_{n,0}(s) \| G_{n,1}(s).$$

To evaluate a function on an m-bit input $x = x(1) \cdots x(m)$, start from a secret root seed k and follow a path determined by those bits:

$$s_0 = k, \qquad s_i = G_{n,x(i)}(s_{i-1}), \qquad F_k(x) = s_m.$$

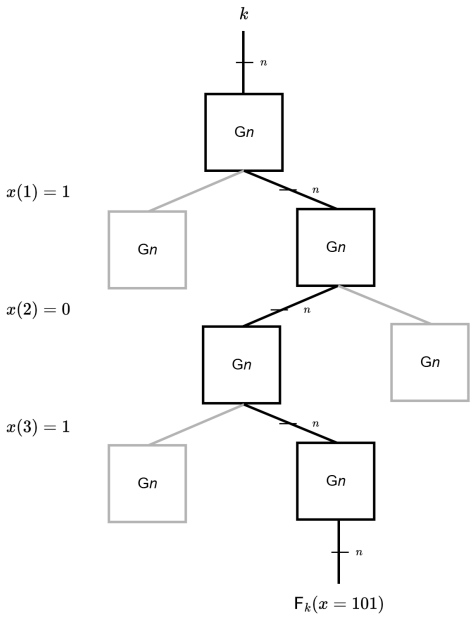


Figure 3.2: The input bits 101 choose right, left, then right among successive generator output halves.

This is the Goldreich–Goldwasser–Micali (GGM) construction. The entire binary tree describes a function, but evaluating one input visits only one path. Its cost is m generator evaluations, rather than constructing all 2^m leaves.

In Figure 3.2, a 0 means keep the left n-bit output half of the generator, and a 1 means keep the right half. Starting from $s_0 = k$, input 101 therefore computes

$$s_1 = G_{n,1}(k), \qquad s_2 = G_{n,0}(s_1), \qquad s_3 = G_{n,1}(s_2), \qquad F_k(101) = s_3.$$

The grey branches are the alternatives discarded along this evaluation, not additional random keys that must be sampled. Input 100 shares the first two choices and differs only at the last branch; repeating 101 follows exactly the same path. Read the drawing as a branching route through seeds; the formula and algorithm fix the evaluation count at three generator calls for three input bits, returning the reached seed without an additional terminal expansion.

Algorithm 3.2 GGM function evaluation

Require: Secret seed $k \in \{0,1\}^n$, input bits $x(1), \ldots, x(m)$
Ensure: The n-bit output $F_k(x)$

```
s ← k
for i = 1, ..., m do
    (s_left, s_right) ← the two halves of G_n(s)
    if x(i) = 0 then
        s ← s_left
    else
        s ← s_right
    end if
end for
return s
```

Remark 3.6 (Why a shared tree can imitate a random function). Two inputs with a common prefix share part of their path; repeating an input follows exactly the same path and returns the

same value. This supplies the consistency required of a function oracle. A hybrid experiment is an intermediate version of the experiment: some generated child seeds have been replaced by independent uniform ones, while the rest are still computed normally. Compare two consecutive versions, differing at just one expansion. A procedure able to distinguish these versions would test whether that expansion was a real generator output or a uniform pair of seeds. Repeating the comparison connects the real tree to the ideal behaviour. With at most q queries and depth m, a standard schematic loss is

$$\varepsilon_F \lesssim qm\,\varepsilon_{G_n}.$$

The factor reflects the number of places where the proof may need to replace an expansion, not a claim that attack events are independent. The distinguisher's simulation also has a computational cost.

In the reverse direction, a PRF generates a pseudorandom string by evaluating it on distinct encoded counters and concatenating the outputs. For example, if the input domain contains 0 and 1 and outputs have n bits, $F_k(0)\|F_k(1)$ gives $2n$ bits from the key. In the ideal-function world these are independent uniform outputs. Reusing a counter repeats a block, so distinct inputs are essential.

3.2.2 Expanding one extra bit into a longer output

Suppose a generator G_1 expands n bits to $n+1$ bits. Write its output as an updated n-bit state and one output bit:

$$G_1(s_{i-1}) = s_i\|y(i).$$

Starting from $s_0 = k$, iterate n times and output

$$G_n(k) = y(1)\|\cdots\|y(n)\|s_n.$$

The retained state contributes n bits and the extracted bits contribute another n. A generator with greater initial stretch can first be truncated to $n+1$ output bits.

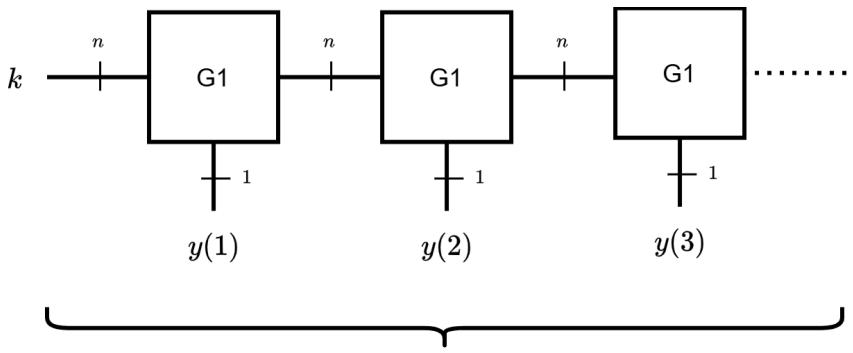


Figure 3.3: Each box emits one output bit downward and passes an updated n-bit state to the next box on the right.

In Figure 3.3, the horizontal arrow is not another one-bit output: it carries the entire next state. For a seed of $n = 3$ bits, three boxes emit $y(1), y(2), y(3)$ and leave a final three-bit state s_3. The doubling construction returns the six bits $y(1)\|y(2)\|y(3)\|s_3$. Emitting only the three downward bits would give no length expansion in this example. The label G_3 under the brace names the three-step illustration; the general G_n uses n such steps. Every invocation uses the same public algorithm G_1 on a changing state.

Algorithm 3.3 A doubling generator from a one-bit-stretch generator

Require: Seed $k \in \{0,1\}^n$ and generator $G_1 : \{0,1\}^n \to \{0,1\}^{n+1}$
Ensure: A $2n$-bit pseudorandom output, assuming G_1 is secure
1: $s \leftarrow k$
2: **for** $i = 1, \ldots, n$ **do**
3: Parse $G_1(s)$ as $s' \| y(i)$ with $|s'| = n$ and $|y(i)| = 1$
4: $s \leftarrow s'$
5: **end for**
6: **return** $y(1) \| \cdots \| y(n) \| s$

A hybrid argument replacing one expansion at a time gives a loss of at most n generator distinguishing gaps at suitable simulation resources: $\varepsilon_{G_n} \lesssim n\varepsilon_{G_1}$. Conversely, projecting a secure doubling generator onto $n+1$ output bits gives a one-bit-stretch generator: a test on the projection would also test the full output. Keeping just one output bit would not be a length-expanding generator.

3.2.3 The efficiency challenge

Example 3.7 (Following the cost through the constructions)**.** Suppose $n = m = 128$, one G_1 evaluation takes 1 millisecond, and we use 16 Feistel rounds. Each round calls the GGM function, whose path has 128 generator invocations; each doubling generator invokes G_1 128 times. Thus one block costs approximately

$$16 \cdot 128 \cdot 128 = 2^{18} \text{ milliseconds} = 262.144 \text{ seconds} \approx 4.37 \text{ minutes}.$$

This is a cost calculation for the hypothetical construction, not a performance estimate for the Advanced Encryption Standard (AES). With 128-bit halves, its block size is 256 bits.

The generic constructions establish important implication chains, but their nested costs can be unsuitable for practical encryption. Direct block-cipher design seeks a fast PRP without implementing every layer of that chain. Its confidence must then come from the design rationale and sustained cryptanalysis; a short execution time is not itself evidence of pseudorandomness.

3.3 Why easy descriptions need careful design

3.3.1 A random table is too large

A uniformly chosen permutation on 128-bit blocks could be stored as a lookup table, but the table would require 2^{128} entries of 128 bits:

$$128 \cdot 2^{128} = 2^{135} \text{ bits} = 2^{132} \text{ bytes}.$$

A table can answer a query by reading one entry, but the storage is already infeasible. For contrast, even a table on just 8-bit blocks needs 256 one-byte entries; moving to 128 bits changes the number of entries to 2^{128}, not merely 128 times as many. The conceptual ideal permutation has no requirement to admit a compact description. A real cipher uses a compact algorithm and a short key to select a tiny family of permutations that an efficient observer should nevertheless fail to distinguish from that ideal.

3.3.2 A linear or affine map is too transparent

Let $\mathbb{F}_2 = \{0, 1\}$ be the field with addition XOR and multiplication AND. Bit strings can be treated as vectors over this field. For a public invertible binary matrix A, consider

$$y = A(x \oplus k) = Ax \oplus Ak.$$

For fixed k, this is an affine map: a linear transformation followed by a constant shift. One known pair (x, y) reveals $k = x \oplus A^{-1}y$. The permutation is invertible, but its algebraic structure exposes the key.

Proposition 3.8 (Recovering an unknown affine map)**.** *Even when the matrix and shift in $E(x) = Ax \oplus b$ are unknown, $n + 1$ chosen inputs determine the entire map on n-bit blocks.*

Proof. Query 0 to obtain b. For the vector e_i with a single 1 in position i, $E(e_i) \oplus b = Ae_i$ is column i of A. These columns determine the matrix. Gaussian elimination then inverts an invertible A in $O(n^3)$ elementary field operations. □

Remark 3.9 (Counting linear maps correctly). An arbitrary $n \times n$ binary matrix has n^2 freely chosen bits, giving 2^{n^2} linear maps. The number of invertible ones is

$$|\operatorname{GL}(n, 2)| = \prod_{i=0}^{n-1} (2^n - 2^i).$$

The span of chosen columns is the set of their XOR combinations. With no column yet chosen, only the zero vector is excluded, leaving $2^n - 1$ choices for the first column. One chosen nonzero column excludes zero and itself, leaving $2^n - 2$ choices for the next. Each further independent column doubles the number of existing combinations; this gives the successive factors in the product. Allowing an arbitrary affine shift multiplies this count by 2^n. These counts describe a family of maps; the chosen-input attack explains why even a large such family is unsuitable as a PRP.

3.4 Key alternation, confusion, and diffusion

Definition 3.10 (S-box and round key)**.** A substitution box (S-box) is a small nonlinear function applied to a group of bits. In the substitution layers here, it is a permutation on b-bit groups. A round key is a subkey used at a particular stage; a key schedule derives the round keys from a master key. A key-alternating cipher interleaves key XORs with public transformation layers.

Nonlinear substitutions complicate the relation between key and observations, traditionally called *confusion*. *Diffusion* spreads dependencies across groups: it prevents an attacker from treating the message as several unrelated small ciphers. These are design objectives, not standalone security theorems.

Pick the leftmost four top wires in Figure 3.4. They enter the first S-box, remain in the leftmost four positions, and enter the leftmost S-box in the next round. Changing a key bit used only in that lane cannot affect any of the other twelve output bits. An attacker can therefore test that lane's candidate key bits against its own four-bit observations, then attack the other lanes separately. Making each local S-box nonlinear does not remove this separation. The following search estimate counts this structural saving, with the number of unknown key bits stated explicitly.

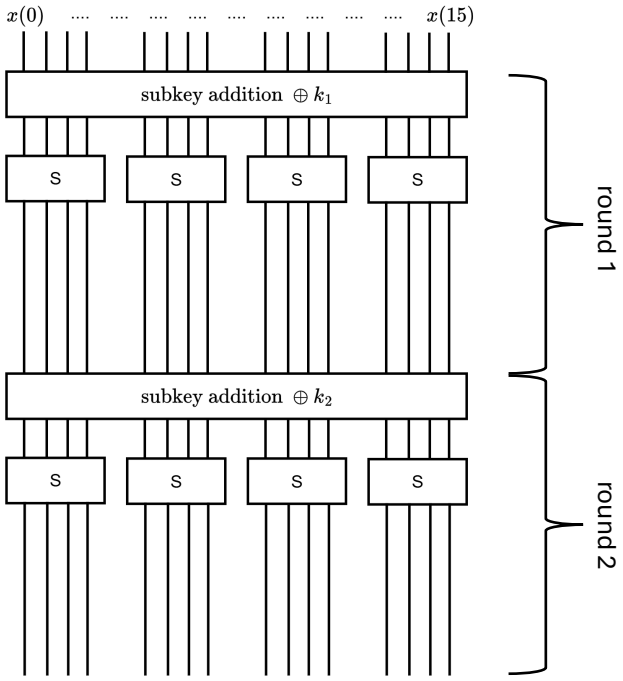


Figure 3.4: Straight wires keep four groups separate, so information in the first group never reaches the other three groups.

Example 3.11 (A large key split into independent small searches)**.** If a 16-bit unknown subkey splits into four independent 4-bit pieces, each recoverable from its own input/output group, a search can test 2^4 values for each piece: about $4 \cdot 2^4 = 64$ local candidates instead of enumerating 2^{16} full keys. Sufficient observations are needed to verify candidates. If both displayed 16-bit round keys are independently unknown, there are instead 8 unknown key bits per lane, and the analogous separated search is $4 \cdot 2^8$ versus a full 2^{32} search. The exact number depends on which keys are unknown; the weakness is that the lanes can be attacked separately.

Compare Figure 3.5 with the straight-wire version. Follow the four output wires from the leftmost first-round S-box: the wiring distributes them among different next-round groups. Each of those next S-boxes can now combine a bit from this group with bits arriving from other groups. Conversely, predicting one next-round output may require information from several previous groups, so the earlier four independent searches no longer apply in the same way. Crossing wires is only a reordering; the next nonlinear substitutions are what combine the redistributed dependencies.

3.5 Algebraic cryptanalysis

3.5.1 Turning observations into equations

An algebraic attack represents the cipher as equations in unknown key bits and known input/output bits. Over $\mathbb{F}_2$, XOR is addition and a product of bits is their AND. Boolean variables satisfy $a^2 = a$, so powers can be reduced when writing these polynomials.

For a small example, let x, k have two bits, set $y = x \oplus k$, and consider

$$z(0) = y(0)y(1), \qquad z(1) = y(0) \oplus y(1).$$

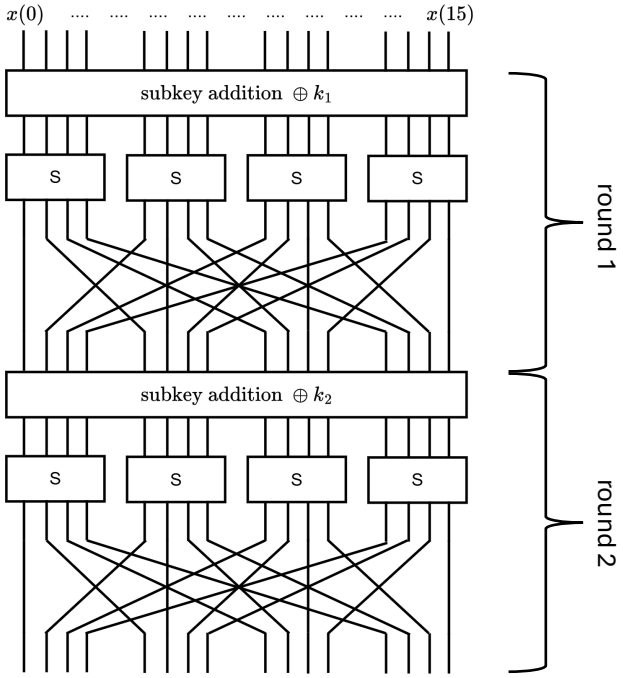


Figure 3.5: The crossed wires send bits from one S-box to several different S-boxes in the next round.

Expanding the first equation gives

$$z(0) = x(0)x(1) \oplus x(0)k(1) \oplus x(1)k(0) \oplus k(0)k(1),$$
$$z(1) = x(0) \oplus x(1) \oplus k(0) \oplus k(1).$$

The second output uses $y(0) \oplus y(1)$: writing $y(1) \oplus y(1)$ would always give zero and would contradict the expanded expression. This two-bit Boolean map illustrates equations, but is not an invertible S-box: inputs 01 and 10 have the same output.

Definition 3.12 (Algebraic degree)**.** The algebraic normal form of a Boolean function is its unique XOR sum of square-free monomials in the input bits. Its algebraic degree is the largest number of distinct variables in a monomial with nonzero coefficient. A vector-valued function's degree is the maximum degree of its coordinate functions.

Example 3.13 (Linearising the quadratic key term)**.** In the example, introduce a temporary variable $v = k(0)k(1)$. Each known pair (x, z) now gives a linear equation in $k(0), k(1), v$:

$$x(1)k(0) \oplus x(0)k(1) \oplus v = z(0) \oplus x(0)x(1).$$

Several inputs may supply enough independent equations to solve this enlarged linear system. For $k = (1, 0)$, the inputs $(0, 0), (1, 0), (0, 1)$ give respectively $v = 0$, $k(1) \oplus v = 0$, and $k(0) \oplus v = 1$, recovering the key. Candidate solutions must still satisfy $v = k(0)k(1)$ and the original cipher equations. Introducing an extra variable relaxes the system; it does not automatically create an extra independent observation.

3.5.2 Degree growth and the number of monomials

If a round's nonlinear layer has degree at most d and its other layers are affine, after r rounds the degree is at most d^r, also capped by the number of input variables. For a permutation on

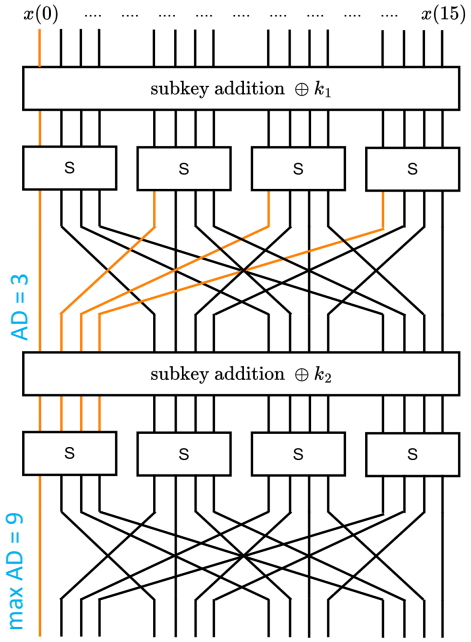


Figure 3.6: AD denotes algebraic degree: first-layer degree at most 3 can grow to at most 9 after another nonlinear layer.

$N \geq 2$ bits, each coordinate is balanced and its degree is at most $N - 1$. These are upper bounds: cancellations and limited diffusion can prevent reaching them.

The orange paths in Figure 3.6 indicate how a variable can influence several intermediate groups. Why might degree reach 9? A second-layer S-box can multiply three input expressions, each already of degree at most 3. A representative product is

$$(x_1x_2x_3)(x_4x_5x_6)(x_7x_8x_9),$$

which has degree 9. This requires sufficiently different dependencies. If the three factors instead repeat the same three variables, Boolean identities such as $x_i^2 = x_i$ collapse the product to degree 3. Diffusion makes new combinations possible, while actual cancellations still have to be checked. The labels AD = 3 and max AD = 9 distinguish the local degree from the composition's upper bound.

Proposition 3.14 (Counting monomials)**.** *In N Boolean variables there are $\binom{N}{d}$ square-free monomials of degree exactly d, and $\sum_{i=0}^{d}\binom{N}{i}$ of degree at most d. For a uniformly chosen Boolean function, each algebraic-normal-form coefficient is uniformly random, so the expected number present at degree d is $\frac{1}{2}\binom{N}{d}$.*

Proof. A degree-d monomial is determined by choosing its d distinct variables. The conversion from truth tables to algebraic-normal-form coefficients is a bijection, so uniform truth tables yield uniform coefficient vectors. □

For example, with four Boolean variables there are six quadratic products:

$$x_1x_2, \quad x_1x_3, \quad x_1x_4, \quad x_2x_3, \quad x_2x_4, \quad x_3x_4.$$

Linearisation may treat these as additional unknowns, even though they ultimately depend on only four bits. With larger N and higher degree, the same counting effect can make the relaxed linear system enormous. For even N, the central binomial coefficient satisfies

$\binom{N}{N/2} \sim 2^N/\sqrt{\pi N/2}$. Thus a linearisation involving many degrees can introduce exponentially many monomial variables. Increasing degree and spreading dependencies obstruct the simple attack above. Degree alone remains a proxy: a high-degree expression might still have a special structure an attacker can exploit.

3.6 Linear cryptanalysis: finding a statistical imbalance

3.6.1 Approximations and bias

Linear cryptanalysis searches for a parity relation that holds more or less often than a fair coin would predict. It does not assert that the nonlinear cipher is exactly linear. For bit vectors a, x, write $a \cdot x = \bigoplus_i a(i)x(i)$ for their binary inner product: the mask a selects the bits whose XOR is taken.

Definition 3.15 (Linear approximation and bias)**.** For an S-box S and nontrivial masks a, b, consider $a \cdot Y = b \cdot S(Y)$ with uniform input Y. If it holds with probability p, its bias is $\epsilon = p - 1/2$ and its correlation is $C = 2p - 1 = 2\epsilon$. Positive bias favours equality; negative bias favours inequality. The strength is measured by the absolute value, so a large negative bias is also useful.

Proposition 3.16 (Piling-up lemma)**.** *Let independent error bits $E_1, \ldots, E_h$ satisfy $P(E_i = 0) = 1/2 + \epsilon_i$. Then*

$$P(E_1 \oplus \cdots \oplus E_h = 0) - \frac{1}{2} = 2^{h-1} \prod_{i=1}^{h} \epsilon_i.$$

Proof. Write $\mathbb{E}$ for expectation. For each bit, $\mathbb{E}[(-1)^{E_i}] = 2\epsilon_i$. Independence makes the expectation of the product equal the product of expectations. The product is $(-1)^{E_1 \oplus \cdots \oplus E_h}$, whose expectation is twice the desired bias. □

In Figure 3.7, read the wire labels vertically: x is before the first key XOR, y after it, z after the first S-box layer, u after the next key XOR, and v after the next S-box layer. An orange wire means its bit participates in a parity test; it does not mean the bit equals 1 or is known to the attacker. Begin with the top selected bit: $y(1) = x(1) \oplus k_1(1)$. The first approximation replaces this by $z(0) \oplus z(3)$. The wiring and next key XOR express that parity through $u(0), u(12)$ and two key bits. The last two approximations replace those selected u bits by $v(0), v(12)$, producing the formula below. Each replacement can be wrong on some inputs; the bias calculation measures how often their combined parity is right.

For the illustrated toy network, suppose the relations $y(1) = z(0) \oplus z(3)$, $u(0) = v(0)$, and $u(12) = v(12)$ have biases $\epsilon_1, \epsilon_2, \epsilon_3$. Combining them through the wiring and key XORs gives the approximation

$$x(1) \oplus k_1(1) = v(0) \oplus k_2(0) \oplus v(12) \oplus k_2(12).$$

Under the independent-error model its bias is $4\epsilon_1\epsilon_2\epsilon_3$. If all three are 2^{-2}, the resulting bias is 2^{-4}, or equality probability 9/16. This differs from 1/2 by only 1/16, but repeated observations can reveal the imbalance.

Remark 3.17 (Why a key XOR can preserve a useful signal). For a fixed key, XOR with a key bit either preserves a parity relation or flips it. A bias of $+\epsilon$ may become $-\epsilon$, but its magnitude survives. This is why an attacker often measures absolute empirical bias while testing key guesses. The piling-up lemma needs independence; errors inside a real cipher cannot simply be assumed independent. A single-trail product describes one chosen sequence of intermediate masks; the full approximation may include many such trails.

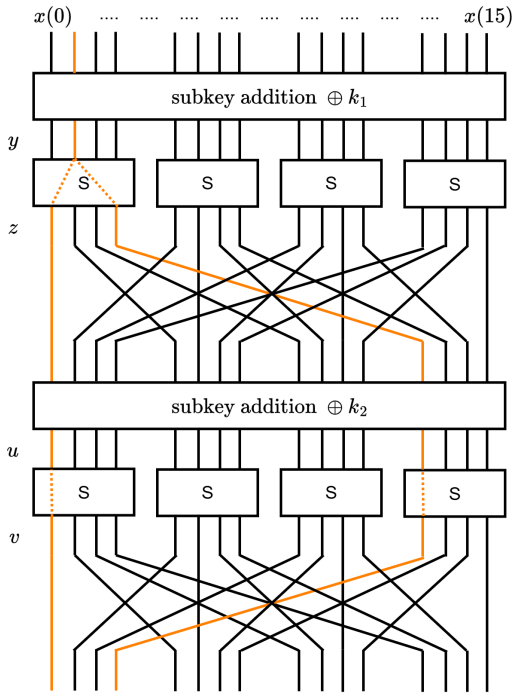


Figure 3.7: Orange wires select bits for three local parity tests, which are combined into one approximate relation across two rounds.

3.6.2 Using the bias to rank key guesses

Suppose plaintext blocks x and ciphertext blocks v are observed. Guess the four bits of the first subkey entering one S-box and compute its candidate output $z^* = S(x_{\text{group}} \oplus k^*)$. Here x_{group} is the selected four-bit plaintext group and k^* the guessed nibble. The remaining $r-1$ rounds are represented by a biased relation rather than fully decrypted.

Only the small orange key group is guessed in Figure 3.8; the grey rectangle hides the remaining rounds rather than assuming their keys are known. For each candidate, the attacker combines its known plaintext group with that guess, applies the public S-box, and obtains candidate z^* bits. The ciphertext supplies the selected v bits. Counting how often the displayed parity is zero produces that candidate's score. For example, out of 256 observed pairs, counts of 144 and 130 give biases $1/16$ and $1/128$ respectively. The first candidate has the stronger signal in this data set, but further data and key verification are needed before accepting it. This is how an approximate relation becomes a key-search aid without decrypting all the grey rounds.

For the illustrated relation, estimate

$$\widehat{\epsilon}_{k^*} = \frac{\#\{\text{pairs with } z^*(1) \oplus z^*(3) \oplus v(0) \oplus v(4) \oplus v(8) \oplus v(9) = 0\}}{q} - \frac{1}{2}.$$

All displayed operations inside the parity are XORs. Rank guesses by $|\widehat{\epsilon}_{k^*}|$. The intended approximation should become visible for the correct guess, while incorrect guesses are hoped to obscure it. This is a statistical ranking procedure: it does not guarantee that every incorrect guess has zero bias or that one data set always ranks the true key first.

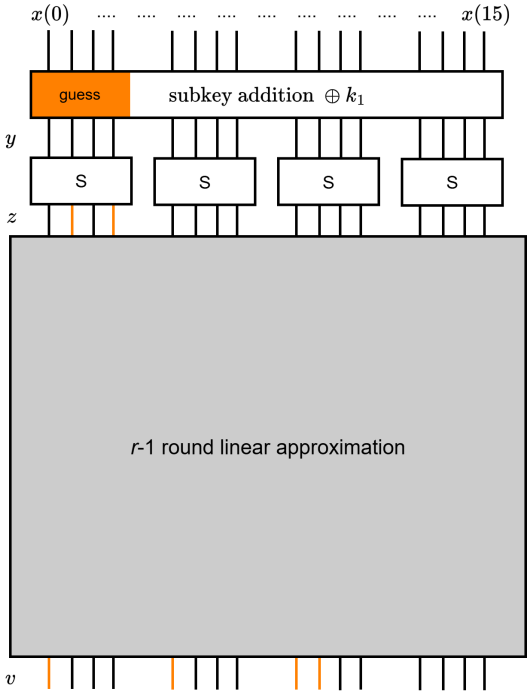


Figure 3.8: Guess four key bits to calculate selected z bits, then compare their parity with observable ciphertext bits.

Algorithm 3.4 Ranking a four-bit subkey using a linear approximation

Require: q observed plaintext/ciphertext pairs (x_j, v_j), a known S-box S
Require: The selected input group and the displayed parity approximation
Ensure: Subkey candidates ranked by absolute empirical bias

for each four-bit candidate $k^* \in \{0,1\}^4$ **do**
  $N_{k^*} \leftarrow 0$
  for $j = 1, \ldots, q$ **do**
    $z_j^* \leftarrow S(x_{j,\text{group}} \oplus k^*)$
    $e \leftarrow z_j^*(1) \oplus z_j^*(3) \oplus v_j(0) \oplus v_j(4) \oplus v_j(8) \oplus v_j(9)$
    if $e = 0$ **then**
      $N_{k^*} \leftarrow N_{k^*} + 1$
    end if
  end for
  $\widehat{\epsilon}_{k^*} \leftarrow N_{k^*}/q - 1/2$
end for
return candidates in decreasing order of $|\widehat{\epsilon}_{k^*}|$

Proposition 3.18 (Why the data scale is inverse squared bias)**.** *For q independent observations of a Bernoulli event, the empirical frequency has standard deviation $\sqrt{p(1-p)/q} \leq 1/(2\sqrt{q})$. Resolving a bias of magnitude $|\epsilon|$ at a fixed signal-to-noise ratio therefore requires on the order of $1/\epsilon^2$ observations.*

For $|\epsilon| = 1/16$, this scale is 256 pairs. The proportionality constant depends on the desired confidence, the separation between key scores, and how many candidates are compared. Additional approximations target other subkey groups; the surviving full-key candidates must be verified. Repeatedly counting identical pairs does not create independent evidence.

3.7 A worked SPN: from an S-box table to a linear key ranking

3.7.1 Fixing the cipher and the indexing

A substitution-permutation network (SPN) alternates parallel S-boxes, a permutation of bit positions, and round-key XORs. To calculate an attack rather than just describe its shape, fix a 16-bit example with four rounds and five 16-bit subkeys $K_1, \ldots, K_5$. In this worked example, bits are numbered *1 to 16 from left to right*, so the first bit is the most significant bit. This differs from the zero-based wire labels used in some earlier diagrams.

Let P be the plaintext block and C the ciphertext block. Write U_i for the input to round i's S-box layer and V_i for its output. The first input is $U_1 = P \oplus K_1$. Each S-box acts on one four-bit group using the following public permutation, with entries in hexadecimal:

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

For rounds 1 to 3, a wiring permutation ρ sends bit $j = 4a + b + 1$ to position $4b + a + 1$, where $0 \le a, b < 4$. Thus $U_{i+1} = \rho(V_i) \oplus K_{i+1}$. The last round has no wiring permutation and ends with $C = V_4 \oplus K_5$. The final key XOR is often called *whitening*. Without it, anyone could apply the public inverse S-boxes directly to the ciphertext and remove the last substitution layer.

Follow Figure 3.9 from $P_1, \ldots, P_{16}$ at the top. In S_{ij}, the first index is the round and the second is the four-bit group. Each crossed-wire region redistributes S-box output bits to different next-round boxes. At the bottom, the extra K_5 XOR changes the public value V_4 into the observed ciphertext. Given a guess for a small group of K_5 bits, the attacker can undo just those XORs and S-boxes, recovering candidate bits of U_4 without knowing the full key. This is the last-round counterpart of the earlier first-round guessing attack. For analysing the example, the round keys are treated as independent; recovering eight bits of K_5 does not recover five independent 16-bit keys.

3.7.2 Computing a linear approximation table

Definition 3.19 (Count-offset LAT convention)**.** For a b-bit S-box, define the linear approximation table (LAT) by

$$\mathrm{LAT}[a, b'] = \#\{x : a \cdot x = b' \cdot S(x)\} - 2^{b-1}.$$

Here a, b' are input and output masks, and the dot selects the parity of the masked bits. The prime on b' distinguishes a mask from the S-box width b. In this convention, bias is $\mathrm{LAT}[a, b']/2^b$ and correlation is twice that bias. Other conventions store a Walsh sum or a probability, so the numerical convention must be stated before reading a table.

For our four-bit box, hexadecimal mask $6 = 0110$ selects $X_2 \oplus X_3$, while mask $B = 1011$ selects $Y_1 \oplus Y_3 \oplus Y_4$. Enumerate all 16 inputs, evaluate $S(x)$, and count equality. This gives 12 matches, hence LAT entry $12 - 8 = 4$, equality probability $3/4$, bias $1/4$, and correlation $1/2$.

Input mask	Output mask	Matches	LAT entry	Bias
6	B	12	4	$1/4$
9	4	8	0	0
3	9	2	-6	$-3/8$
B	4	12	4	$1/4$
4	5	4	-4	$-1/4$

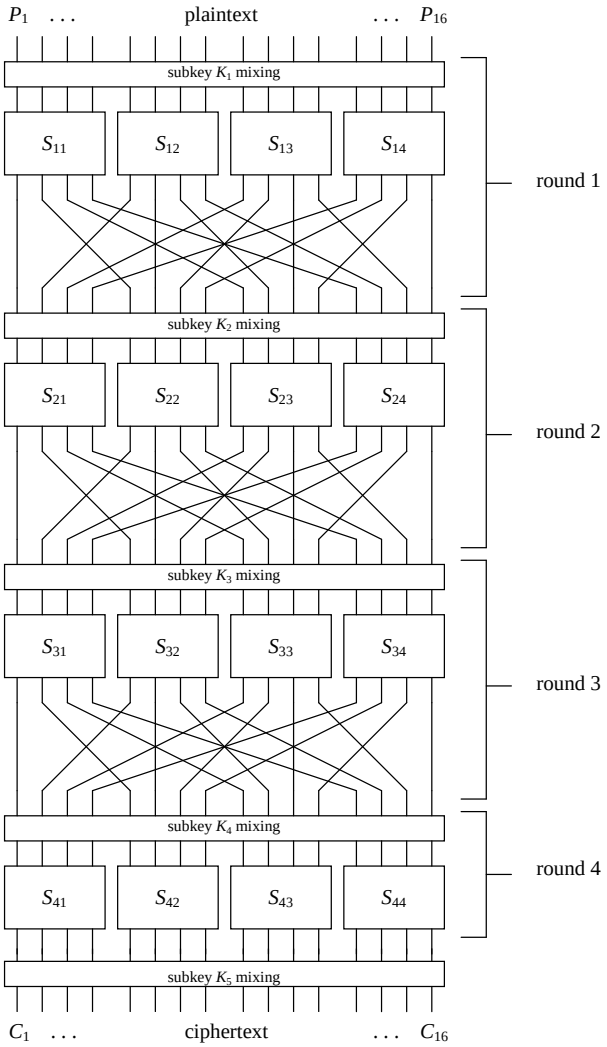


Figure 3.9: Four substitution rounds, three wiring layers and five key XORs; the final key hides the input of the last S-box layer.

For example, input mask 3 and output mask 9 mean $X_3 \oplus X_4 = Y_1 \oplus Y_4$. It is rarely true, but its *negation* is true in 14 of 16 cases, so the negative entry is a strong signal. The entry $(0, 0)$ is 8 because $0 = 0$ always holds; the other entries in row or column zero are zero for a permutation, since a nonzero parity on uniform input or output is balanced.

3.7.3 Cancelling intermediate parities across three rounds

Use the approximation with masks $(B, 4)$ in S_{12}, then $(4, 5)$ in S_{22}, S_{32}, S_{34}. Their biases are $+1/4, -1/4, -1/4, -1/4$. The first relation is

$$U_{1,5} \oplus U_{1,7} \oplus U_{1,8} \;\approx\; V_{1,6}.$$

The symbol $\approx$ here denotes a statistical parity relation, not numerical closeness. Wiring carries $V_{1,6}$ into position 6 before the next key XOR, so $U_{2,6} = V_{1,6} \oplus K_{2,6}$. Combining with $U_{2,6} \approx V_{2,6} \oplus V_{2,8}$ eliminates that intermediate $V_{1,6}$. In round 3 the two selected boxes yield parities on $V_{3,6}, V_{3,8}, V_{3,14}, V_{3,16}$. Wiring and the next key addition convert these into four selected U_4 bits.

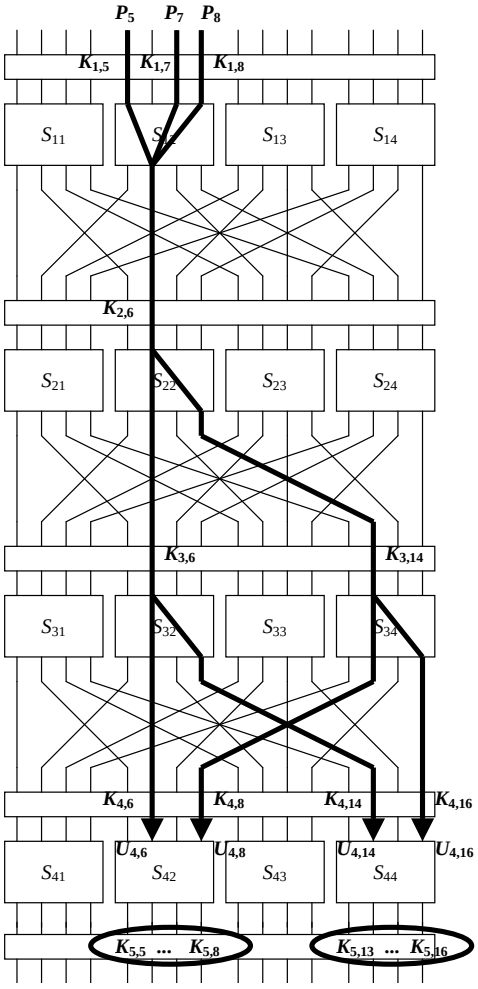


Figure 3.10: Bold wires select the parities whose intermediate bits cancel; the bottom ellipses identify two guessed groups of the final subkey.

In Figure 3.10, begin at the three bold plaintext bits 5, 7 and 8. Follow their parity through the selected S-box in each of the first two rounds, then through two boxes in round 3. Thin wires still carry data, but their bits are not selected in this approximation. After accounting for the

displayed key XORs, the relation is

$$P_5 \oplus P_7 \oplus P_8 \oplus U_{4,6} \oplus U_{4,8} \oplus U_{4,14} \oplus U_{4,16} \oplus \sigma_K \;\approx\; 0,$$

where the fixed key parity is

$$\begin{aligned}\sigma_K = {} & K_{1,5} \oplus K_{1,7} \oplus K_{1,8} \oplus K_{2,6} \oplus K_{3,6} \oplus K_{3,14} \\ & \oplus K_{4,6} \oplus K_{4,8} \oplus K_{4,14} \oplus K_{4,16}.\end{aligned}$$

Under the independent-error estimate, piling up the four approximations gives bias

$$2^3(1/4)(-1/4)^3 = -1/32.$$

With σ_K omitted, equality therefore has estimated probability 15/32 or 17/32, according to its unknown value. This explains why the attacker looks for an absolute bias, and why several moderate local approximations can leave only a small global signal. Dependencies and other trails can change the actual fixed-key bias; the product is a trail estimate rather than a full-cipher guarantee.

3.7.4 Guess only the final-key bytes the relation needs

The needed U_4 bits lie in S-box groups 2 and 4. Write $C^{(j)}$ for ciphertext nibble j, and guess $a = K_5^{(2)}$, $b = K_5^{(4)}$. Each candidate pair gives

$$u_2^* = S^{-1}(C^{(2)} \oplus a), \qquad u_4^* = S^{-1}(C^{(4)} \oplus b).$$

The selected bits 6 and 8 are positions 2 and 4 inside nibble 2; positions 14 and 16 are positions 2 and 4 inside nibble 4. Their nibble mask is therefore $5 = 0101$. Count how often

$$P_5 \oplus P_7 \oplus P_8 \oplus (5 \cdot u_2^*) \oplus (5 \cdot u_4^*) = 0.$$

Only $2^8 = 256$ candidates are tested, not every full subkey. With q known pairs and equality count $N_{a,b}$, rank candidates by $|N_{a,b}/q - 1/2|$. For example, count 5336 among 10000 samples gives bias magnitude 0.0336, close to the estimated 1/32. Count 4664 gives the same magnitude with the opposite sign. These numbers illustrate the score, not a guaranteed rank or a claim that the remaining key bits have been recovered.

Algorithm 3.5 Last-round linear ranking on the worked SPN

Require: Known plaintext/ciphertext pairs (P_j, C_j), public inverse S-box
Ensure: Ranked candidates for final-key nibbles 2 and 4

for $a, b \in \{0, \ldots, 15\}$ **do**
  $N_{a,b} \leftarrow 0$
  for each observed pair (P, C) **do**
    $u_2 \leftarrow S^{-1}(C^{(2)} \oplus a)$; $u_4 \leftarrow S^{-1}(C^{(4)} \oplus b)$
    $e \leftarrow P_5 \oplus P_7 \oplus P_8 \oplus (5 \cdot u_2) \oplus (5 \cdot u_4)$
    if $e = 0$ **then**
      $N_{a,b} \leftarrow N_{a,b} + 1$
    end if
  end for
  score$(a, b) \leftarrow |N_{a,b}/q - 1/2|$
end for
return candidates ranked by decreasing score; verify surviving full keys

The earlier estimate q of order $1/\epsilon^2$ gives a scale 1024 for $|\epsilon| = 1/32$. Higher confidence and competition among 256 correlated scores can require a larger multiple. An incorrect guess need not have exactly zero bias. Ranking a partial key, proving a distinguishing advantage, and recovering the whole master key are three separate achievements.

3.8 Differential cryptanalysis: following chosen XOR differences

3.8.1 What changes between two encryptions?

Definition 3.20 (XOR difference and differential)**.** For two inputs X, X', define $\Delta X = X \oplus X'$. Their outputs have difference $\Delta Y = Y \oplus Y'$. A differential specifies an input difference α and an output difference β; for a fixed function f, its probability is

$$P\big(f(X) \oplus f(X \oplus \alpha) = \beta\big)$$

with X uniform unless another input distribution is stated. A differential trail, also called a characteristic, additionally specifies the intermediate differences at the round boundaries.

The attacker chooses $P' = P \oplus \alpha$ and obtains both encryptions under the same key. Unlike linear cryptanalysis on ordinary known pairs, this basic attack needs *chosen pairs* with the desired difference. Here "difference" means XOR, not integer subtraction: $0011 \oplus 0101 = 0110$ regardless of which value is considered first.

A key XOR cancels in the pair:

$$(X \oplus k) \oplus (X' \oplus k) = X \oplus X'.$$

It does not make the two actual states known. It says that adding the same round key preserves their XOR difference. The nonlinear S-box can change that difference with a distribution depending on ΔX.

For an ideal random permutation on N bits and distinct fixed inputs, output difference zero is impossible; every fixed nonzero output difference has probability $1/(2^N - 1)$, averaging over that ideal permutation. The familiar 2^{-N} is a large-domain approximation. A fixed concrete permutation need not distribute its differences exactly at this ideal average.

3.8.2 Counting an S-box's difference distribution

Definition 3.21 (Difference distribution table)**.** For a b-bit S-box, its difference distribution table (DDT) is

$$\mathrm{DDT}[\alpha, \beta] = \#\{x : S(x) \oplus S(x \oplus \alpha) = \beta\}.$$

Dividing the count by 2^b gives the transition probability for uniform x. Unlike a LAT entry, this count is nonnegative and is not offset by half the domain size.

For the worked S-box, transition $B \to 2$ has count 8. In hexadecimal, its matching base inputs are

$$0, \quad 1, \quad 3, \quad 6, \quad 8, \quad A, \quad B, \quad D.$$

For instance, $0 \oplus B = B$, and $S(0) \oplus S(B) = E \oplus C = 2$. The reversed base B produces the same unordered pair and is also counted, so eight matching bases represent four unordered pairs. The probability is $8/16 = 1/2$, not $4/16$.

ΔX	ΔY	DDT count	Probability
B	2	8	1/2
4	6	6	3/8
2	5	6	3/8
8	B	4	1/4
4	A	0	0

Every row sums to 16. For a permutation, every column also sums to 16, $\mathrm{DDT}[0,0] = 16$, and other entries in row or column zero vanish. For $\alpha \neq 0$, matching inputs come in pairs $x, x \oplus \alpha$, so every entry in that row is even. These constraints explain why an idealised table containing a 1 in every cell is impossible.

Algorithm 3.6 Enumerating LAT and DDT entries for a public S-box

Require: A b-bit S-box S; dot denotes binary parity
Ensure: Count-offset LAT and ordinary nonnegative DDT

```
for each mask pair (a, d) do
    LAT[a, d] ← −2^{b−1}
    for each b-bit input x do
        if a · x = d · S(x) then
            Increment LAT[a, d]
        end if
    end for
end for
Initialise all DDT entries to zero
for each input difference α and each input x do
    β ← S(x) ⊕ S(x ⊕ α)
    Increment DDT[α, β]
end for
```

3.8.3 A three-round difference trail

Use the same four-round SPN as before. Choose plaintext difference $\Delta P = \texttt{0B00}$. The trail uses transitions $B \to 2$ in S_{12}, $4 \to 6$ in S_{23}, and $2 \to 5$ in both S_{32} and S_{33}. Nibbles with zero difference stay zero through a permutation S-box.

In Figure 3.11, begin with the second plaintext nibble, whose difference is $B = 1011$. The first selected S-box outputs difference 2; the wiring moves that active bit into nibble 3 of the next round. Follow the next selected transition and its two output bits: after redistribution, they activate nibbles 2 and 3 in round 3. The full boundary calculation is

Round	Input difference ΔU_i	S-box output ΔV_i	After wiring
1	0B00	0200	0040
2	0040	0060	0220
3	0220	0550	0606

Thus the predicted difference entering the final S-box layer is $\Delta U_4 = \texttt{0606}$. Round keys do not appear in this difference table because the same key is XORed into both members of each pair. Under the usual independent-transition estimate, the selected trail has probability

$$p_D \approx \frac{8}{16}\frac{6}{16}\left(\frac{6}{16}\right)^2 = \frac{27}{1024} \approx 0.02637.$$

This is the probability of following the specified intermediate path in that approximation. It is not a bias around 1/2, and it is not automatically the exact fixed-key probability of the external differential.

3.8.4 Filtering and partially decrypting chosen pairs

If the predicted ΔU_4 occurs, final S-boxes 1 and 3 receive identical inputs in the two encryptions. Their ciphertext nibbles must therefore be equal as well, since the output XOR with K_5 cancels.

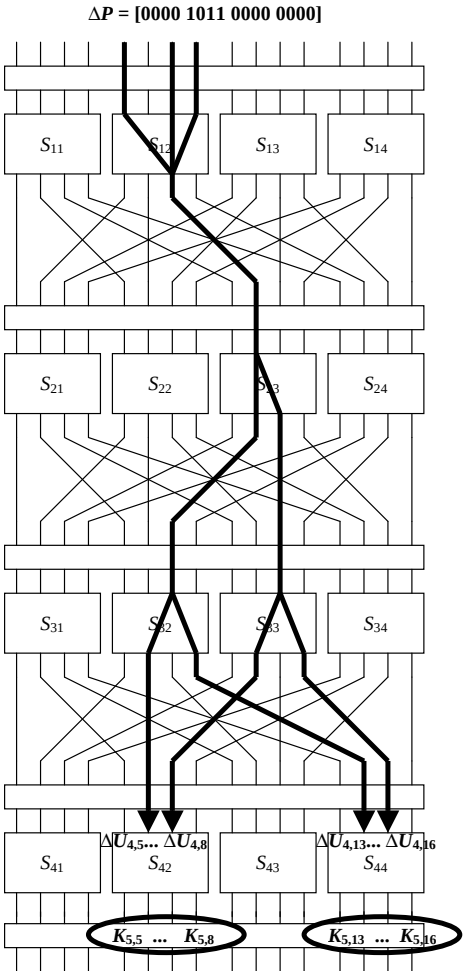


Figure 3.11: Bold wires mark nonzero differences through three rounds; the resulting last-round input differences affect only groups 2 and 4.

Discard pairs with $\Delta C^{(1)} \neq 0$ or $\Delta C^{(3)} \neq 0$ before guessing any key bits. This is a *necessary* test for this endpoint difference, not a proof that a surviving pair followed the chosen internal path.

For each surviving pair and candidate nibbles (a, b) of K_5, invert the last S-boxes on groups 2 and 4 as in the linear attack. Now do it for *both* ciphertexts and compare the candidate differences with 6:

$$S^{-1}(C^{(2)} \oplus a) \oplus S^{-1}(C'^{(2)} \oplus a) = 6,$$

and the analogous equation for nibble 4 and key guess b. Count candidates satisfying both tests. The desired key should tend to reveal the relatively frequent endpoint difference. Other keys can also receive nontrivial scores, so keep ranked candidates and verify rather than declaring the largest count a proof.

Algorithm 3.7 Differential ranking of the last subkey's groups 2 and 4

Require: Ciphertext pairs for chosen $P' = P \oplus$ `0B00`, inverse S-box
Ensure: A count and rank for each eight-bit candidate (a, b)

```
Initialise all candidate counts to zero
for each ciphertext pair (C, C') do
    if C^(1) ≠ C'^(1) or C^(3) ≠ C'^(3) then
        Skip this pair
    else
        for a, b ∈ {0, ..., 15} do
            d_2 ← S^-1(C^(2) ⊕ a) ⊕ S^-1(C'^(2) ⊕ a)
            d_4 ← S^-1(C^(4) ⊕ b) ⊕ S^-1(C'^(4) ⊕ b)
            if d_2 = 6 and d_4 = 6 then
                Increment the count for (a, b)
            end if
        end for
    end if
end for
return candidates ranked by decreasing count
```

3.8.5 Data requirements and the difference between a trail and a differential

With q chosen pairs, the expected number following a trail of probability p_D is qp_D. To see a few such events suggests a scale of $1/p_D$, about 38 pairs when $p_D = 27/1024$. This is only a rule of thumb for obtaining signal: successful key ranking also depends on wrong-key background, the filter, candidate correlations, and confidence. With 5000 pairs, the estimate gives about 132 trail occurrences, and a count of 122 corresponds to observed frequency 0.0244. A chosen pair may require two encryption queries; counting pairs and counting individual plaintexts gives different data totals.

Several intermediate trails can reach the same ΔU_4. Their exact fixed-key probabilities add because the complete internal difference histories are disjoint events. Therefore the endpoint differential can be stronger than any one trail. In contrast to linear hull correlations, these probabilities are nonnegative and do not cancel by opposite signs. A pair may reach the endpoint without following the selected trail, so an endpoint count must not be identified exactly with that trail's count.

	Linear attack	Differential attack
Input access	Known plaintext/ciphertext pairs	Chosen plaintext pairs with fixed XOR difference
S-box tool	LAT: signed deviation from half matches	DDT: count of an output difference
Signal	Parity bias ϵ	Unusually frequent transition p_D
Basic data scale	Order $1/\epsilon^2$	Heuristic order $1/p_D$ when background permits
Combination	Signed trail correlations form a hull	Disjoint trail probabilities sum to a differential

Both attacks exploit public S-box information and partial key guesses. Both are hindered by small local signals and many active S-boxes, but "active" has different meanings: a nonzero parity mask in a linear trail, and a nonzero input difference in a differential trail. The defence discussion that follows should be read with that distinction.

Refinements change the event being counted. A truncated differential predicts only part of the difference. An impossible differential rules out an endpoint transition and eliminates key candidates implying it; impossibility of just one internal trail is insufficient. Higher-order differences repeatedly apply $D_a f(x) = f(x) \oplus f(x \oplus a)$; for a Boolean polynomial of degree at most d, more than d such difference operations give zero. These ideas connect statistical attacks back to algebraic degree, but their full analysis goes beyond the basic examples here.

3.9 Making useful approximations hard to obtain

3.9.1 Active S-boxes and diffusion

Definition 3.22 (Active S-box in a linear trail)**.** An S-box is active when the trail assigns a nonzero parity mask to it. It then contributes a nontrivial local correlation. Inactive S-boxes with zero input and output masks contribute correlation 1, so they do not reduce the trail's magnitude.

If a active S-boxes each have bias magnitude at most $\epsilon_{\max}$, the corresponding product bound on a single trail is

$$|\epsilon_{\text{trail}}| \leq \frac{1}{2}(2\epsilon_{\max})^a.$$

Thus a designer seeks small local biases and diffusion that forces many active boxes. Increasing the number of rounds helps when it forces further nontrivial factors; merely repeating a layer without examining propagation is insufficient.

In Figure 3.12, orange marks an S-box participating in a nonzero parity-mask pattern. Such a box need not carry a nonzero data byte: activity refers to the tested parity, not the actual message value. The grey linear layer links those masks, so they cannot be chosen independently on its two sides. For a four-byte mixing layer with branch number 5, one nonzero input byte forces all four output bytes nonzero, because their total count must be at least 5. The corresponding AES mask constraint likewise forces several active substitutions. Each active box contributes a local correlation factor whose magnitude is below 1, reducing the chosen trail's product. The next definition states precisely which minimum is taken, including the transpose relation for parity masks.

Definition 3.23 (Branch number)**.** For an invertible linear layer L on groups of bits or bytes, let $\mathrm{wt}(a)$ count nonzero groups of a. Its branch number is

$$B(L) = \min_{a \neq 0} \big(\mathrm{wt}(a) + \mathrm{wt}(L(a))\big).$$

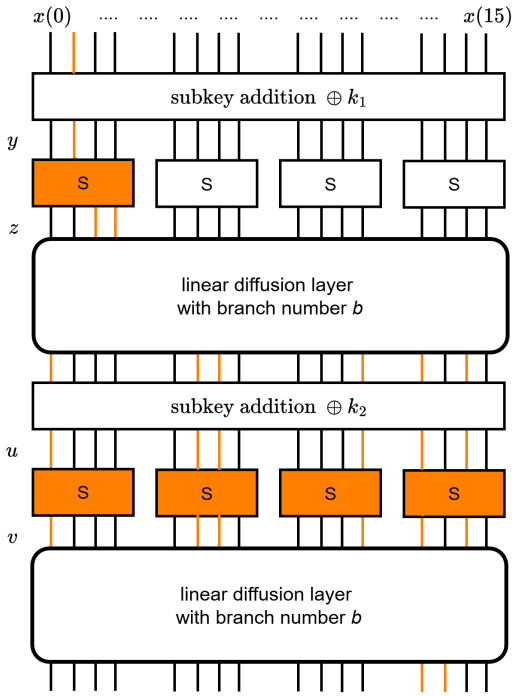


Figure 3.12: An orange first-round S-box is followed by activity in several next-round boxes; the linear layer forces the spread.

A large value prevents a nonzero input and its output from both involving very few groups. Linear-mask propagation uses the corresponding transpose relation: if the output mask is b, the input mask is $L^T b$. Its relevant minimum is consequently $\min_{b \neq 0}(\mathrm{wt}(b) + \mathrm{wt}(L^T b))$.

In AES, the byte substitution has maximum linear bias magnitude 2^{-4}. MixColumns mixes four bytes with branch number 5 (also for the corresponding mask bound); ShiftRows redistributes bytes among columns. Together these operations force at least 25 active S-boxes over a nontrivial four-round trail in the standard wide-trail argument. This is the "square amplification" idea: combining local mixing with inter-column redistribution produces a four-round bound of 5^2.

Example 3.24 (From 25 active boxes to a trail bound)**.** The AES bound gives a correlation magnitude at most $(2 \cdot 2^{-4})^{25} = 2^{-75}$ for a single such trail, hence bias magnitude at most 2^{-76}. Its squared bias is at most 2^{-152}. This is a bound for individual trails under the stated structure; the next distinction matters before treating it as the bias of the entire cipher.

3.9.2 A trail versus a linear hull

A *linear trail* specifies exactly which parity bits are selected at each intermediate stage, as the orange path did earlier. A *linear hull* keeps only the input and output parity choices fixed and gathers all intermediate trails connecting them. The attacker observes that overall input/output relation; the intermediate routes are a device for analysing it. Their signed correlations add, with key XORs affecting the signs. Consequently the complete relation can be stronger or weaker than its most prominent individual trail.

Read the vertical axis of Figure 3.13 in powers of two: a height of -2 means bias magnitude $2^{-2} = 1/4$, whereas -10 means $2^{-10} = 1/1024$. Lower on the graph means a weaker imbalance. The black curve tracks the best individual sequence of intermediate masks. The orange results concern an approximation that gathers several such sequences with the same external masks.

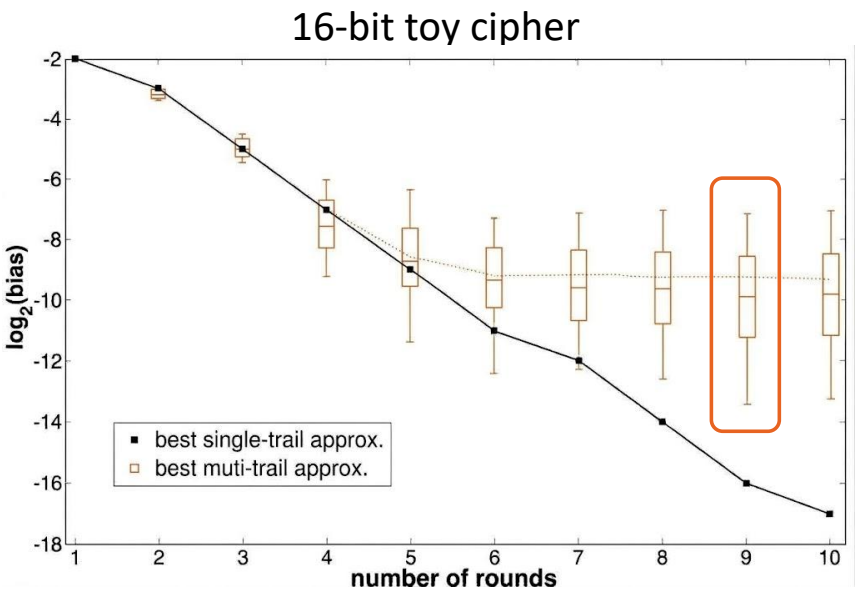


Figure 3.13: The black single-trail curve decreases with rounds, while the orange multi-trail results remain larger and show a spread.

For example, two compatible trail contributions of correlation 2^{-8} and 2^{-9} sum to $3 \cdot 2^{-9}$ when their signs agree, or to 2^{-9} when their signs oppose. Key-dependent signs therefore change the aggregate signal. This explains why following only the shrinking black curve can miss the stronger total approximation; the plot illustrates a 16-bit toy cipher, not an attack measurement for AES.

Remark 3.25 (Why a statistical background remains). A random permutation on a fixed finite domain also has small accidental parity imbalances. For fixed nonzero external masks on an N-bit domain, its squared bias has scale 2^{-N}. It is therefore unreasonable to demand that the actual best full-cipher bias vanish merely by adding rounds at fixed block size. The plot's lesson is the difference between a shrinking trail and a whole approximation; a literal limit for every key and round sequence would require further assumptions. A useful design target is to push exploitable structure below the random-permutation scale, while separately considering hull effects and key recovery.

High algebraic degree, small local bias, and many active S-boxes are complementary criteria. Each obstructs a class of shortcuts; none by itself proves the PRP definition against every efficient algorithm.

3.10 AES as a concrete design

The Advanced Encryption Standard (AES) is specified in Federal Information Processing Standard (FIPS) 197. It operates on 128-bit blocks; its 128-, 192-, and 256-bit keys select 10, 12, and 14 rounds respectively. The key size changes the key search space, not the block domain.

Its state is an array of 16 bytes. SubBytes applies nonlinear byte substitutions, ShiftRows rearranges byte positions, MixColumns mixes columns linearly, and AddRoundKey XORs a subkey. A key schedule supplies those subkeys. MixColumns is omitted in the last round. Byte arithmetic can be represented in the binary field $\mathbb{F}_{2^8}$, a field of 256 elements; this is not ordinary integer arithmetic modulo 256. Appendix A constructs this arithmetic and works out byte multiplication, inversion and the MixColumns matrix. Its coding-theory section derives

the optimal branch-number bound rather than assuming it.

Algorithm 3.8 AES encryption: high-level round organisation

Require: A 128-bit plaintext x, an AES key k of 128, 192, or 256 bits
Ensure: A 128-bit ciphertext; N_r is 10, 12, or 14 according to the key size

Derive round keys $K_0, \ldots, K_{N_r}$ using the AES key schedule
$s \leftarrow x \oplus K_0$
for $i = 1, \ldots, N_r - 1$ **do**
    $s \leftarrow \text{SubBytes}(s)$
    $s \leftarrow \text{ShiftRows}(s)$
    $s \leftarrow \text{MixColumns}(s)$
    $s \leftarrow s \oplus K_i$
end for
$s \leftarrow \text{ShiftRows}(\text{SubBytes}(s))$
return $s \oplus K_{N_r}$

For one AES block, imagine the 16 bytes arranged in four rows and four columns. SubBytes changes the value of each byte through the S-box. ShiftRows moves the bytes to different columns. MixColumns then combines the four byte values in each column, so later substitutions depend on several earlier bytes. AddRoundKey XORs the round key into these values. Repeating this sequence prevents the four columns or sixteen bytes from remaining independent small problems. The precise transformations and number of rounds matter; this reading explains the role of each named step in the algorithm. These arguments explain why familiar attack templates are obstructed. Treating AES as a secure PRP remains a computational assumption, rather than a consequence of one degree or trail calculation. A mode still needs its own security argument, as in Chapter 2.

3.11 Quantum search and the composition challenge

3.11.1 What the square-root speedup means

For a uniformly chosen κ-bit key and a recognisable correct candidate, classical exhaustive search has work on the order of 2^κ. Grover's quantum search reduces the generic number of coherent search iterations to order $2^{\kappa/2}$. Thus 128- and 256-bit keys correspond to idealised query scales 2^{64} and 2^{128}.

This compares search exponents. Each quantum iteration must reversibly implement and check the cipher, and actual resources include gates, memory, and error correction. Doubling the key length restores the exponent against this generic search model; it is not a universal rule that accounts for every attack or gives a practical runtime estimate.

3.11.2 Why double encryption does not double classical strength

Consider $E_{k_1,k_2}(x) = E_{k_2}(E_{k_1}(x))$ for a block cipher E with κ-bit keys. There are $2^{2\kappa}$ key pairs, but an attacker can compare intermediate values instead of trying every pair separately.

Definition 3.26 (Meet-in-the-middle attack)**.** Given a known plaintext/ciphertext pair (x, c) for double encryption, list the values $E_a(x)$ for all first-key candidates a. For each second-key candidate b, compute $E_b^{-1}(c)$ and look for a matching list entry. A match proposes a key pair because both sides then agree at the hidden intermediate block.

Algorithm 3.9 Meet-in-the-middle key search on double encryption

Require: Known pairs (x_j, c_j) encrypted with the same pair of κ-bit keys
Ensure: Key pairs consistent with the supplied observations

```
Create a table T that retains all keys sharing a stored intermediate value
for each first-key candidate a ∈ {0,1}^κ do
    Store a in T[E_a(x_1)]
end for
for each second-key candidate b ∈ {0,1}^κ do
    w ← E_b^{-1}(c_1)
    for each a stored in T[w] do
        Test whether E_b(E_a(x_j)) = c_j for all remaining pairs
        Retain (a, b) if all tests pass
    end for
end for
return the retained pairs
```

Example 3.27 (Two 128-bit keys)**.** Building and probing the tables takes about $2 \cdot 2^{128} = 2^{129}$ primitive evaluations, with order 2^{128} stored entries before memory/time trade-offs. This is far below 2^{256} pairwise trials. One pair is insufficient to identify the true keys: for a B-bit block, the idealised expected number of wrong matching pairs is about $2^{2\kappa-B}$. With $B = \kappa = 128$, another independent pair reduces that count by a factor about 2^{-128}; further pairs provide stronger verification. Candidate filtering adds work and must be included in concrete estimates.

Remark 3.28 (Key length and block length are separate). Double encryption keeps the same block domain while enlarging the key description. It therefore does not extend the block length, and its effective key-search resistance is limited by the composition attack above. AES-256 is a separately specified 256-bit-key design with a 128-bit block, not two AES-128 encryptions. A longer key, a wider block, and more layers are distinct changes, each requiring its own analysis. The classical meet-in-the-middle argument also does not by itself specify the optimal quantum attack on the composition.

3.12 Review questions

Exercise 3.29 (Connect each design claim to its assumptions)**.**

1. Invert a Feistel round when its round function is not injective. Why does invertibility alone not establish pseudorandomness?
2. In the three-round bound, why does the collision denominator involve the half-block length? What changes when inverse queries are allowed?
3. Explain the PRP-to-PRF switching term by comparing sampling with and without replacement.
4. Evaluate a GGM path for input 101. What makes repeated inputs consistent, and why is evaluating one leaf efficient?
5. Build a doubling generator from one-bit stretch. Which output bits, including the final state, must be retained?
6. Reproduce the four-minute cost estimate. Which input length and block length does the example use?

7. Recover an affine cipher from chosen inputs. Derive the product counting invertible binary matrices.

8. Explain the four independent nibble searches and distinguish an unknown 16-bit subkey from two independent 16-bit subkeys.

9. Expand the two-bit polynomial example, linearise its key product, and state which consistency constraint must still be checked.

10. Why is d^r an upper bound on degree rather than a guarantee? How many monomial variables can linearisation introduce?

11. Prove the piling-up lemma. Why does its independence condition matter, and how does the bias differ from correlation?

12. Derive the scale $1/\epsilon^2$ for data requirements. Why is absolute empirical bias used to rank key guesses?

13. Explain active S-boxes, branch number, and the roles of SubBytes, MixColumns and ShiftRows.

14. Distinguish a linear trail from a linear hull. Why does a tiny trail bound not automatically bound the entire approximation?

15. Give the AES block and key sizes, describe Grover's idealised exponent, and explain why double encryption does not supply a 2^{256} classical search barrier.

16. Construct a LAT entry from the four-bit S-box. How does a negative count offset become a useful parity approximation?

17. Trace the worked SPN's first three rounds and explain why only final-key nibbles 2 and 4 must be guessed for the displayed approximation.

18. Compute the DDT transition $B \to 2$, keeping ordered base inputs distinct from unordered pairs. Why do nonzero input differences have even counts?

19. Derive the boundary differences `0B00` $\to$ `0606` and the estimate 27/1024. What does the ciphertext filter establish, and what does it not establish?

20. Contrast adding differential-trail probabilities with adding linear-trail correlations. State the assumptions behind each data-scale estimate.

Chapter 4

Message authentication

An attacker who cannot read a ciphertext may still alter it, replay it, or replace it with another valid-looking object. Message authentication asks the receiver to accept data only when it carries suitable evidence from a party holding the authentication key. This chapter makes that goal precise, builds MACs from block ciphers, and explains why padding, protocol state, implementation behaviour, and composition all enter the security argument.

4.1 What must the receiver be able to check?

Definition 4.1 (Message authentication code)**.** A message authentication code (MAC) consists of key generation, tag generation, and verification:

$$k \leftarrow \mathrm{Gen}(n), \qquad t \leftarrow \mathrm{Mac}_k(m), \qquad \mathrm{Ver}_k(m,t) \in \{\top, \bot\}.$$

Here n is the security parameter, m a message, t its tag, $\top$ means accept, and $\bot$ means reject. Correctness requires acceptance of legitimately generated tags. The sender and verifier share a secret key.

The tag binds the message to knowledge of the key. Bob can recompute or check it; an outsider should be unable to create acceptable evidence for an unauthorised message. This leaves the message readable unless encryption is also used. Since Bob holds the same key, a MAC cannot by itself prove to a third party that Alice, rather than Bob, generated a particular tag.

Start at Alice in Figure 4.1. She sends a message and its authentication tag through the exposed channel. Eve may copy the pair or replace either component. Bob's box checks the *received* message against the *received* tag before using the instruction. If Eve changes "pay 10" to "pay 90", the old tag should no longer verify for the changed message, and Eve should be unable to supply a replacement. This is a test of authenticity, not an attempt to keep the amount secret.

Example 4.2 (An authenticated software update)**.** For an update, the important question is whether the installed bytes are authorised, not merely whether they can be downloaded without being read. A device sharing a secret with its update server can check a MAC. For a public release intended to be verified by many mutually untrusted users, a digital signature is more appropriate: giving every user the MAC verification key would also give them the ability to generate tags. The required trust relationship determines which mechanism fits.

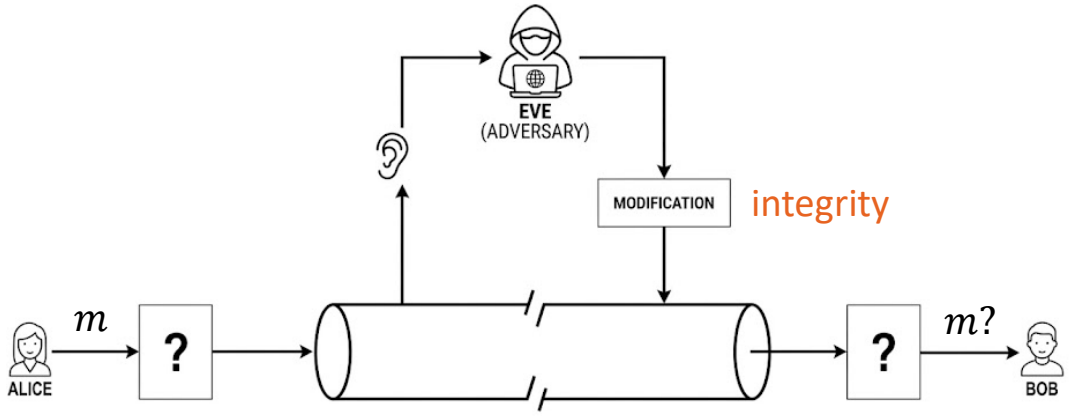


Figure 4.1: Alice's message crosses a channel on which Eve can modify it; Bob needs a way to reject unauthorised changes.

4.2 Why checksums and short tags fail

4.2.1 Detecting accidents does not stop an adversary

Suppose a tag is just the XOR of equal-length message blocks:

$$t = m(0) \oplus m(1) \oplus \cdots \oplus m(l-1).$$

An accidental isolated bit flip changes this checksum. An attacker, however, can choose coordinated changes: replace two blocks by $m(0) \oplus \delta$ and $m(1) \oplus \delta$, for any chosen nonzero block δ. The two copies of δ cancel in the checksum, so the message changes while the tag stays the same.

Example 4.3 (A checksum-preserving change)**.** For two four-bit blocks $1010, 0110$, the checksum is 1100. Change both blocks by XOR with 0011, obtaining $1001, 0101$. The checksum is still 1100. More generally, an unkeyed public checksum can simply be recomputed after changing the message. Resistance to malicious modification therefore requires some authentication information the attacker cannot generate, such as a secret-key MAC or a public-key signature.

4.2.2 The tag length is a separate security parameter

Even an ideal secret-key mechanism has a guessing limit. If a fresh message's valid tag is an independent uniform τ-bit string, one blind guess succeeds with probability $2^{-\tau}$. For a 5-bit tag, this is $1/32$, about 3.1%: a long secret key does not make that tag difficult to guess.

For v verification attempts, the union bound gives at most $v/2^{\tau}$ success in the ideal guessing model, capped at 1. If the attempts enumerate distinct tags for one fixed fresh message with a unique uniform valid tag, the probability is exactly $\min(v, 2^{\tau})/2^{\tau}$. Real bounds may also include weaknesses in the primitive and the construction. Key length, tag length, and allowed attempts must therefore be considered separately.

4.3 Unforgeability as an experiment

4.3.1 What counts as a forgery?

Let Q record messages for which the attacker has requested legitimate tags, and let S record the returned message/tag pairs. Both sets start empty. Queries can be adaptive: after seeing a tag, the attacker may choose its next message based on that answer.

Definition 4.4 (Existential unforgeability)**.** A successful ordinary forgery is an accepted pair (m^*, t^*) with $m^* \notin Q$. A MAC is existentially unforgeable under adaptive chosen-message attack if every probabilistic polynomial-time (PPT) adversary wins with negligible probability:

$$P(\mathrm{Exp}^{\mathrm{unf}}_{\mathrm{MAC},\mathcal{A}}(n) = 1) \leq \mathrm{negl}(n).$$

The word existential means that the adversary may choose which fresh message to forge; it need not hit a target fixed in advance. Chosen-message access means that observing tags on attacker-selected messages is explicitly allowed.

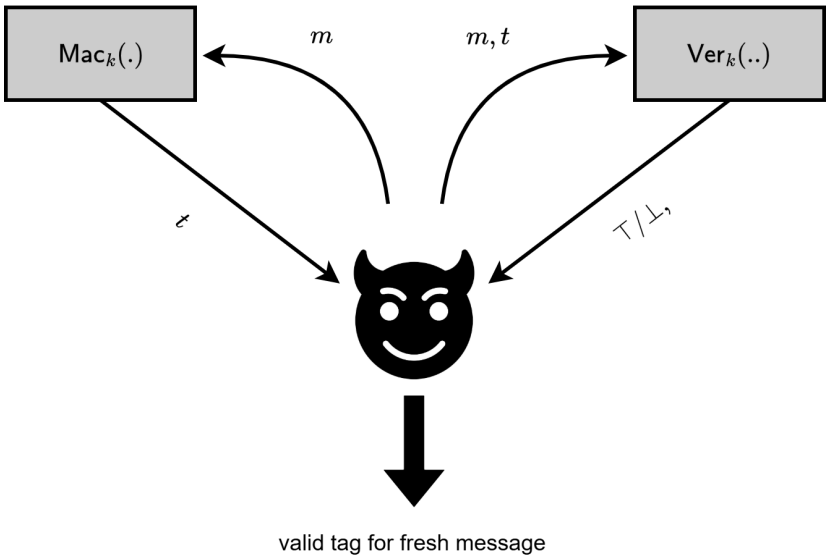


Figure 4.2: The attacker requests genuine tags on the left and submits candidate pairs to verification on the right.

Read Figure 4.2 as two different permissions. The left box answers "what tag does the legitimate sender attach to this message?" The right box answers "does this submitted pair verify?" Both use the same secret key, which is never handed to the attacker. Getting a tag for "pay 10" on the left and submitting that identical pair on the right is not an ordinary forgery. Getting that tag and then making "pay 90" verify is. We use a version that allows verification feedback and records a win on any fresh accepted attempt; a common simpler version permits tagging queries followed by just one final verification. The number of verification attempts matters in a concrete bound.

Definition 4.5 (Strong unforgeability)**.** A strong forgery is an accepted pair $(m^*, t^*) \notin S$, even when $m^* \in Q$. Strong unforgeability requires negligible probability of producing any new accepted pair, rather than only a tag on a new message. The experiment is written $\mathrm{Exp}^{\mathrm{suf}}_{\mathrm{MAC},\mathcal{A}}(n)$.

Example 4.6 (A distinction that verification can expose)**.** Start from a secure deterministic MAC. Return tags of the form $(t, 0)$, but let the verifier ignore the last bit and check only t. Ordinary unforgeability is preserved: forging for a new message still requires a valid underlying tag. Strong unforgeability fails immediately: after receiving $(t, 0)$ for a queried message, output $(t, 1)$ for that same message. The accepted pair is new even though the message is not.

Algorithm 4.1 MAC experiment with tagging and verification access

Require: Security parameter n, MAC, adversary $\mathcal{A}$, mode unf or suf
Ensure: One bit indicating whether a fresh accepted pair was submitted
$k \leftarrow \text{Gen}(n)$; $Q \leftarrow \varnothing$; $S \leftarrow \varnothing$; $w \leftarrow 0$
function TAG-ORACLE(m)
    $t \leftarrow \text{Mac}_k(m)$; add m to Q and (m, t) to S
    Return t to $\mathcal{A}$
end function
function VERIFY-ORACLE(m, t)
    $a \leftarrow \text{Ver}_k(m, t)$
    In mode unf, set $w \leftarrow 1$ if $a = \top$ and $m \notin Q$
    In mode suf, set $w \leftarrow 1$ if $a = \top$ and $(m, t) \notin S$
    Return a to $\mathcal{A}$
end function
Run $\mathcal{A}$ with these tagging and verification handlers
Process the final output pair using the same verification handler
return w

Remark 4.7 (When the two notions coincide)*.* For deterministic tag generation with *canonical verification*, the verifier accepts exactly the recomputed tag $\text{Mac}_k(m)$, with one valid encoding. After a legitimate query on m, the only acceptable pair for that message is already in S. A new accepted pair must therefore involve a fresh message. Under this condition, ordinary unforgeability implies strong unforgeability. Canonical verification alone is not a security proof: a trivially predictable tag can have a unique valid value and still be forgeable.

4.3.2 A simple way to connect MACs to pseudorandomness

For fixed-length messages, an ideal random function assigning an τ-bit tag to each message supplies a useful reference point. A tagging query reveals one table entry. The tag on an unqueried message remains uniform, so the adversary must guess it. Replacing that random function by a secure PRF gives a deterministic MAC, with security bounded by the PRF distinguishing gap plus the tag-guessing contribution. With one final attempt, the latter is $2^{-\tau}$.

The reduction's logic is concrete: if a forger succeeds substantially more often with the real keyed function than with the ideal table, run the forger using the unknown function oracle and output whether its forgery verifies. That success event distinguishes the two worlds. For long messages, however, a fixed-input-length primitive still needs a secure way to process and encode all the message blocks.

4.4 CBC-MAC: what the chain authenticates

4.4.1 Raw construction and its fixed-length scope

For this construction, n denotes the block length, κ the key length, and P_k a permutation on n-bit blocks. The two lengths need not coincide. For a nonempty, block-aligned message of l

blocks, set

$$h_{-1} = 0^n, \qquad h_i = P_k(h_{i-1} \oplus m(i)), \quad 0 \le i < l, \qquad t = h_{l-1}.$$

Only the final state is transmitted as the tag. Raw CBC-MAC is designed for a message domain with one fixed block count under a key; its security does not automatically extend to messages of arbitrary lengths.

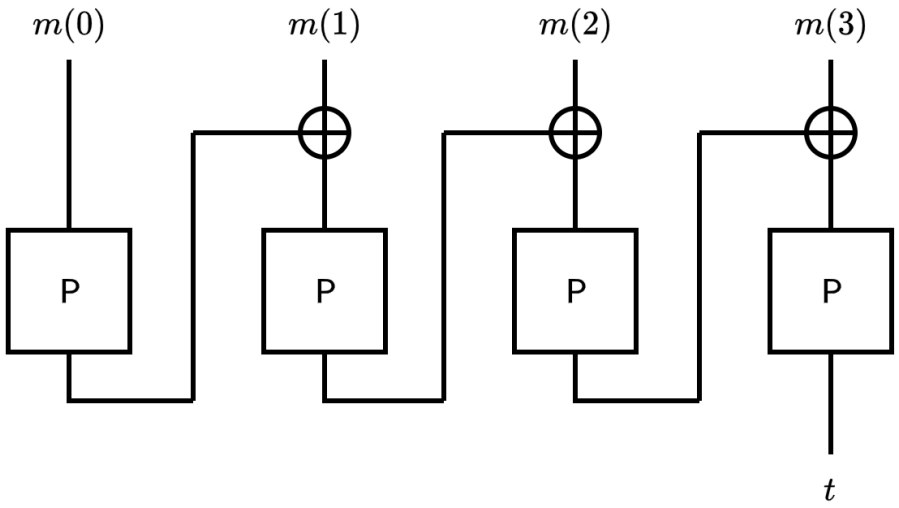


Figure 4.3: CBC-MAC starts at zero, chains through all message blocks, and outputs only the last state as tag t.

Follow Figure 4.3 from left to right. The first P box receives $m(0)$, since XOR with the initial zero does nothing. Its output travels to the next XOR and is combined with $m(1)$. The same key is used in every box. In contrast to CBC encryption, the intermediate outputs are not published as ciphertext blocks, and there is no transmitted random IV. The receiver runs the same chain on the received message and compares the final result with t.

Algorithm 4.2 Raw CBC-MAC on its specified full-block message domain

Require: Key k, nonempty blocks $m(0), \ldots, m(l-1)$ of n bits
Ensure: One n-bit tag; the permitted block count is a domain restriction

```
h ← 0^n
for i = 0, ..., l − 1 do
    h ← P_k(h ⊕ m(i))
end for
return h
```

Remark 4.8 (Why determinism is acceptable for a MAC). A MAC is not required to hide which message was sent, or to hide repeated messages. Its security question is whether a fresh accepted pair can be manufactured. Therefore a deterministic tag does not create the same contradiction as deterministic stateless encryption in the CPA game. Introducing an attacker-controlled IV into the raw chain would instead create an immediate problem: changing both the IV and the first message block by the same XOR difference leaves the chain unchanged.

4.4.2 A tag becomes a continuation state

Let messages m_a, m_b have raw tags t_a, t_b. Form

$$m^* = m_a \| (m_b(0) \oplus t_a) \| m_b(1) \| \cdots .$$

The first appended block is *modified*; it is not generally the ordinary concatenation $m_a \| m_b$.

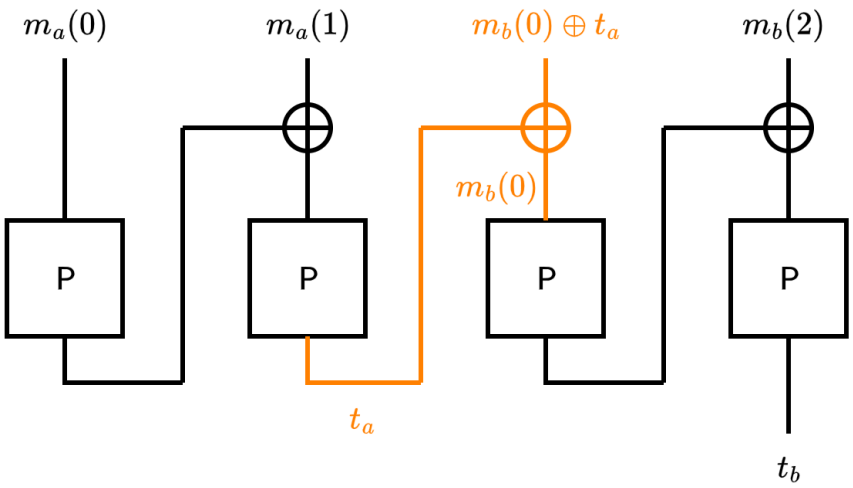


Figure 4.4: The orange modification cancels the exposed state t_a, so the appended computation follows the original chain for m_b.

At the orange connection in Figure 4.4, the computation has just finished m_a, so its state is t_a. The next XOR combines that state with the chosen block $m_b(0) \oplus t_a$:

$$t_a \oplus (m_b(0) \oplus t_a) = m_b(0).$$

The next P box therefore receives exactly the first input it received when authenticating m_b from zero. Every remaining block now repeats that old computation, giving the known final tag t_b. This is a valid new-message forgery if variable lengths are admitted and the constructed message was not queried. The tags reveal enough of the chaining computation to splice two histories together.

Example 4.9 (Two queried blocks give a longer forgery)**.** For an arithmetic illustration, suppose a four-bit permutation has $P_k(1010) = 0011$ and $P_k(0110) = 1110$. Query the one-block messages 1010 and 0110, obtaining tags 0011 and 1110. The forged two-block message is

$$(1010,\ \ 0110 \oplus 0011) = (1010, 0101).$$

Its first state is 0011; its second permutation input is $0011 \oplus 0101 = 0110$, so its tag is 1110. No query on that two-block message was necessary. Fixing all legitimate messages to one block would exclude this forged length, which is why the domain restriction matters.

4.5 Marking message boundaries securely

4.5.1 Separating the final operation

One repair uses independent keys k_1, k_2: apply P_{k_1} to every nonfinal block and P_{k_2} to the final block. In this variant, the transmitted tag is the result of a designated final operation, rather than an ordinary internal state under k_1.

In Figure 4.5, stop at the last orange P_{k_2} box. Its output is not the P_{k_1} state that would have existed had more message blocks followed. Consequently, substituting that tag as a continuation state does not reproduce the earlier chain used in the extension attack. The separation marks

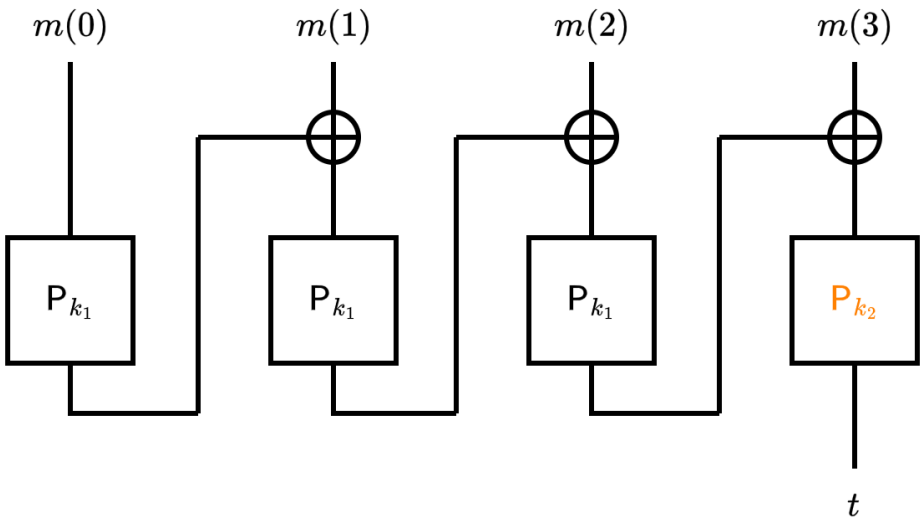


Figure 4.5: The final block uses the independent key k_2; earlier blocks use k_1.

"this is the end" cryptographically. With two independent κ-bit keys the stored key material is 2κ bits. Messages must still have an unambiguous block encoding; this repair addresses finalisation, not arbitrary ambiguous parsing.

4.5.2 Putting the length first

Another repair encodes the true message length in a fixed-width first block. Let $\mathrm{len}_n(m)$ encode the original bit length as an n-bit integer, restricting messages to lengths below 2^n. Using the marker padding described below, whose output length depends only on the original bit length, authenticate

$$\mathrm{len}_n(m) \| \mathrm{pad}(m)$$

with the ordinary zero-initialised chain.

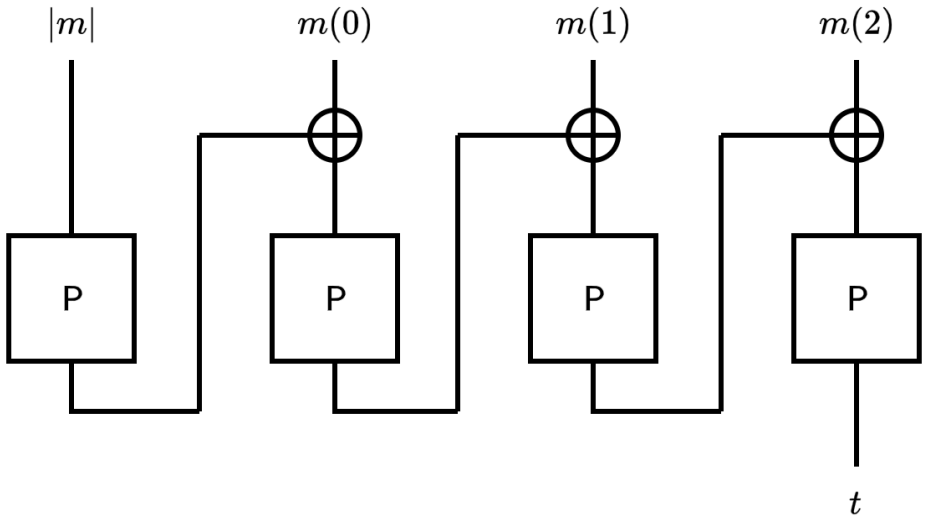


Figure 4.6: The first block encodes the true message length, so changing the message length changes the chain from its first step.

The first input in Figure 4.6 is $|m|$, not a payload block. In the earlier splice attack, the old tag came from a computation beginning with the shorter message's length. A longer proposed forgery must begin with a different length block, so it cannot simply reuse that old computation as its own prefix. The encoding is *prefix-free*: no valid encoded message is a proper initial segment of another. Indeed, equal length fields imply equal total encoded lengths; unequal fields already differ in the first block. This gives the boundary information missing from raw variable-length CBC-MAC. The trade-off is that the sender must know the message length before starting the chain.

Algorithm 4.3 Two boundary-aware CBC-MAC variants

Require: Nonempty full-block encoding $M(0), \ldots, M(l-1)$; appropriate keys
Ensure: A tag with final-key separation, or a length-prefixed CBC tag

function FINAL-KEY-MAC(k_1, k_2, M)
    $h \leftarrow 0^n$
    for $i = 0, \ldots, l-2$ **do**
        $h \leftarrow P_{k_1}(h \oplus M(i))$
    end for
    return $P_{k_2}(h \oplus M(l-1))$
end function
function LENGTH-PREFIX-MAC(k, m)
    $M \leftarrow \mathrm{len}_n(m) \| \mathrm{pad}(m)$
    return the raw CBC chain on M with key k
end function

4.5.3 Padding must preserve the identity of the message

Definition 4.10 (Injective encoding)**.** An encoding is injective if distinct original messages always produce distinct encoded strings. For a padding map this means

$$m \neq m' \quad \Longrightarrow \quad \mathrm{pad}(m) \neq \mathrm{pad}(m').$$

Without this property, a tag on one message may automatically authenticate another because the cryptographic calculation sees identical bytes.

Zero-padding *alone* is ambiguous. With four-bit blocks, 101 and 1010 both become 1010 if one appends only enough zeroes to reach the next boundary. A verifier cannot tell whether the final zero was data or padding.

Appending a marker bit 1 followed by enough zeroes to reach a boundary gives the bit-padding rule $m\|1\|0\cdots0$. Always append the marker, including when the original message is aligned. The messages above then encode as 1011 and 1010$\|$1000, respectively. To undo the rule, remove trailing zeroes and the final 1 marker; the remaining bits are the message.

Remark 4.11 (Which part of the encoding removes ambiguity?). Injective padding and prefix-free encoding solve related but different problems. The marker makes the original message recoverable from its padded bytes; length-prefixing additionally ensures that no full valid encoding continues another. If a fixed-width prefix already records the *true original bit length*, the complete encoding can distinguish 101 from 1010 even with zero-filled final blocks: their length fields differ. It would be incorrect to claim that their complete length-prefixed encodings are identical. The security check concerns the whole representation, including its length unit and field boundaries.

4.6 Reading a concrete unforgeability bound

The authentication bound must name the secure variant and its input domain. Let σ count the total blocks processed across the attack's relevant tagging and verification computations, let v count verification attempts, and let τ_P be the cost of a permutation call. Suppose the primitive's PRP distinguishing gap is bounded by ε_P for computations taking at most time t. For suitable CBC-based MACs and their encodings, a conservative schematic form is

$$\varepsilon_{\text{unf}} \lesssim \varepsilon_P + C\frac{\sigma^2}{2^n} + \frac{v}{2^\tau},$$

at attacker time roughly $t - \sigma\tau_P$, with additional simulation overhead. Here ε_P bounds the primitive's PRP distinguishing gap, τ is the transmitted tag length, and C suppresses construction-dependent constants. This is a guide to interpreting resource terms, not one exact theorem covering every CBC-MAC variant.

The three terms ask three separate questions: can the attacker recognise a weakness in the primitive, can enough chaining values interact through collisions, and can enough tag guesses succeed? If q messages each have at most l blocks, then σ scales with ql, including any encoded length or padding blocks and verification work. A shorthand $q^2/2^n$ collision term must be read with its definition of q and fixed-length assumptions; it must not silently erase message length. Even with zero tagging queries, a blind final forgery still has a guessing probability, so a zero collision term does not mean zero forgery probability.

Example 4.12 (A bound is not an exact attack probability)**.** For $n = 128$ and $\sigma = 2^{32}$, the expression $\sigma^2/2^n$ is 2^{-64}. With a 64-bit tag and $v = 2^{20}$ attempts, the ideal guessing contribution is 2^{-44}. Both parameters enter the estimate; strengthening one does not remove the other. Tightness asks whether an actual attack on the specified variant approaches the bound at comparable resources. A large upper bound alone does not construct that attack.

4.7 Protocol state and implementation behaviour

4.7.1 An authentic message can still be a replay

A recorded legitimate pair verifies again because its mathematical validity has not changed. To protect a command protocol, authenticate an unambiguously encoded record such as

$$\text{encode}(\text{device identifier}, \text{counter}, \text{command}).$$

The receiver checks the tag and enforces an appropriate counter policy. For example, after accepting command number 42, it rejects a repeated number 42. Without the receiver's remembered state, authenticating the counter bytes alone does not stop replay. The policy must also address restarts, counter wraparound, and which sender/key context the state belongs to.

4.7.2 Early rejection can leak a correct prefix

Suppose a verifier compares a 16-byte tag from left to right and exits on the first incorrect byte. For a fixed fresh message, an attacker tries all 256 values of the first tag byte. The correct first byte causes at least one additional comparison, potentially yielding a detectable timing difference. After fixing that byte, the attacker searches the next, and so on. The search decomposes into byte-sized tasks instead of one search over 2^{128} tags.

In a clean model this takes at most about $16 \cdot 256$ candidate tests; real timing measurements may require many repetitions to separate the signal from noise. The ideal verification oracle

returns just accept/reject, whereas this implementation additionally leaks how far the comparison progressed. A constant-time comparison avoids secret-dependent early exits and compares the complete tag; consistent failure handling must also prevent other useful timing distinctions. The implementation must preserve the information boundary assumed by the proof.

4.8 Collision-resistant hashing

4.8.1 Public fingerprints and their security task

Definition 4.13 (Hash function and collision resistance)**.** A cryptographic hash maps arbitrary finite bit strings to a fixed-length digest:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^o.$$

The star allows any finite input length, and o is the output length. A collision is a pair of *distinct* messages $m_0 \neq m_1$ with $H(m_0) = H(m_1)$. Collision resistance means it is computationally infeasible to find such a pair. In an asymptotic formulation, publicly sampled hash parameters and output lengths depend on the security parameter; the collision-finding probability of any PPT adversary must be negligible.

The adversary knows the hash algorithm and may evaluate it freely. The goal is not to prevent collisions from existing: a finite output set cannot uniquely represent all messages. The goal is to make finding a colliding pair difficult. Collision resistance also differs from preimage resistance, which asks for a message producing a *given* digest.

For q distinct inputs to an ideal o-bit random function, the probability of at least one collision is

$$1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{2^o}\right) \approx \frac{q(q-1)}{2 \cdot 2^o}$$

when q is well below $2^{o/2}$. There are $q(q-1)/2$ pairs of outputs to compare. This explains the birthday work scale $2^{o/2}$, rather than 2°, for finding *any* collision. A 256-bit digest therefore has an ideal generic collision scale of about 2^{128} evaluations.

Remark 4.14 (A public hash is not a secret-key MAC). Sending $(m, H(m))$ over an attacker-controlled channel does not authenticate the message. Eve can replace both components by $(m', H(m'))$ without finding a collision. A digest is useful as a fingerprint only when its expected value is itself trusted or authenticated. By contrast, a MAC's key prevents Eve from computing a replacement tag freely.

4.8.2 Building compression from a block cipher

In Figure 4.7, cst is a fixed public initial state, the final h is the digest state, and each top input $m(i)$ enters the block cipher's *key* port. Let E have o-bit blocks and let the message blocks have its supported key length. A compression step has the Davies–Meyer form

$$h_{i+1} = E_{m(i)}(h_i) \oplus h_i.$$

The lower feedback wire carries a copy of h_i to the XOR, while the upper path computes the block-cipher output. This feed-forward XOR changes the compression function from merely applying a reversible cipher. Message padding and length encoding must also be specified to define a complete iterated hash.

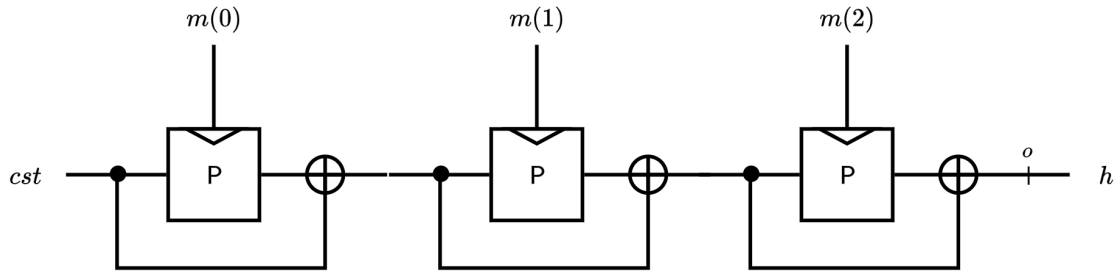


Figure 4.7: Message blocks select block-cipher keys; each output is XORed with the incoming hash state before continuing.

The key word is misleading if read as "secret" here: the attacker chooses message blocks and therefore chooses these block-cipher keys. The usual PRP assumption describes an unknown randomly selected secret key, so it is not enough by itself to justify this chosen-key public hashing use. Security arguments need an appropriate stronger model or analysis of the actual hash construction. The diagram is a construction pattern, not a proof that any PRP yields collision resistance in this role.

4.9 Sponge hashing and SHA-3

4.9.1 Where the message enters and the output leaves

Let a public permutation P act on a b-bit state, split into an r-bit *rate* part and a c-bit *capacity* part, where $b = r + c$. Start from the all-zero state. After applying the specified domain-separation and padding rules, divide the input into r-bit blocks. XOR one block into the rate part, then apply P to the entire state; repeat until all input blocks have been absorbed. To produce output, read rate bits, apply P again if more output is needed, and continue.

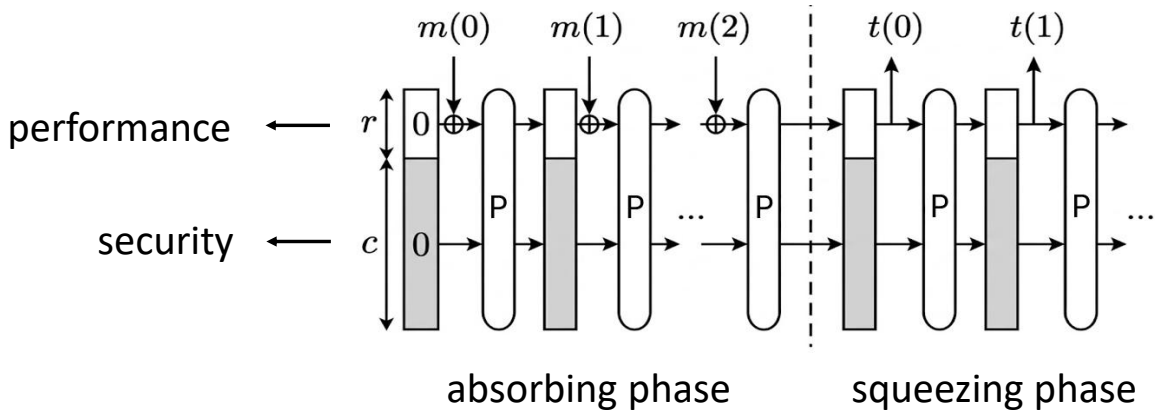


Figure 4.8: Absorption XORs message blocks into the rate; squeezing reads rate bits. Each P mixes the entire rate-plus-capacity state.

Read Figure 4.8 from the two initial zero fields. The upper r bits meet $m(0)$ at an XOR, while the lower c bits receive no direct message XOR. Both fields then pass through the same P box and can influence one another. Repeat with $m(1), m(2), \ldots$ until the dashed divider. On its right, the arrows $t(0), t(1)$ take output from the rate part; no new message blocks enter. The $t(i)$ labels here mean hash-output chunks, not secret-key MAC tags. Capacity bits are not directly output, but they participate in every permutation.

Algorithm 4.4 Sponge absorption and squeezing at a structural level

Require: Public b-bit permutation P, rate r, capacity $c = b - r$
Require: Message m, requested output length o, specified domain and padding rules
Ensure: An o-bit digest or output string according to these parameters

Apply the specified domain separation and padding; obtain r-bit blocks M_i
$s \leftarrow 0^b$
for each block M_i in order **do**
    $s \leftarrow P(s \oplus (M_i \| 0^c))$
end for
$z \leftarrow$ the first r bits of s
while $|z| < o$ **do**
    $s \leftarrow P(s)$
    Append the first r bits of s to z
end while
return the first o bits of z

Example 4.15 (Tracing an absorption step)**.** For a toy state of 8 bits with rate 5 and capacity 3, start from 00000 000. Absorbing 10110 gives 10110 000 before the permutation. Suppose this illustrative permutation maps that value to 01101 110. If the next message block is 00101, the next permutation input is

$$(01101 \oplus 00101)\,110 = 01000\,110.$$

The capacity is retained during the XOR and then transformed along with the rate by the next P. After the final absorption, squeezing would first read the rate part of the *permuted* state, not the last plaintext block.

4.9.2 Rate, capacity, and output length

With a fixed state width b, a larger r absorbs more message bits per permutation call but leaves a smaller c. In the ideal-permutation analysis, the capacity controls internal collision effects with a birthday scale around $2^{c/2}$, while an o-bit digest has its own output-collision scale around $2^{o/2}$. Both limits matter when interpreting a collision-security target. This explains the diagram's "performance" and "security" arrows; they do not mean that rate and capacity are independently adjustable without changing the construction.

The Secure Hash Algorithm 3 (SHA-3) family uses a public Keccak permutation with specified parameters and encodings. SHA3-256 has $b = 1600$, $r = 1088$, $c = 512$, and an output of 256 bits. Its generic output-collision scale is consequently 2^{128}, despite the larger capacity. The precise standard includes domain separation and bit conventions; the structural algorithm above explains the flow rather than replacing that specification.

4.10 Combining confidentiality and authentication

4.10.1 Three arrangements, three information flows

Use independent keys k_E for encryption and k_M for the MAC. The symbol c denotes the *complete* encryption output, including any IV, nonce, and required decoding fields. A nonce is a value whose permitted use, typically nonrepetition under a key, is specified by the encryption scheme.

In Figure 4.9, the ENC boxes perform encryption and the MAC boxes generate authentication tags. Trace the input of the MAC box in each arrangement: it receives the plaintext on the left and in the middle, but the ciphertext on the right. This determines both what the transmitted

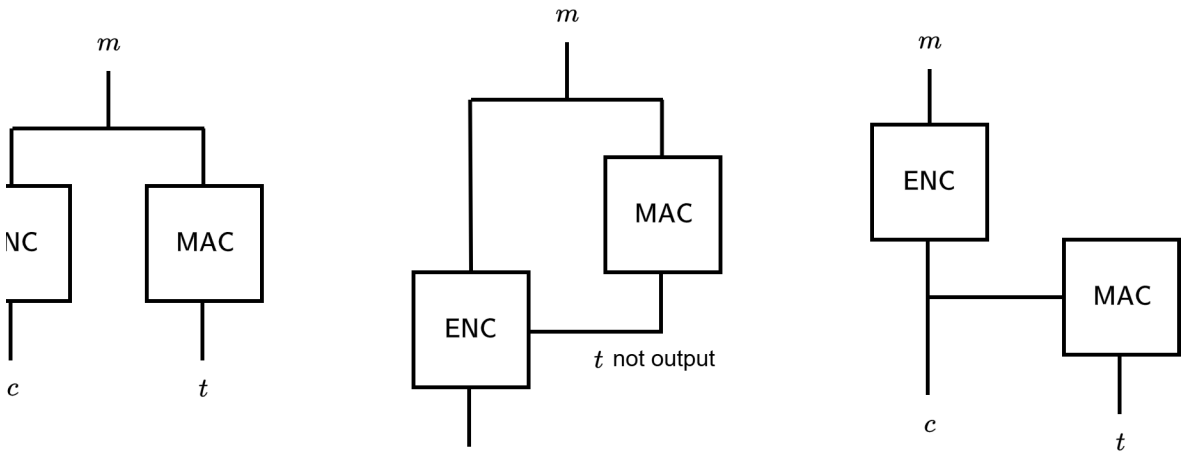


Figure 4.9: Left: encrypt and authenticate the plaintext separately. Middle: authenticate before encrypting. Right: authenticate the complete ciphertext.

tag might disclose and what Bob must do before checking it. The diagrams show sender-side generation; the receiver must process the components in the corresponding reverse order.

Definition 4.16 (Encrypt-and-MAC)**.** Compute $c = \mathrm{Enc}_{k_E}(m)$ and $t = \mathrm{Mac}_{k_M}(m)$ independently, and transmit (c, t). The plaintext tag is exposed alongside the ciphertext.

MAC unforgeability places no general restriction on message information revealed by a tag. To see the problem precisely, start with a secure MAC $T_k(m)$ and define a new tag $T_k(m)\|m(0)$, where $m(0)$ now denotes the first message bit. Verification checks both components. Forgery remains difficult because the attacker still needs $T_k(m)$, but the tag openly reveals that first bit. A CPA adversary chooses two equal-length messages with different first bits and reads which was encrypted from the exposed tag. Thus secure encryption and an unforgeable MAC do not generically make this arrangement confidential.

Definition 4.17 (MAC-then-encrypt)**.** Compute $t = \mathrm{Mac}_{k_M}(m)$, then transmit $c = \mathrm{Enc}_{k_E}(m\|t)$ using an unambiguous encoding. The tag is inside the encryption.

Bob must first decrypt to obtain the message and tag, then check the tag. An attacker may therefore cause decryption or padding processing before authenticity is known. If the receiver exposes whether padding failed or tag verification failed, the first failure can become the padding oracle of Chapter 2. This arrangement can be secure with additional construction-specific assumptions, but CPA encryption plus an arbitrary secure MAC does not by itself give the desired generic guarantee. Processing decrypted bytes internally also does not mean they must be released to the application: no unauthenticated plaintext should be acted upon.

Definition 4.18 (Encrypt-then-MAC)**.** Compute $c = \mathrm{Enc}_{k_E}(m)$ and a tag over the complete encoded ciphertext, $t = \mathrm{Mac}_{k_M}(c)$, and transmit (c, t). The receiver checks the tag before invoking decryption.

Here Bob can reject an attacker-modified ciphertext without decrypting it. With independent keys, suitable encodings, CPA-secure encryption, and a strongly unforgeable MAC, the standard composition can achieve indistinguishability under chosen-ciphertext attack (IND-CCA). This stronger experiment permits decryption queries except for the exact challenge object. Strong unforgeability prevents turning that forbidden object into a new accepted pair merely by changing its tag. A secure canonical deterministic MAC supplies the required strong property.

Algorithm 4.5 Encrypt-then-MAC with authenticated public context

Require: Independent keys k_E, k_M; message m and associated data a
Ensure: Ciphertext/tag generation, or verified plaintext recovery with failure $\perp$

function PROTECT(k_E, k_M, m, a)
  $c \leftarrow \mathrm{Enc}_{k_E}(m)$ using its prescribed randomness or nonce rules
  $u \leftarrow \mathrm{encode}(a, c)$ with unambiguous field boundaries
  $t \leftarrow \mathrm{Mac}_{k_M}(u)$
  return (c, t)
end function
function OPEN(k_E, k_M, c, t, a)
  if $\mathrm{Ver}_{k_M}(\mathrm{encode}(a, c), t) = \perp$ **then**
    return $\perp$ ▷ No decryption or padding check before this point
  end if
  $m \leftarrow \mathrm{Dec}_{k_E}(c)$
  return m or $\perp$ according to decryption validity
end function

The public context a, called associated data, can contain a device identity or message type. It is authenticated without being encrypted. Field boundaries matter: simple concatenation could make two different field pairs encode identically. Authenticating an IV together with the ciphertext also matters: leaving it outside the MAC would allow the first plaintext block of CBC to be changed without changing the authenticated ciphertext blocks. None of this automatically enforces freshness; replay checks still belong to the protocol.

4.11 Concrete constructions and how to recognise their roles

4.11.1 Counter mode provides encryption

Counter (CTR) mode generates a keystream from distinct counter blocks ctr_i:

$$z_i = \mathrm{AES}_k(\mathrm{ctr}_i), \qquad c_i = m_i \oplus z_i.$$

Counter blocks commonly encode a per-message nonce and a block index. They must not repeat under the same key, including across messages or after a counter wraps. Each z_i depends on its counter rather than the previous ciphertext, so blocks can be computed in parallel. Reusing a counter repeats a pad and exposes the XOR of overlapping plaintexts. CTR provides confidentiality under its assumptions, not authentication.

4.11.2 CMAC provides block-cipher-based authentication

Cipher-based MAC (CMAC) is a standardised CBC-based design using one master key and derived final-block masks. For an n-bit block cipher, two n-bit masks K_1, K_2 are obtained by the specified finite-field doubling of $L = P_k(0^n)$. For 128-bit blocks, doubling shifts left one bit and, if the old leading bit was 1, XORs the constant $0^{120}\|\mathtt{87}$, where `87` is hexadecimal. Set $K_1 = \mathrm{dbl}(L)$ and $K_2 = \mathrm{dbl}(K_1)$.

If the final message block is complete and the message is nonempty, XOR K_1 into that block. Otherwise pad its final fragment with a 1 marker and zeroes, then XOR K_2 into it; the empty message uses this second case. Process the resulting blocks with the zero-initialised CBC chain and output the required prefix of the final state as the tag.

Algorithm 4.6 CMAC final-block handling and chaining

Require: Master key k, message m, block length n, tag length $1 \le \tau \le n$
Ensure: A τ-bit tag using the specified CMAC doubling rule
1: $L \leftarrow P_k(0^n)$; $K_1 \leftarrow \mathrm{dbl}(L)$; $K_2 \leftarrow \mathrm{dbl}(K_1)$
2: **if** m is nonempty and has a complete final block **then**
3: Split into blocks $M(0), \ldots, M(l-1)$
4: $M(l-1) \leftarrow M(l-1) \oplus K_1$
5: **else**
6: Split into complete blocks and a final fragment, allowing an empty fragment
7: Pad the fragment with $1 \| 0 \cdots 0$ to length n and XOR K_2 into it
8: Call the resulting complete sequence $M(0), \ldots, M(l-1)$
9: **end if**
10: $h \leftarrow 0^n$
11: **for** $i = 0, \ldots, l-1$ **do**
12: $h \leftarrow P_k(h \oplus M(i))$
13: **end for**
14: **return** the first τ bits of h

The two masks distinguish a complete final block from a padded fragment. They are derived from the key according to the construction, rather than chosen as arbitrary public constants. This supplies finalisation without storing two independent master keys. It should not be confused with the earlier two-independent-key variant, whose last permutation uses a different key directly.

4.11.3 HMAC provides hash-based authentication

Hash-based MAC (HMAC) uses a cryptographic hash H and an appropriately normalised secret key block K:

$$\mathrm{HMAC}_k(m) = H\big((K \oplus \mathrm{opad}) \| H((K \oplus \mathrm{ipad}) \| m)\big).$$

The normalisation hashes an overlong key if necessary and zero-fills to the hash's input block size. The fixed inner and outer pad strings, ipad and opad, repeat bytes `36` and `5C`, respectively, in hexadecimal.

Read the formula from inside outward. First mix the key block with the inner pad and hash it together with the message. Then mix the key block with the outer pad and hash that prefix together with the *inner digest*. The exposed tag is the outer result, not the internal hash state for a raw secret-prefix message. HMAC is a specific analysed construction; substituting $H(k\|m)$ merely because it contains a secret key does not inherit the same guarantee. Its security also requires suitable properties of the underlying hash, not just the definition of collision resistance alone.

4.11.4 Authenticated-encryption schemes combine the roles

Authenticated encryption with associated data (AEAD) provides confidentiality and ciphertext integrity together, while authenticating designated readable context. AES in Galois/Counter Mode (AES-GCM) combines counter-mode encryption with a keyed polynomial authentication mechanism. Ascon-AEAD128 is a permutation-based construction designed for constrained devices. Both are specified complete algorithms rather than instructions to combine arbitrary encryption and MAC functions.

Correct nonce management, tag length, data-volume limits, context encoding, and rejection of unauthenticated plaintext remain part of their use. An AEAD interface makes the desired

combination explicit; it does not by itself stop replay or repair a repeated nonce. The main distinction to retain is which task each component serves: encryption hides data, a MAC detects unauthorised message creation or modification under the chosen model, and the composition must preserve both guarantees.

4.12 Review questions

Exercise 4.19 (Explain what changes at each boundary)**.**

1. Construct two different two-block messages with the same XOR checksum. Why does a random-fault argument miss this attack?
2. Compute the guessing limit for a 5-bit tag and explain how multiple verification attempts enter the bound.
3. Distinguish a new message, a new accepted pair, and a replay. Give a MAC that is ordinarily but not strongly unforgeable.
4. Why does unique canonical verification make ordinary and strong unforgeability coincide for a secure deterministic MAC?
5. Run raw CBC-MAC on two blocks, then derive the modified first appended block in the extension forgery. Why is plain concatenation not the same formula?
6. Explain why changing the final permutation key blocks that splice, and why prefixing the true message length changes the first step of a longer forgery.
7. Distinguish injective padding from a prefix-free complete encoding. When can a true-length prefix remove an ambiguity that zero-padding alone would retain?
8. Identify the primitive, collision, and guessing terms in a resource bound. What information about lengths and queries is needed to interpret $q^2/2^n$?
9. Why does authenticating a counter require receiver state? What extra information can an early-exit byte comparison reveal?
10. Define collision resistance with distinct messages and derive the birthday scale. Why is $(m, H(m))$ not message authentication on its own?
11. Trace one block-cipher compression step and explain why the message-selected key changes the applicable security assumption.
12. Follow two absorption blocks and one squeezing block through the sponge. What do the rate, capacity, and output length each control?
13. Give an unforgeable tag that leaks a plaintext bit. Use it to break the generic confidentiality claim for encrypt-and-MAC.
14. For the other two compositions, state what the receiver checks before decrypting and which errors could become observable.
15. Explain the roles and boundary conditions of CTR, CMAC, HMAC, and AEAD. Why is CMAC's derived-mask finalisation different from two independent final-round keys?

Appendix A

Finite fields and the algebra behind AES diffusion

AES treats a byte both as eight bits and as one element of a field with 256 elements. The bits specify a representation; the field specifies how addition, multiplication, and inversion work. This appendix develops those operations, then explains why a linear mixing step can be invertible even when the larger polynomial structure containing it is not a field. The final connection is between linear codes and the branch number used in Chapter 3.

A.1 Groups, rings, and fields: which operations can be undone?

Definition A.1 (Abelian group)**.** An abelian group $(G, +)$ is a set with an operation closed on G, associative and commutative, with a neutral element 0 and an additive inverse $-a$ for every a. Thus $a + 0 = a$ and $a + (-a) = 0$. The operation need not be ordinary integer addition.

For example, integers modulo n form an additive group: adding 5 and 4 modulo 7 gives 2, and the additive inverse of 5 is 2. This only concerns addition. It says nothing yet about dividing by a nonzero element.

Definition A.2 (Ring and field)**.** A ring with identity has an abelian additive group, an associative closed multiplication with identity 1, and both distributive laws. Multiplication need not commute in a general ring; it does in a commutative ring. A field is a commutative ring with $0 \neq 1$ in which every nonzero element has a multiplicative inverse.

Example A.3 (Why a finite arithmetic system may fail to be a field)**.** Modulo 7, $3 \cdot 5 = 1$, so 5 is the multiplicative inverse of 3. Every nonzero residue has such an inverse because 7 is prime. Modulo 8, the nonzero elements 2 and 4 multiply to zero. Neither can have an inverse: multiplying $2 \cdot 4 = 0$ by a hypothetical inverse of 2 would give $4 = 0$. A ring may have such zero divisors; a field cannot.

The distinction matters for cryptography because an invertible S-box or mixing layer must be reversible. Working in a field guarantees that a nonzero coefficient can be divided out. Working in a ring requires checking that the particular coefficient or transformation is invertible, rather than assuming all nonzero ones are.

A.2 Finite fields, characteristic, and coordinates

Theorem A.4 (Orders of finite fields)**.** *A finite field exists with q elements exactly when $q = p^m$ for a prime p and a positive integer m. Fields of the same order are isomorphic: a bijection can translate between them while preserving both operations. The notation $\mathbb{F}_q$ or $\mathrm{GF}(q)$ denotes a field of that order.*

The characteristic is the smallest positive number of copies of 1 that sum to zero. In a field it must be prime: if it were ab with a, b smaller positive integers, the two nonzero sums of a and b copies of 1 would multiply to zero. A finite field of characteristic p contains the residues $\mathbb{F}_p$, and is a finite-dimensional vector space over them; a basis of length m gives exactly p^m coordinate choices. This explains the necessary prime-power form. Existence and uniqueness up to isomorphism are the general finite-field theorem, not consequences of counting alone.

Remark A.5 (Same size does not mean the same encoded multiplication)*.* $\mathbb{F}_{256}$ has characteristic 2 and $1 + 1 = 0$. The integer residue ring $\mathbb{Z}/256\mathbb{Z}$ has $1 + 1 = 2$ and contains zero divisors. These cannot be the same arithmetic. Even two isomorphic representations of $\mathbb{F}_{256}$ may assign different bit strings to corresponding elements; one must transform the representation when moving between them.

Definition A.6 (Vector space and basis)**.** A vector space V over a field F has an abelian vector addition and scalar multiplication satisfying

$$a(v + w) = av + aw, \quad (a + b)v = av + bv, \quad (ab)v = a(bv), \quad 1v = v.$$

A basis $\mathcal{B} = (e_1, \ldots, e_m)$ expresses every vector uniquely as $v = \sum_i v_i e_i$ with $v_i \in F$. The column of coefficients is written $[v]_{\mathcal{B}}$.

For a byte viewed over $\mathbb{F}_2$, the eight bits are coordinates and adding coordinates is XOR. Multiplication by a *fixed* field element is linear over $\mathbb{F}_2$, since field multiplication distributes over addition. Multiplication with both operands variable is not a linear map in the two inputs jointly; inversion is nonlinear. These observations distinguish AES's linear mixing from the nonlinear part of its substitution.

Definition A.7 (Linear map)**.** A map $T : V \to W$ over F is linear if $T(v + w) = T(v) + T(w)$ and $T(av) = aT(v)$. Once bases are chosen, a matrix M represents it through $[T(v)]_{\mathcal{C}} = M[v]_{\mathcal{B}}$. Its columns are the coordinates of the images of the input basis vectors.

Thus "linear" always names a scalar field. A byte transformation can be linear over $\mathbb{F}_2$ without being multiplication by a single scalar of $\mathbb{F}_{256}$. For MixColumns, it is convenient to regard four bytes as a vector over $\mathbb{F}_{256}$; expanding each byte into bits also gives a 32-bit binary linear transformation.

A.3 Representing field elements by polynomials

A.3.1 Bit strings as formal coefficients

A polynomial $a(z) = \sum_i a_i z^i$ over F stores coefficients from F. Here z is a formal symbol, not a real number at which the polynomial is evaluated. In $\mathbb{F}_2[z]$, adding coefficients is XOR. Encode the byte $a_7 \cdots a_0$ as

$$a_7 z^7 + a_6 z^6 + \cdots + a_1 z + a_0.$$

The rightmost bit is the constant coefficient. Hexadecimal `57`, for example, is 01010111 and represents $z^6 + z^4 + z^2 + z + 1$; `83` represents $z^7 + z + 1$. Their sum is `D4`, since equal powers cancel in pairs:

$$\texttt{57} \oplus \texttt{83} = \texttt{D4}.$$

There are no carries. Polynomial multiplication is different from integer multiplication for the same reason: intermediate coefficients are reduced modulo 2 rather than carried into the next position.

A.3.2 Reduction and why irreducibility matters

Multiplying two polynomials of degree below m can produce degree up to $2m - 2$. To return to m coordinates, choose a monic degree-m polynomial $f(z)$ and replace the product by its remainder on division by f. Two polynomials represent the same element if their difference is divisible by f. The resulting quotient ring is written $F[z]/(f(z))$.

Definition A.8 (Irreducible polynomial)**.** A nonconstant polynomial f over F is irreducible if it cannot be written as a product of two positive-degree polynomials over F. Irreducibility depends on the coefficient field.

Theorem A.9 (Polynomial field construction)**.** *If f is irreducible of degree m over $\mathbb{F}_p$, then $\mathbb{F}_p[z]/(f)$ is a field with p^m elements. If f factors into positive-degree factors, its quotient has zero divisors and is not a field.*

Proof. Remainders have degree below m, so there are p^m coordinate choices. For any nonzero remainder a, irreducibility makes $\gcd(a, f) = 1$. The polynomial extended Euclidean algorithm gives $ua + vf = 1$. Reducing modulo f yields $ua = 1$, so u represents an inverse. Conversely, a factorisation $f = gh$ makes the nonzero residue classes of g and h multiply to zero. □

Example A.10 (A field with four elements)**.** Take $f(z) = z^2 + z + 1$ over $\mathbb{F}_2$. It has no root in $\{0, 1\}$, so this quadratic is irreducible. Let α be the class of z. The four elements are $0, 1, \alpha, \alpha+1$, and reduction gives $\alpha^2 = \alpha + 1$. Consequently

$$\alpha(\alpha + 1) = \alpha^2 + \alpha = 1,$$

so $\alpha^{-1} = \alpha + 1$. As two-bit encodings, $10 \cdot 11 = 01$. Ordinary integer multiplication modulo 4 would instead give $2 \cdot 3 = 2$. The bit representation does not determine the operations without the chosen polynomial.

A.4 AES byte arithmetic, calculated step by step

A.4.1 The reduction polynomial and multiplication by two

AES uses

$$f(z) = z^8 + z^4 + z^3 + z + 1,$$

an irreducible polynomial over $\mathbb{F}_2$. Multiplying a byte polynomial by z shifts its bits left. If its old leading bit was 1, the resulting z^8 term is replaced by z^4+z^3+z+1, encoded as hexadecimal `1B`. The operation is commonly called xtime, multiplication by the field element `02`.

For `57`, the leading bit is zero, so xtime(`57`) = `AE`. For `AE`, the shift would exceed eight bits; keeping the low eight gives `5C`, then reduction gives $\texttt{5C} \oplus \texttt{1B} = \texttt{47}$. These are polynomial reductions, not integer overflow rules being used as a substitute for field arithmetic.

Algorithm A.1 AES byte multiplication using shifts and XORs

Require: Bytes a, b; byte values are written in hexadecimal where indicated
Ensure: Their product in $\mathbb{F}_2[z]/(z^8 + z^4 + z^3 + z + 1)$

function XTIME(a)
    $h \leftarrow$ the old leading bit of a
    $a \leftarrow$ the low eight bits of a shifted left by one
    if $h = 1$ **then**
        $a \leftarrow a \oplus \texttt{1B}$
    end if
    return a
end function
function FIELD-MULTIPLY(a, b)
    $r \leftarrow 0$
    for $i = 0, \ldots, 7$ **do**
        if the low bit of b is 1 **then**
            $r \leftarrow r \oplus a$
        end if
        $a \leftarrow$ xtime(a); shift b right by one
    end for
    return r
end function

A.4.2 A complete product

Calculate $\texttt{57} \cdot \texttt{83}$. Since 83 represents $z^7 + z + 1$, take the XOR of 57, its product by z, and its product by z^7. Repeated xtime gives

$$\texttt{57},\ \texttt{AE},\ \texttt{47},\ \texttt{8E},\ \texttt{07},\ \texttt{0E},\ \texttt{1C},\ \texttt{38}.$$

Select positions 0, 1 and 7: $\texttt{57} \oplus \texttt{AE} \oplus \texttt{38} = \texttt{C1}$. Equivalently, multiply without reducing, obtaining the polynomial encoded by 2B79, then eliminate its leading terms:

$$\texttt{2B79} \oplus (\texttt{11B} \ll 5) = \texttt{0819}, \qquad \texttt{0819} \oplus (\texttt{11B} \ll 3) = \texttt{00C1}.$$

The notation $\ll j$ shifts the polynomial coefficient encoding by j positions, corresponding to multiplication by z^j. Here 11B encodes all nine coefficients of f; its leading bit is retained during long reduction.

A.4.3 Computing inverses

The extended Euclidean algorithm maintains polynomial combinations of a and f. If the final greatest common divisor is 1, the coefficient of a is its inverse modulo f.

Algorithm A.2 Polynomial inversion over a binary field

Require: A nonzero polynomial a of degree below m, irreducible degree-m f
Ensure: a^{-1} modulo f; all coefficient operations are in $\mathbb{F}_2$

$(r_0, r_1) \leftarrow (f, a)$; $(t_0, t_1) \leftarrow (0, 1)$
while $r_1 \neq 0$ **do**
    Divide polynomials: $r_0 = q r_1 + r_2$ with $\deg r_2 < \deg r_1$
    $(r_0, r_1) \leftarrow (r_1, r_2)$
    $(t_0, t_1) \leftarrow (t_1, t_0 \oplus q t_1)$ ▷ Simultaneous update
end while
return the remainder of t_0 modulo f ▷ Here $r_0 = 1$

For example, AES byte values 53 and CA multiply to 01. No nonzero byte lacks an inverse. Zero does: AES's S-box uses a separately defined value at zero when implementing its inversion step, followed by an affine binary transformation. Multiplication by a fixed byte stays linear; taking an input's multiplicative inverse supplies the essential nonlinear operation.

A.5 Four-byte columns: an invertible map in a ring

A.5.1 A second polynomial variable

Now treat each byte as a coefficient in $F = \mathbb{F}_{256}$ and a column as

$$a(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3.$$

The variable t records *byte positions*; it is different from z, which recorded bits inside a byte. Multiply these column polynomials modulo $t^4 + 1$. The coefficient products use the AES byte field, and the outer reduction identifies t^4 with 1 because the characteristic is 2.

Unlike the byte modulus, this outer polynomial is reducible:

$$t^4 + 1 = (t + 1)^4.$$

Thus $F[t]/(t^4+1)$ is a ring, not a field. For instance, the nonzero classes of $t+1$ and t^3+t^2+t+1 multiply to zero. Nevertheless a particular multiplier $b(t)$ is invertible if it is coprime to $t^4 + 1$. Here the only irreducible factor is $t + 1$, so this is equivalent to $b(1) \neq 0$.

A.5.2 Deriving the MixColumns matrix

AES selects the fixed polynomial

$$b(t) = \mathtt{02} + \mathtt{01}t + \mathtt{01}t^2 + \mathtt{03}t^3.$$

Its value at 1 is $\mathtt{02} \oplus \mathtt{01} \oplus \mathtt{01} \oplus \mathtt{03} = \mathtt{01}$, so the multiplier is invertible in this ring. Expanding $d(t) = b(t)a(t)$ modulo $t^4 + 1$ gives

$$\begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} \mathtt{02} & \mathtt{03} & \mathtt{01} & \mathtt{01} \\ \mathtt{01} & \mathtt{02} & \mathtt{03} & \mathtt{01} \\ \mathtt{01} & \mathtt{01} & \mathtt{02} & \mathtt{03} \\ \mathtt{03} & \mathtt{01} & \mathtt{01} & \mathtt{02} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}.$$

To obtain the first row, collect terms of degree 0 or 4: they give $b_0a_0 + b_3a_1 + b_2a_2 + b_1a_3$. The terms of degree 4 wrap to degree 0, explaining the cyclic structure rather than requiring the matrix to be memorised.

Example A.11 (One actual column)**.** Consider the following input and output columns:

$$(\mathtt{D4}, \mathtt{BF}, \mathtt{5D}, \mathtt{30})^T \longmapsto (\mathtt{04}, \mathtt{66}, \mathtt{81}, \mathtt{E5})^T.$$

For the first output byte,

$$d_0 = (\mathtt{02} \cdot \mathtt{D4}) \oplus (\mathtt{03} \cdot \mathtt{BF}) \oplus \mathtt{5D} \oplus \mathtt{30} = \mathtt{B3} \oplus \mathtt{DA} \oplus \mathtt{5D} \oplus \mathtt{30} = \mathtt{04}.$$

Multiplication by 03 is multiplication by 02 followed by XOR with the original byte, since $\mathtt{03} = z+1$. This explains how the linear mixing reduces to the byte operations already calculated.

The inverse column multiplier is $\mathtt{0E} + \mathtt{09}t + \mathtt{0D}t^2 + \mathtt{0B}t^3$. Multiplying it by $b(t)$ modulo $t^4 + 1$ gives 1. Invertibility ensures decryption is possible; the stronger question of how many byte positions must become active is answered by branch number.

A.6 Linear codes and the meaning of an optimal branch number

A.6.1 Weight, distance, and matrix descriptions

Definition A.12 (Hamming weight and distance)**.** The Hamming weight $\mathrm{wt}(v)$ counts the nonzero *coordinates* of a vector. The distance is $d_H(v, w) = \mathrm{wt}(v - w)$. Over a binary field of characteristic 2, subtraction equals addition. A byte value such as `FF` has weight 1 as a one-coordinate byte vector, but weight 8 as its eight-bit coordinate vector. The coordinate unit must therefore be stated.

Definition A.13 (Linear code)**.** A linear $[N, k, d]$ code over F is a k-dimensional subspace of F^N whose smallest distance between distinct codewords is d. By linearity, this equals the minimum weight of a nonzero codeword. A full-row-rank generator matrix G has dimensions $k \times N$ and maps row vectors u to codewords uG.

A full-rank parity-check matrix H has dimensions $(N - k) \times N$ and characterises codewords by $Hv^T = 0$. Its number of columns must be N to multiply a length-N coordinate column. The rows of H span the dual code $C^\perp$, the vectors orthogonal to every codeword; $HG^T = 0$. For a systematic generator $G = [I_k \ A]$, one can take $H = [-A^T \ I_{N-k}]$. In characteristic 2, the minus sign is the same as plus.

Example A.14 (A parity check is an equation, not an authenticator)**.** Over $\mathbb{F}_2$, let

$$G = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \qquad H = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}.$$

The codewords are $000, 101, 011, 110$, all with even parity. A one-bit error leaves the code and is detected; a change from one valid codeword to another passes the test. This is a public linear consistency check, not a MAC. The coding theory is useful here to analyse diffusion, rather than to replace cryptographic authentication.

A.6.2 Singleton bound and MDS codes

Theorem A.15 (Singleton bound)**.** *For a linear $[N, k, d]$ code over a finite field, $d \le N - k + 1$. A code meeting equality is called maximum distance separable (MDS).*

Proof. Delete any $d - 1$ coordinate positions. Distinct codewords cannot become identical: otherwise they would have differed in at most $d-1$ positions. The remaining $N-d+1$ coordinates must therefore distinguish all $|F|^k$ codewords, so $|F|^k \le |F|^{N-d+1}$. Taking exponents gives the bound. □

A code has minimum distance d exactly when every set of $d - 1$ columns of its parity-check matrix is independent and some set of d columns is dependent. A dependent set gives a nonzero codeword supported in those positions; a minimum-weight codeword supplies such a dependence. For a systematic generator $[I_k \ A]$, the MDS property is equivalent to every square submatrix of A being nonsingular. This criterion concerns *all* square sizes, not just whether the full matrix is invertible.

A.6.3 The graph of a mixing layer is a code

Let M be an invertible linear map on s field coordinates. Form the code

$$C_M = \{(a, Ma) : a \in F^s\} \subseteq F^{2s}.$$

Its dimension is s, and the weight of a codeword is $\mathrm{wt}(a) + \mathrm{wt}(Ma)$. Its minimum distance is therefore exactly the branch number $B(M)$. The Singleton bound immediately gives

$$B(M) \leq 2s - s + 1 = s + 1.$$

In row-vector notation a generator is $[I_s \; M^T]$, accounting for the transpose when comparing with the column-vector mixing formula.

AES's four-byte MixColumns matrix has branch number 5, meeting this bound for $s = 4$. This is the precise sense in which its diffusion is optimal at the column level: a nonzero one-byte input difference must yield four nonzero output bytes; an input involving two bytes must yield at least three, and so on. Invertibility alone would guarantee only that the output is nonzero, allowing a single active byte to remain a single active byte. The MDS property supplies the stronger spreading constraint used by the wide-trail argument. Linear cryptanalysis uses the corresponding transpose mask propagation described in Chapter 3; the same AES matrix property provides the matching activity bound.

A.7 Review questions

Exercise A.16 (Calculate the operation before invoking its name)**.**

1. Which inverse is guaranteed by a group, and what extra inverse is required by a field? Why is arithmetic modulo 8 not a field?
2. Why does a finite field have prime-power size? Explain why $\mathbb{F}_{256}$ differs from integer residues modulo 256.
3. Translate `57` to a polynomial, add `83`, and explain why no binary carry occurs.
4. Construct the four-element field using $z^2 + z + 1$ and find the inverse of z in that representation.
5. Derive the `1B` reduction in xtime, then reproduce $\texttt{57} \cdot \texttt{83} = \texttt{C1}$ by two methods.
6. State the invariant behind the extended Euclidean inverse and identify why the input cannot be zero.
7. Why does $t^4 + 1$ give a ring rather than a field, and why can AES's fixed column multiplier still be invertible?
8. Derive the first MixColumns row by collecting wrapped polynomial terms. Recompute its action on the displayed example column.
9. Give the dimensions of G and H and verify $HG^T = 0$ for the three-bit example.
10. Prove the Singleton bound and explain why $B(M)$ is the distance of a $[2s, s]$ code. What does branch number 5 force when the input has one active byte?